VITCC Library Management System
Complete Codebase Reference

Generated: 2026-10-06 00:06 | Files: 24

Table of Contents

01. app.py ? Python - Core Application
02. ddl.sql ? SQL - Schema DDL
03. plsql.sql ? SQL - PL/SQL Procedures and Triggers
04. requirements.txt ? Text - Python Dependencies
05. seed_all_data.py ? Python - Database Seeder
06. test_user_requirements.py ? Python - Test Suite
07. templates/base.html ? HTML - Base Layout
08. templates/login.html ? HTML - Login Page
09. templates/register.html ? HTML - Register Page
10. templates/dashboard.html ? HTML - Unified Dashboard
11. templates/catalog.html ? HTML - Book Catalog
12. templates/category_books.html ? HTML - Category Books
13. templates/loan_requests.html ? HTML - Loan Requests Queue
14. templates/issue_loan.html ? HTML - Issue Loan
15. templates/my_fines.html ? HTML - My Fines
16. templates/manage_users.html ? HTML - Manage Users
17. templates/add_book.html ? HTML - Add Book
18. templates/change_password.html ? HTML - Change Password
19. templates/unauthorized.html ? HTML - Unauthorized
20. templates/books.html ? HTML - Books Redirect
21. templates/dashboard_admin.html ? HTML - Dashboard Admin
22. templates/dashboard_librarian.html ? HTML - Dashboard Librarian
23. templates/dashboard_staff.html ? HTML - Dashboard Staff
24. templates/dashboard_student.html ? HTML - Dashboard Student

app.py

Python - Core Application

```python
"""
Enterprise Library Management System - Core Application
Engineered with Flask, Oracle Database (python-oracledb), and Role-Based Access Control (RBAC).
"""

from datetime import datetime
from functools import wraps
import re
import oracledb
from flask import Flask, flash, redirect, render_template, request, session, url_for
from flask_session import Session
from werkzeug.security import check_password_hash, generate_password_hash

# ==============================================================================
# Application Initialization & Configuration
# ==============================================================================
app = Flask(__name__)

# Application Secret Key
app.config["SECRET_KEY"] = "enterprise-library-secret-key-change-in-production"

# Flask-Session Configuration (Server-side filesystem sessions)
app.config["SESSION_PERMANENT"] = False
app.config["SESSION_TYPE"] = "filesystem"
Session(app)


# ==============================================================================
# 1. Database Connection & Dict Cursor Management
# ==============================================================================
class DictCursor:
    """Wrapper around oracledb cursor to automatically return rows as lowercase dictionaries."""
    def __init__(self, cursor):
        self._cursor = cursor

    def execute(self, *args, **kwargs):
        res = self._cursor.execute(*args, **kwargs)
        if self._cursor.description:
            columns = [col[0].lower() for col in self._cursor.description]
            self._cursor.rowfactory = lambda *row_args: dict(zip(columns, row_args))
        return res

    def __getattr__(self, name):
        return getattr(self._cursor, name)

    def __setattr__(self, name, value):
        if name == "_cursor":
            super().__setattr__(name, value)
        elif name == "rowfactory":
            # Handled automatically after execute when description is populated
            pass
        else:
            setattr(self._cursor, name, value)


def get_db_connection():
    """Establishes and returns a direct connection to the Oracle 21c Database with DictCursor support."""
    conn = oracledb.connect(
        user="system",
        password="root",
        dsn="localhost:1521/XEPDB1",
    )
    orig_cursor = conn.cursor
    conn.cursor = lambda *args, **kwargs: DictCursor(orig_cursor(*args, **kwargs))
    return conn


def make_dict_factory(cursor):
    """Safely return None; DictCursor automatically manages rowfactory when description is ready."""
```

```
    return None


# ==============================================================================
# 2. Authentication & Authorization Decorators (RBAC)
# ==============================================================================
def login_required(f):
    """Ensures user is authenticated before accessing the view."""

    @wraps(f)
    def decorated_function(*args, **kwargs):
        if "user_id" not in session:
            flash("Please log in to access this page.", "warning")
            return redirect(url_for("login"))
        return f(*args, **kwargs)

    return decorated_function


def role_required(allowed_roles):
    """Restricts view access to specific user roles."""

    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if "user_id" not in session:
                flash("Please log in to proceed.", "warning")
                return redirect(url_for("login"))

            user_role = session.get("role")
            if user_role not in allowed_roles:
                flash("Access Denied: You do not have permissions for this action.", "danger")
                return render_template("unauthorized.html", user_role=user_role), 403

            return f(*args, **kwargs)

        return decorated_function

    return decorator


def validate_password_strength(password):
    """
    Validates that password meets all specified security criteria:
    - Minimum 5 to maximum 13 characters
    - At least 1 uppercase letter (A-Z)
    - At least 1 lowercase letter (a-z)
    - At least 1 number (0-9)
    - At least 1 special character
    Returns (is_valid: bool, error_message: str or None)
    """
    if not password:
        return False, "Password cannot be empty."
    if len(password) < 5 or len(password) > 13:
        return False, "Password must be between 5 and 13 characters in length."
    if not re.search(r'[A-Z]', password):
        return False, "Password must contain at least 1 uppercase letter (A-Z)."
    if not re.search(r'[a-z]', password):
        return False, "Password must contain at least 1 lowercase letter (a-z)."
    if not re.search(r'[0-9]', password):
        return False, "Password must contain at least 1 number (0-9)."
    if not re.search(r'[^a-zA-Z0-9]', password):
        return False, "Password must contain at least 1 special character (e.g. !@#$%^&*)."
    return True, None


def validate_email_format(email):
    """
    Validates that email compulsory contains '@' and '.'.
    Returns (is_valid: bool, error_message: str or None)
    """
    if not email:
```

```
        return False, "Email address cannot be empty."
    email = email.strip()
    if "@" not in email or "." not in email:
        return False, "Email address must contain both '@' and '.' characters (e.g. user@domain.com)."
    parts = email.split("@")
    if len(parts) != 2 or not parts[0] or not parts[1] or "." not in parts[1]:
        return False, "Please enter a valid email address with '@' and '.' (e.g. user@domain.com)."
    return True, None


# ============================================================================
# 3. Routes & Business Logic
# ============================================================================

# ----------------------------------------------------------------------------
# Route: / & /login (Handles Authentication)
# ----------------------------------------------------------------------------
@app.route("/", methods=["GET", "POST"])
@app.route("/login", methods=["GET", "POST"])
def login():
    # Redirect already authenticated users straight to the dashboard
    if "user_id" in session:
        return redirect(url_for("dashboard"))

    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")

        if not username or not password:
            flash("Username and Password are required.", "danger")
            return render_template("login.html")

        conn = None
        cursor = None
        try:
            conn = get_db_connection()
            cursor = conn.cursor()
            cursor.rowfactory = make_dict_factory(cursor)

            query = """
                SELECT User_ID, Username, Password_Hash, Role, Account_Status
                FROM Users
                WHERE LOWER(Username) = LOWER(:username)
            """
            cursor.execute(query, {"username": username})
            user = cursor.fetchone()

            if user and check_password_hash(user["password_hash"], password):
                if user["account_status"] != "Active":
                    flash("Account is suspended. Please contact the library administrator.", "danger")
                    return render_template("login.html")

                # Set session details
                session["user_id"] = user["user_id"]
                session["username"] = user["username"]
                session["role"] = user["role"]

                flash(f"Welcome back, {user['username']}!", "success")
                return redirect(url_for("dashboard"))
            else:
                flash("Invalid username or password.", "danger")

        except Exception as e:
            flash(f"Database error during authentication: {str(e)}", "danger")
        finally:
            if cursor:
                cursor.close()
            if conn:
                conn.close()

    return render_template("login.html")

```

```python

# ------------------------------------------------------------------------
# Route: /register (Self-Registration for Students & Staff)
# ------------------------------------------------------------------------
@app.route("/register", methods=["GET", "POST"])
def register():
    # Redirect authenticated users straight to their dashboard
    if "user_id" in session:
        return redirect(url_for("dashboard"))

    if request.method == "POST":
        username = request.form.get("username", "").strip()
        email = request.form.get("email", "").strip()
        password = request.form.get("password", "")
        role = request.form.get("role", "").strip()

        # Security: Self-registration is strictly restricted to Student or Staff
        if role not in ["Student", "Staff"]:
            flash("Security Violation: Self-registration only permits 'Student' or 'Staff' accounts.", "danger")
            return render_template("register.html")

        if not username or not email or not password:
            flash("All fields (Username, Email, Password, Role) are mandatory.", "warning")
            return render_template("register.html")

        # Validate Email Format (Compulsory @ and .)
        is_email_valid, email_err = validate_email_format(email)
        if not is_email_valid:
            flash(f"Invalid Email: {email_err}", "warning")
            return render_template("register.html")

        # Validate Password Strength (5-13 chars, 1 uppercase, 1 lowercase, 1 number, 1 special char)
        is_pass_valid, pass_err = validate_password_strength(password)
        if not is_pass_valid:
            flash(f"Invalid Password: {pass_err}", "warning")
            return render_template("register.html")

        conn = None
        cursor = None
        try:
            conn = get_db_connection()
            cursor = conn.cursor()

            # Verify uniqueness of Username and Email
            cursor.execute("""
                SELECT User_ID FROM Users
                WHERE LOWER(Username) = LOWER(:u) OR LOWER(Email) = LOWER(:e)
            """, {"u": username, "e": email})

            if cursor.fetchone():
                flash("Username or Email is already registered. Please sign in or use different details.", "warning")
                return render_template("register.html")

            # Hash password securely using werkzeug
            pw_hash = generate_password_hash(password)

            cursor.execute("""
                INSERT INTO Users (Username, Password_Hash, Email, Role, Account_Status)
                VALUES (:username, :pw_hash, :email, :role, 'Active')
            """, {
                "username": username,
                "pw_hash": pw_hash,
                "email": email,
                "role": role
            })
            conn.commit()

            flash(f"Account for '{username}' ({role}) created successfully! Please sign in.", "success")
            return redirect(url_for("login"))

        except Exception as e:
            if conn:
```

```
                conn.rollback()
            flash(f"Registration failed: {str(e)}", "danger")
        finally:
            if cursor:
                cursor.close()
            if conn:
                conn.close()

    return render_template("register.html")


# ------------------------------------------------------------------------
# Route: /logout (Clears Session)
# ------------------------------------------------------------------------
@app.route("/logout")
def logout():
    session.clear()
    flash("You have been successfully logged out.", "info")
    return redirect(url_for("login"))


# ------------------------------------------------------------------------
# Route: /dashboard (Role-Specific Workspaces)
# ------------------------------------------------------------------------
@app.route("/dashboard")
@login_required
def dashboard():
    role = session.get("role")
    user_id = session.get("user_id")

    conn = None
    cursor = None
    data = {}

    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.rowfactory = make_dict_factory(cursor)

        if role in ["Admin", "Librarian"]:
            cursor.execute("SELECT COUNT(*) AS total_books FROM Books")
            data["total_books"] = cursor.fetchone()["total_books"]

            cursor.execute("SELECT COUNT(*) AS active_loans FROM Loans WHERE Return_Date IS NULL")
            data["active_loans"] = cursor.fetchone()["active_loans"]

            cursor.execute("""
                SELECT COUNT(*) AS overdue_loans
                FROM Loans
                WHERE Return_Date IS NULL AND Due_Date < SYSDATE
            """)
            data["overdue_loans"] = cursor.fetchone()["overdue_loans"]

            cursor.execute("SELECT NVL(SUM(Fine_Amount), 0) AS unpaid_fines FROM Fines WHERE Payment_Status = 'Unpaid'")
            data["unpaid_fines"] = cursor.fetchone()["unpaid_fines"]

            cursor.execute("SELECT COUNT(*) AS pending_requests FROM Loan_Requests WHERE Status = 'Pending'")
            data["pending_requests"] = cursor.fetchone()["pending_requests"]

            # Monitor 1: Who and all are currently loaning books
            cursor.execute("""
                SELECT l.Loan_ID, u.Username AS borrower_name, u.Email AS borrower_email, u.Role AS borrower_role,
                       b.Title AS book_title, bc.Copy_ID, l.Issue_Date, l.Due_Date,
                       NVL((SELECT f.Fine_Amount FROM Fines f WHERE f.Loan_ID = l.Loan_ID AND f.Payment_Status = 'Unpaid'),
                FROM Loans l
                JOIN Users u ON l.Borrower_ID = u.User_ID
                JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
                JOIN Books b ON bc.Book_ID = b.Book_ID
                WHERE l.Return_Date IS NULL
                ORDER BY l.Due_Date ASC
            """)
            data["active_loans_list"] = cursor.fetchall()
```

```
358
359             # Monitor 2: Who are overdue
360             cursor.execute("""
361                 SELECT l.Loan_ID, u.Username AS borrower_name, u.Email AS borrower_email, u.Role AS borrower_role,
362                        b.Title AS book_title, bc.Copy_ID, l.Issue_Date, l.Due_Date,
363                        TRUNC(SYSDATE) - TRUNC(l.Due_Date) AS days_overdue
364                 FROM Loans l
365                 JOIN Users u ON l.Borrower_ID = u.User_ID
366                 JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
367                 JOIN Books b ON bc.Book_ID = b.Book_ID
368                 WHERE l.Return_Date IS NULL AND l.Due_Date < SYSDATE
369                 ORDER BY l.Due_Date ASC
370             """)
371             data["overdue_loans_list"] = cursor.fetchall()
372
373             # Monitor 3: Who didn't pay fine
374             cursor.execute("""
375                 SELECT f.Fine_ID, f.Fine_Amount, f.Payment_Status,
376                        u.Username AS borrower_name, u.Email AS borrower_email,
377                        b.Title AS book_title, l.Loan_ID, l.Due_Date
378                 FROM Fines f
379                 JOIN Loans l ON f.Loan_ID = l.Loan_ID
380                 JOIN Users u ON l.Borrower_ID = u.User_ID
381                 JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
382                 JOIN Books b ON bc.Book_ID = b.Book_ID
383                 WHERE f.Payment_Status = 'Unpaid'
384                 ORDER BY f.Fine_Amount DESC
385             """)
386             data["unpaid_fines_list"] = cursor.fetchall()
387
388             if role == "Admin":
389                 cursor.execute("SELECT COUNT(*) AS total_users FROM Users")
390                 data["total_users"] = cursor.fetchone()["total_users"]
391                 return render_template("dashboard_admin.html", data=data)
392             else:
393                 return render_template("dashboard_librarian.html", data=data)
394
395         elif role == "Staff":
396             cursor.execute("""
397                 SELECT l.Loan_ID, b.Title, bc.Copy_ID, l.Issue_Date, l.Due_Date,
398                        NVL((SELECT f.Fine_Amount FROM Fines f WHERE f.Loan_ID = l.Loan_ID AND f.Payment_Status = 'Unpaid'), 
399                 FROM Loans l
400                 JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
401                 JOIN Books b ON bc.Book_ID = b.Book_ID
402                 WHERE l.Borrower_ID = :user_id AND l.Return_Date IS NULL
403                 ORDER BY l.Due_Date ASC
404             """, {"user_id": user_id})
405             data["current_loans"] = cursor.fetchall()
406
407             cursor.execute("""
408                 SELECT NVL(SUM(f.Fine_Amount), 0) AS total_fines
409                 FROM Fines f
410                 JOIN Loans l ON f.Loan_ID = l.Loan_ID
411                 WHERE l.Borrower_ID = :user_id AND f.Payment_Status = 'Unpaid'
412             """, {"user_id": user_id})
413             data["total_fines"] = cursor.fetchone()["total_fines"]
414
415             cursor.execute("""
416                 SELECT lr.Request_ID, b.Title AS book_title, lr.Request_Date, lr.Status
417                 FROM Loan_Requests lr
418                 JOIN Books b ON lr.Book_ID = b.Book_ID
419                 WHERE lr.Borrower_ID = :user_id
420                 ORDER BY lr.Request_ID DESC
421             """, {"user_id": user_id})
422             data["loan_requests"] = cursor.fetchall()
423
424             return render_template("dashboard_staff.html", data=data)
425
426         elif role == "Student":
427             cursor.execute("""
428                 SELECT l.Loan_ID, b.Title, l.Issue_Date, l.Due_Date,
429                        CASE WHEN l.Due_Date < SYSDATE THEN 1 ELSE 0 END AS is_overdue,
```

```
                        NVL((SELECT f.Fine_Amount FROM Fines f WHERE f.Loan_ID = l.Loan_ID AND f.Payment_Status = 'Unpaid'), 0
                    FROM Loans l
                    JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
                    JOIN Books b ON bc.Book_ID = b.Book_ID
                    WHERE l.Borrower_ID = :user_id AND l.Return_Date IS NULL
                    ORDER BY l.Due_Date ASC
                """, {"user_id": user_id})
                data["active_loans"] = cursor.fetchall()

                cursor.execute("""
                    SELECT NVL(SUM(f.Fine_Amount), 0) AS total_fines
                    FROM Fines f
                    JOIN Loans l ON f.Loan_ID = l.Loan_ID
                    WHERE l.Borrower_ID = :user_id AND f.Payment_Status = 'Unpaid'
                """, {"user_id": user_id})
                data["total_fines"] = cursor.fetchone()["total_fines"]

                cursor.execute("""
                    SELECT lr.Request_ID, b.Title AS book_title, lr.Request_Date, lr.Status
                    FROM Loan_Requests lr
                    JOIN Books b ON lr.Book_ID = b.Book_ID
                    WHERE lr.Borrower_ID = :user_id
                    ORDER BY lr.Request_ID DESC
                """, {"user_id": user_id})
                data["loan_requests"] = cursor.fetchall()

                return render_template("dashboard_student.html", data=data)

            else:
                flash("Unknown user role.", "danger")
                return redirect(url_for("logout"))

        except Exception as e:
            flash(f"Error fetching dashboard metrics: {str(e)}", "danger")
            return render_template("dashboard.html", data={})
        finally:
            if cursor:
                cursor.close()
            if conn:
                conn.close()


    # ------------------------------------------------------------------------
    # Route: /books & /catalog (Browse Department Categories Directory)
    # ------------------------------------------------------------------------
    @app.route("/catalog", methods=["GET"])
    @app.route("/books", methods=["GET"])
    @login_required
    def books():
        # If category_id query parameter is provided, redirect to dedicated category page
        category_id_param = request.args.get("category_id", type=int)
        if category_id_param:
            return redirect(url_for("category_books", category_id=category_id_param))

        conn = None
        cursor = None
        categories_list = []

        try:
            conn = get_db_connection()
            cursor = conn.cursor()
            cursor.rowfactory = make_dict_factory(cursor)

            # Fetch all categories with real-time book count
            cursor.execute("""
                SELECT
                    c.Category_ID,
                    c.Name,
                    c.Section_Code,
                    COUNT(b.Book_ID) AS book_count
                FROM Categories c
                LEFT JOIN Books b ON c.Category_ID = b.Category_ID
```

```
            GROUP BY c.Category_ID, c.Name, c.Section_Code
            ORDER BY c.Name ASC
        """)
        categories_list = cursor.fetchall()

    except Exception as e:
        flash(f"Error retrieving category catalog: {str(e)}", "danger")
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    return render_template(
        "catalog.html",
        categories=categories_list
    )


# ------------------------------------------------------------------------
# Route: /category/<int:category_id> (Dedicated Category Books Page with Filters)
# ------------------------------------------------------------------------
@app.route("/category/<int:category_id>", methods=["GET"])
@login_required
def category_books(category_id):
    conn = None
    cursor = None
    category = None
    book_list = []
    user_requests = set()

    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.rowfactory = make_dict_factory(cursor)

        # 1. Fetch category metadata
        cursor.execute("""
            SELECT Category_ID, Name, Section_Code
            FROM Categories
            WHERE Category_ID = :cid
        """, {"cid": category_id})
        category = cursor.fetchone()

        if not category:
            flash("The requested category was not found in the library registry.", "warning")
            return redirect(url_for("books"))

        # 2. Fetch all books belonging to this category
        cursor.execute("""
            SELECT
                b.Book_ID,
                b.ISBN,
                b.Title,
                b.Publication_Year,
                b.Category_ID,
                c.Name AS category_name,
                p.Name AS publisher_name,
                (
                    SELECT LISTAGG(a.First_Name || ' ' || a.Last_Name, ', ') WITHIN GROUP (ORDER BY a.Last_Name)
                    FROM Book_Authors ba
                    JOIN Authors a ON ba.Author_ID = a.Author_ID
                    WHERE ba.Book_ID = b.Book_ID
                ) AS authors,
                (SELECT COUNT(*) FROM Book_Copies bc WHERE bc.Book_ID = b.Book_ID) AS total_copies,
                (SELECT COUNT(*) FROM Book_Copies bc WHERE bc.Book_ID = b.Book_ID AND bc.Status = 'Available') AS available_
            FROM Books b
            JOIN Categories c ON b.Category_ID = c.Category_ID
            LEFT JOIN Publishers p ON b.Publisher_ID = p.Publisher_ID
            WHERE b.Category_ID = :cid
            ORDER BY b.Title ASC
        """, {"cid": category_id})
```

```
        book_list = cursor.fetchall()

        # 3. If Student or Staff, check pending loan requests to update button states
        user_id = session.get("user_id")
        user_role = session.get("role")
        user_borrowing_blocked = False
        block_reason = ""
        user_active_loans = set()

        if user_id and user_role in ["Student", "Staff"]:
            # Books with a PENDING request
            cursor.execute("""
                SELECT Book_ID
                FROM Loan_Requests
                WHERE Borrower_ID = :user_id AND Status = 'Pending'
            """, {"user_id": user_id})
            for req in cursor.fetchall():
                user_requests.add(req["book_id"])

            # Books the user ALREADY holds an active loan for (same-book guard)
            cursor.execute("""
                SELECT DISTINCT bc.Book_ID
                FROM Loans l
                JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
                WHERE l.Borrower_ID = :user_id AND l.Return_Date IS NULL
            """, {"user_id": user_id})
            for row in cursor.fetchall():
                user_active_loans.add(row["book_id"])

            # Check if patron has unpaid fines
            cursor.execute("""
                SELECT COUNT(*) AS unpaid_count, NVL(SUM(f.Fine_Amount), 0) AS unpaid_total
                FROM Fines f
                JOIN Loans l ON f.Loan_ID = l.Loan_ID
                WHERE l.Borrower_ID = :user_id AND f.Payment_Status = 'Unpaid'
            """, {"user_id": user_id})
            f_check = cursor.fetchone()

            # Check if patron has overdue loans
            cursor.execute("""
                SELECT COUNT(*) AS overdue_count
                FROM Loans
                WHERE Borrower_ID = :user_id AND Return_Date IS NULL AND Due_Date < SYSDATE
            """, {"user_id": user_id})
            od_check = cursor.fetchone()

            if f_check and f_check["unpaid_count"] > 0:
                user_borrowing_blocked = True
                block_reason = f"Unpaid fines (Rs. {f_check['unpaid_total']:.2f}). Please pay overdue fines before borrowing
            elif od_check and od_check["overdue_count"] > 0:
                user_borrowing_blocked = True
                block_reason = f"{od_check['overdue_count']} loan(s) overdue. Please return overdue books before borrowing."

    except Exception as e:
        flash(f"Error retrieving category book collection: {str(e)}", "danger")
        return redirect(url_for("books"))
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    return render_template(
        "category_books.html",
        category=category,
        books=book_list,
        user_requests=user_requests,
        user_active_loans=user_active_loans,
        user_borrowing_blocked=user_borrowing_blocked,
        block_reason=block_reason
    )

```

```
646
647 # ------------------------------------------------------------------------
648 # Route: /adjust_inventory/<int:book_id> (Increase/Decrease Physical Book Copies)
649 # ------------------------------------------------------------------------
650 @app.route("/adjust_inventory/<int:book_id>", methods=["POST"])
651 @login_required
652 @role_required(["Admin", "Librarian"])
653 def adjust_inventory(book_id):
654     action = request.form.get("action")
655     category_id = request.form.get("category_id")
656     quantity_str = request.form.get("quantity", "1").strip()
657     conn = None
658     cursor = None
659
660     # Enforce strictly numerical positive input
661     if not re.match(r"^\d+$", quantity_str):
662         flash("Invalid quantity: Only positive numerical digits are allowed.", "warning")
663         if category_id:
664             try:
665                 return redirect(url_for("category_books", category_id=int(category_id)))
666             except (ValueError, TypeError):
667                 pass
668         return redirect(url_for("books"))
669
670     qty = int(quantity_str)
671     if qty <= 0:
672         flash("Quantity must be at least 1.", "warning")
673         if category_id:
674             try:
675                 return redirect(url_for("category_books", category_id=int(category_id)))
676             except (ValueError, TypeError):
677                 pass
678         return redirect(url_for("books"))
679
680     try:
681         conn = get_db_connection()
682         cursor = conn.cursor()
683
684         # Verify target book
685         cursor.execute("SELECT Title FROM Books WHERE Book_ID = :bid", {"bid": book_id})
686         book = cursor.fetchone()
687         if not book:
688             flash("Target book not found in library registry.", "warning")
689             return redirect(url_for("books"))
690
691         book_title = book["title"]
692
693         if action == "increase":
694             # Add qty new physical copies to Book_Copies
695             for _ in range(qty):
696                 cursor.execute("""
697                     INSERT INTO Book_Copies (Book_ID, Status, Condition)
698                     VALUES (:bid, 'Available', 'Good')
699                 """, {"bid": book_id})
700             conn.commit()
701
702             # Get updated counts
703             cursor.execute("""
704                 SELECT COUNT(*) AS total,
705                        SUM(CASE WHEN Status = 'Available' THEN 1 ELSE 0 END) AS avail
706                 FROM Book_Copies WHERE Book_ID = :bid
707             """, {"bid": book_id})
708             counts = cursor.fetchone()
709             flash(f"Successfully added {qty} physical copy/copies for '{book_title}' (Total copies: {counts['total']}, Availa
710
711         elif action == "decrease":
712             removed_count = 0
713             retired_count = 0
714
715             for _ in range(qty):
716                 # 1. Prioritize deleting an Available copy that has never been loaned (clean removal)
717                 cursor.execute("""
```

```
                    SELECT bc.Copy_ID
                    FROM Book_Copies bc
                    WHERE bc.Book_ID = :bid
                      AND bc.Status = 'Available'
                      AND NOT EXISTS (SELECT 1 FROM Loans l WHERE l.Copy_ID = bc.Copy_ID)
                    ORDER BY bc.Copy_ID DESC
                """, {"bid": book_id})
                clean_copy = cursor.fetchone()

                if clean_copy:
                    cid = clean_copy["copy_id"]
                    cursor.execute("DELETE FROM Book_Copies WHERE Copy_ID = :cid", {"cid": cid})
                    removed_count += 1
                else:
                    # 2. Check if any available copy exists with historical loans -> retire to Maintenance
                    cursor.execute("""
                        SELECT Copy_ID
                        FROM Book_Copies
                        WHERE Book_ID = :bid AND Status = 'Available'
                        ORDER BY Copy_ID DESC
                    """, {"bid": book_id})
                    hist_copy = cursor.fetchone()

                    if hist_copy:
                        cid = hist_copy["copy_id"]
                        cursor.execute("""
                            UPDATE Book_Copies
                            SET Status = 'Maintenance'
                            WHERE Copy_ID = :cid
                        """, {"cid": cid})
                        retired_count += 1
                    else:
                        break  # No more available copies exist to reduce

            conn.commit()
            total_reduced = removed_count + retired_count

            if total_reduced > 0:
                msg = f"Successfully reduced inventory for '{book_title}' by {total_reduced} copy/copies ({removed_count} de
                if total_reduced < qty:
                    msg += f" Note: Could not reduce the full requested quantity ({qty}) because only {total_reduced} copy/c
                flash(msg, "success")
            else:
                flash(f"Cannot decrease copies for '{book_title}': No available copies currently in stock (all copies may be

        else:
            flash("Invalid inventory adjustment action.", "danger")

    except Exception as e:
        if conn:
            conn.rollback()
        flash(f"Error updating inventory: {str(e)}", "danger")
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    if category_id:
        try:
            return redirect(url_for("category_books", category_id=int(category_id)))
        except (ValueError, TypeError):
            pass
    return redirect(url_for("books"))


# ------------------------------------------------------------------------
# Route: /add_book (Restricted to Admin, Librarian)
# ------------------------------------------------------------------------
@app.route("/add_book", methods=["GET", "POST"])
@login_required
@role_required(["Admin", "Librarian"])
```

```
def add_book():
    conn = None
    cursor = None
    categories = []
    publishers = []
    authors = []

    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.rowfactory = make_dict_factory(cursor)

        # Handle Book Insertion
        if request.method == "POST":
            isbn = request.form.get("isbn", "").strip()
            title = request.form.get("title", "").strip()
            pub_year = request.form.get("publication_year")
            category_id = request.form.get("category_id")
            publisher_id = request.form.get("publisher_id")
            selected_authors = request.form.getlist("author_ids")
            num_copies = int(request.form.get("num_copies", 1))

            # On-The-Fly Category Creation
            new_cat_name = request.form.get("new_category_name", "").strip()
            new_cat_code = request.form.get("new_category_code", "").strip()
            if new_cat_name:
                cursor.execute("SELECT Category_ID FROM Categories WHERE LOWER(Name) = LOWER(:name)", {"name": new_cat_name}
                existing_c = cursor.fetchone()
                if existing_c:
                    category_id = existing_c["category_id"]
                else:
                    new_cid = cursor.var(oracledb.NUMBER)
                    cursor.execute("INSERT INTO Categories (Name, Section_Code) VALUES (:name, :code) RETURNING Category_ID
                        "name": new_cat_name, "code": new_cat_code or None, "cid": new_cid
                    })
                    category_id = int(new_cid.getvalue()[0])

            # On-The-Fly Publisher Creation
            new_pub_name = request.form.get("new_publisher_name", "").strip()
            new_pub_email = request.form.get("new_publisher_email", "").strip()
            if new_pub_name:
                cursor.execute("SELECT Publisher_ID FROM Publishers WHERE LOWER(Name) = LOWER(:name)", {"name": new_pub_name}
                existing_p = cursor.fetchone()
                if existing_p:
                    publisher_id = existing_p["publisher_id"]
                else:
                    new_pid = cursor.var(oracledb.NUMBER)
                    cursor.execute("INSERT INTO Publishers (Name, Contact_Email) VALUES (:name, :email) RETURNING Publisher_
                        "name": new_pub_name, "email": new_pub_email or None, "pid": new_pid
                    })
                    publisher_id = int(new_pid.getvalue()[0])

            # On-The-Fly Author Creation
            new_auth_first = request.form.get("new_author_first", "").strip()
            new_auth_last = request.form.get("new_author_last", "").strip()
            if new_auth_first and new_auth_last:
                cursor.execute("SELECT Author_ID FROM Authors WHERE LOWER(First_Name) = LOWER(:fn) AND LOWER(Last_Name) = LO
                    "fn": new_auth_first, "ln": new_auth_last
                })
                existing_a = cursor.fetchone()
                if existing_a:
                    selected_authors.append(existing_a["author_id"])
                else:
                    new_aid = cursor.var(oracledb.NUMBER)
                    cursor.execute("INSERT INTO Authors (First_Name, Last_Name) VALUES (:fn, :ln) RETURNING Author_ID INTO :
                        "fn": new_auth_first, "ln": new_auth_last, "aid": new_aid
                    })
                    selected_authors.append(int(new_aid.getvalue()[0]))

            if not isbn or not title:
                flash("ISBN and Title are mandatory fields.", "warning")
            else:
```

```
                # Prepare out variable for Oracle RETURNING clause
                new_book_id = cursor.var(oracledb.NUMBER)

                insert_book_sql = """
                    INSERT INTO Books (ISBN, Title, Publication_Year, Category_ID, Publisher_ID)
                    VALUES (:isbn, :title, :pub_year, :category_id, :publisher_id)
                    RETURNING Book_ID INTO :book_id
                """
                cursor.execute(insert_book_sql, {
                    "isbn": isbn,
                    "title": title,
                    "pub_year": int(pub_year) if pub_year else None,
                    "category_id": int(category_id) if category_id else None,
                    "publisher_id": int(publisher_id) if publisher_id else None,
                    "book_id": new_book_id
                })

                book_id = int(new_book_id.getvalue()[0])

                # Associate selected authors
                insert_author_sql = """
                    INSERT INTO Book_Authors (Book_ID, Author_ID)
                    VALUES (:book_id, :author_id)
                """
                for author_id in selected_authors:
                    cursor.execute(insert_author_sql, {
                        "book_id": book_id,
                        "author_id": int(author_id)
                    })

                # Insert initial physical inventory copies
                insert_copy_sql = """
                    INSERT INTO Book_Copies (Book_ID, Status, Condition)
                    VALUES (:book_id, 'Available', 'New')
                """
                for _ in range(max(1, num_copies)):
                    cursor.execute(insert_copy_sql, {"book_id": book_id})

                conn.commit()
                flash(f"Book '{title}' and {num_copies} copies created successfully!", "success")
                return redirect(url_for("books"))

        # Pre-populate dropdown options for GET / failed POST
        cursor.execute("SELECT Category_ID, Name FROM Categories ORDER BY Name")
        categories = cursor.fetchall()

        cursor.execute("SELECT Publisher_ID, Name FROM Publishers ORDER BY Name")
        publishers = cursor.fetchall()

        cursor.execute("SELECT Author_ID, First_Name || ' ' || Last_Name AS Full_Name FROM Authors ORDER BY Last_Name")
        authors = cursor.fetchall()

    except Exception as e:
        if conn:
            conn.rollback()
        flash(f"Error processing book entry: {str(e)}", "danger")
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    return render_template(
        "add_book.html",
        categories=categories,
        publishers=publishers,
        authors=authors
    )


# ------------------------------------------------------------------------
# Route: /issue_loan (Restricted to Admin, Librarian, Staff)
```

```
# ------------------------------------------------------------------------
@app.route("/issue_loan", methods=["GET", "POST"])
@login_required
@role_required(["Admin", "Librarian", "Staff"])
def issue_loan():
    conn = None
    cursor = None
    available_copies = []
    eligible_borrowers = []

    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.rowfactory = make_dict_factory(cursor)

        if request.method == "POST":
            copy_id = request.form.get("copy_id")
            borrower_id = request.form.get("borrower_id")
            due_in_days = int(request.form.get("due_in_days", 14))
            issued_by = session["user_id"]

            if not copy_id or not borrower_id:
                flash("Please select both a valid physical copy and a borrower.", "warning")
            else:
                # 1. Lock and verify the copy status atomically
                cursor.execute(
                    "SELECT Status FROM Book_Copies WHERE Copy_ID = :copy_id FOR UPDATE",
                    {"copy_id": copy_id}
                )
                copy_row = cursor.fetchone()

                if not copy_row:
                    flash("Book copy not found.", "danger")
                elif copy_row["status"] != "Available":
                    flash("The selected copy is no longer available.", "warning")
                else:
                    # Check if borrower has unpaid fines
                    cursor.execute("""
                        SELECT COUNT(*) AS unpaid_count, NVL(SUM(f.Fine_Amount), 0) AS unpaid_total
                        FROM Fines f
                        JOIN Loans l ON f.Loan_ID = l.Loan_ID
                        WHERE l.Borrower_ID = :bid AND f.Payment_Status = 'Unpaid'
                    """, {"bid": borrower_id})
                    fine_res = cursor.fetchone()

                    # Check if borrower has overdue loans
                    cursor.execute("""
                        SELECT COUNT(*) AS overdue_count
                        FROM Loans
                        WHERE Borrower_ID = :bid AND Return_Date IS NULL AND Due_Date < SYSDATE
                    """, {"bid": borrower_id})
                    od_res = cursor.fetchone()

                    # Verify borrower has not exceeded max 3 active loans
                    cursor.execute("""
                        SELECT COUNT(*) AS active_count
                        FROM Loans
                        WHERE Borrower_ID = :bid AND Return_Date IS NULL
                    """, {"bid": borrower_id})
                    active_loans = cursor.fetchone()["active_count"]

                    # Check if borrower already has an active loan for the SAME book (via copy)
                    cursor.execute("""
                        SELECT COUNT(*) AS same_book_count
                        FROM Loans l
                        JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
                        WHERE l.Borrower_ID = :bid AND l.Return_Date IS NULL
                          AND bc.Book_ID = (SELECT Book_ID FROM Book_Copies WHERE Copy_ID = :copy_id)
                    """, {"bid": borrower_id, "copy_id": copy_id})
                    same_book_res = cursor.fetchone()

                    if fine_res and fine_res["unpaid_count"] > 0:
```

```
                        flash(f"Borrowing Prohibited: Selected borrower has unpaid overdue fines totaling Rs. {fine_res['unpa
                    elif od_res and od_res["overdue_count"] > 0:
                        flash(f"Borrowing Prohibited: Selected borrower currently has {od_res['overdue_count']} overdue loan
                    elif same_book_res and same_book_res["same_book_count"] > 0:
                        flash("Duplicate Loan Prohibited: The selected borrower already has an active loan for this book. A p
                    elif active_loans >= 3:
                        flash(f"Borrowing Threshold Exceeded: Selected borrower already has {active_loans} active loans. The
                    else:
                        # 2. Insert Loan Record
                        insert_loan_sql = """
                            INSERT INTO Loans (Copy_ID, Borrower_ID, Issued_By, Issue_Date, Due_Date)
                            VALUES (:copy_id, :borrower_id, :issued_by, SYSDATE, SYSDATE + :due_days)
                        """
                        cursor.execute(insert_loan_sql, {
                            "copy_id": copy_id,
                            "borrower_id": borrower_id,
                            "issued_by": issued_by,
                            "due_days": due_in_days
                        })

                        # 3. Transition copy status to 'Borrowed'
                        cursor.execute(
                            "UPDATE Book_Copies SET Status = 'Borrowed' WHERE Copy_ID = :copy_id",
                            {"copy_id": copy_id}
                        )

                        conn.commit()
                        flash(f"Loan successfully created for Copy ID #{copy_id}.", "success")
                        return redirect(url_for("dashboard"))

        # Fetch active available copies for issuance
        cursor.execute("""
            SELECT bc.Copy_ID, b.Title, bc.Condition
            FROM Book_Copies bc
            JOIN Books b ON bc.Book_ID = b.Book_ID
            WHERE bc.Status = 'Available'
            ORDER BY b.Title, bc.Copy_ID
        """)
        available_copies = cursor.fetchall()

        # Fetch registered active students/staff
        cursor.execute("""
            SELECT User_ID, Username, Email, Role
            FROM Users
            WHERE Role IN ('Student', 'Staff') AND Account_Status = 'Active'
            ORDER BY Username
        """)
        eligible_borrowers = cursor.fetchall()

    except Exception as e:
        if conn:
            conn.rollback()
        flash(f"Failed to issue loan: {str(e)}", "danger")
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    return render_template(
        "issue_loan.html",
        copies=available_copies,
        borrowers=eligible_borrowers
    )


# ------------------------------------------------------------------------
# Route: /my_fines (Restricted to logged-in user / student)
# ------------------------------------------------------------------------
@app.route("/my_fines", methods=["GET"])
@login_required
def my_fines():
```

```
    user_id = session.get("user_id")
    conn = None
    cursor = None
    fines_list = []

    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.rowfactory = make_dict_factory(cursor)

        query = """
            SELECT
                f.Fine_ID,
                f.Fine_Amount,
                f.Payment_Status,
                l.Loan_ID,
                l.Issue_Date,
                l.Due_Date,
                l.Return_Date,
                b.Title,
                bc.Copy_ID
            FROM Fines f
            JOIN Loans l ON f.Loan_ID = l.Loan_ID
            JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
            JOIN Books b ON bc.Book_ID = b.Book_ID
            WHERE l.Borrower_ID = :borrower_id
            ORDER BY f.Fine_ID DESC
        """
        cursor.execute(query, {"borrower_id": user_id})
        fines_list = cursor.fetchall()

    except Exception as e:
        flash(f"Error fetching fine statements: {str(e)}", "danger")
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    return render_template("my_fines.html", fines=fines_list)


# ------------------------------------------------------------------------
# Route: /manage_users (Restricted to Admin)
# ------------------------------------------------------------------------
@app.route("/manage_users", methods=["GET", "POST"])
@login_required
@role_required(["Admin"])
def manage_users():
    conn = None
    cursor = None
    users_list = []
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        if request.method == "POST":
            action = request.form.get("action")
            if action == "add_user":
                new_uname = request.form.get("username", "").strip()
                new_email = request.form.get("email", "").strip()
                new_pass = request.form.get("password", "")
                new_role = request.form.get("role", "Student")
                if not new_uname or not new_email or not new_pass:
                    flash("All fields are required to create a user.", "warning")
                else:
                    is_email_valid, email_err = validate_email_format(new_email)
                    if not is_email_valid:
                        flash(f"Invalid Email: {email_err}", "warning")
                    else:
                        is_pass_valid, pass_err = validate_password_strength(new_pass)
                        if not is_pass_valid:
```

```
                            flash(f"Invalid Password: {pass_err}", "warning")
                        else:
                            pw_hash = generate_password_hash(new_pass)
                            cursor.execute("""
                                INSERT INTO Users (Username, Password_Hash, Email, Role, Account_Status)
                                VALUES (:username, :pw_hash, :email, :role, 'Active')
                            """, {
                                "username": new_uname,
                                "pw_hash": pw_hash,
                                "email": new_email,
                                "role": new_role
                            })
                            conn.commit()
                            flash(f"User '{new_uname}' ({new_role}) created successfully!", "success")

            elif action == "toggle_status":
                target_user_id = request.form.get("user_id")
                current_status = request.form.get("current_status")
                new_status = "Suspended" if current_status == "Active" else "Active"
                cursor.execute("""
                    UPDATE Users
                    SET Account_Status = :status
                    WHERE User_ID = :user_id
                """, {"status": new_status, "user_id": target_user_id})
                conn.commit()
                flash(f"User ID #{target_user_id} status updated to '{new_status}'.", "info")

        # Exclude the currently logged-in Admin from the management table
        current_admin_id = session.get("user_id")
        cursor.execute("""
            SELECT User_ID, Username, Email, Role, Account_Status
            FROM Users
            WHERE User_ID != :admin_id
            ORDER BY User_ID ASC
        """, {"admin_id": current_admin_id})
        users_list = cursor.fetchall()
    except Exception as e:
        if conn:
            conn.rollback()
        flash(f"Error managing users: {str(e)}", "danger")
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    return render_template("manage_users.html", users=users_list)


# ------------------------------------------------------------------------
# Route: /delete_user/<int:user_id> (Admin-Only User Deletion with Checks)
# ------------------------------------------------------------------------
@app.route("/delete_user/<int:user_id>", methods=["POST"])
@login_required
@role_required(["Admin"])
def delete_user(user_id):
    # Prevent Admin from accidentally deleting themselves
    if user_id == session.get("user_id"):
        flash("Action Prohibited: You cannot delete your own active Admin account.", "danger")
        return redirect(url_for("manage_users"))

    conn = None
    cursor = None
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # Check if the target user is a Librarian (Admin cannot delete Librarian)
        cursor.execute("SELECT Role, Username FROM Users WHERE User_ID = :user_id", {"user_id": user_id})
        target_user = cursor.fetchone()
        if not target_user:
            flash("User not found.", "warning")
```

```
            return redirect(url_for("manage_users"))

        if target_user["role"] == "Librarian":
            flash(f"Action Prohibited: Librarian account '{target_user['username']}' cannot be deleted by policy.", "danger"
            return redirect(url_for("manage_users"))

        # 1. Constraint Check: Check for active loans (Return_Date IS NULL)
        cursor.execute("""
            SELECT COUNT(*) AS active_loans
            FROM Loans
            WHERE Borrower_ID = :user_id AND Return_Date IS NULL
        """, {"user_id": user_id})
        active_loans = cursor.fetchone()["active_loans"]

        # 2. Constraint Check: Check for unpaid fines
        cursor.execute("""
            SELECT COUNT(*) AS unpaid_fines
            FROM Fines f
            JOIN Loans l ON f.Loan_ID = l.Loan_ID
            WHERE l.Borrower_ID = :user_id AND f.Payment_Status = 'Unpaid'
        """, {"user_id": user_id})
        unpaid_fines = cursor.fetchone()["unpaid_fines"]

        # If user has unresolved library obligations, block deletion
        if active_loans > 0 or unpaid_fines > 0:
            flash("Cannot delete user: Active loans or unpaid fines exist.", "danger")
            return redirect(url_for("manage_users"))

        # 3. Clean up historical requests, closed loans, and settled fines to maintain integrity
        cursor.execute("DELETE FROM Loan_Requests WHERE Borrower_ID = :user_id", {"user_id": user_id})

        cursor.execute("BEGIN DBMS_APPLICATION_INFO.SET_CLIENT_INFO('ALLOW_DELETE'); END;")
        cursor.execute("""
            DELETE FROM Fines
            WHERE Loan_ID IN (SELECT Loan_ID FROM Loans WHERE Borrower_ID = :user_id)
        """, {"user_id": user_id})
        cursor.execute("BEGIN DBMS_APPLICATION_INFO.SET_CLIENT_INFO(NULL); END;")

        cursor.execute("""
            DELETE FROM Loans
            WHERE Borrower_ID = :user_id
        """, {"user_id": user_id})

        # Reassign any historical loans issued by this user to current Admin
        cursor.execute("""
            UPDATE Loans
            SET Issued_By = :admin_id
            WHERE Issued_By = :user_id
        """, {"admin_id": session.get("user_id"), "user_id": user_id})

        # 4. Perform final user deletion
        cursor.execute("DELETE FROM Users WHERE User_ID = :user_id", {"user_id": user_id})
        conn.commit()

        flash(f"User ID #{user_id} has been permanently deleted from the system.", "success")

    except Exception as e:
        if conn:
            conn.rollback()
        flash(f"Error during account deletion: {str(e)}", "danger")
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    return redirect(url_for("manage_users"))


# ------------------------------------------------------------------------
# Route: /change_password (Authenticated Users Self-Service Password Change)
# ------------------------------------------------------------------------
```

```python
@app.route("/change_password", methods=["GET", "POST"])
@login_required
def change_password():
    user_id = session.get("user_id")

    if request.method == "POST":
        current_pass = request.form.get("current_password", "")
        new_pass = request.form.get("new_password", "")
        confirm_pass = request.form.get("confirm_password", "")

        if not current_pass or not new_pass or not confirm_pass:
            flash("All password fields are required.", "warning")
            return render_template("change_password.html")

        if new_pass != confirm_pass:
            flash("New password and confirmation do not match.", "warning")
            return render_template("change_password.html")

        is_valid, err_msg = validate_password_strength(new_pass)
        if not is_valid:
            flash(f"Invalid New Password: {err_msg}", "warning")
            return render_template("change_password.html")

        conn = None
        cursor = None
        try:
            conn = get_db_connection()
            cursor = conn.cursor()

            cursor.execute("SELECT Password_Hash FROM Users WHERE User_ID = :user_id", {"user_id": user_id})
            user_row = cursor.fetchone()

            if not user_row or not check_password_hash(user_row["password_hash"], current_pass):
                flash("Current password verification failed. Please try again.", "danger")
                return render_template("change_password.html")

            if check_password_hash(user_row["password_hash"], new_pass):
                flash("New password cannot be identical to your current password.", "warning")
                return render_template("change_password.html")

            new_hash = generate_password_hash(new_pass)
            cursor.execute("""
                UPDATE Users
                SET Password_Hash = :phash
                WHERE User_ID = :user_id
            """, {"phash": new_hash, "user_id": user_id})
            conn.commit()

            flash("Your password has been changed successfully!", "success")
            return redirect(url_for("dashboard"))

        except Exception as e:
            if conn:
                conn.rollback()
            flash(f"Error updating password: {str(e)}", "danger")
        finally:
            if cursor:
                cursor.close()
            if conn:
                conn.close()

    return render_template("change_password.html")


# ------------------------------------------------------------------------
# Route: /request_loan/<int:book_id> (Patron Loan Request Workflow)
# ------------------------------------------------------------------------
@app.route("/request_loan/<int:book_id>", methods=["POST"])
@login_required
def request_loan(book_id):
    user_id = session.get("user_id")
    role = session.get("role")
```

```
1366
1367      if role not in ["Student", "Staff"]:
1368          flash("Action Restricted: Only Students and Staff may request book loans.", "warning")
1369          return redirect(url_for("books"))
1370
1371      conn = None
1372      cursor = None
1373      try:
1374          conn = get_db_connection()
1375          cursor = conn.cursor()
1376
1377          # 1. Check if user has unpaid fines
1378          cursor.execute("""
1379              SELECT COUNT(*) AS unpaid_count, NVL(SUM(f.Fine_Amount), 0) AS unpaid_total
1380              FROM Fines f
1381              JOIN Loans l ON f.Loan_ID = l.Loan_ID
1382              WHERE l.Borrower_ID = :user_id AND f.Payment_Status = 'Unpaid'
1383          """, {"user_id": user_id})
1384          fine_res = cursor.fetchone()
1385          if fine_res and fine_res["unpaid_count"] > 0:
1386              flash(f"Permission Denied: You have unpaid fines of Rs. {fine_res['unpaid_total']:.2f} on your account. Please s
1387              return redirect(url_for("my_fines"))
1388
1389          # 2. Check if user has overdue loans
1390          cursor.execute("""
1391              SELECT COUNT(*) AS overdue_count
1392              FROM Loans
1393              WHERE Borrower_ID = :user_id AND Return_Date IS NULL AND Due_Date < SYSDATE
1394          """, {"user_id": user_id})
1395          overdue_res = cursor.fetchone()
1396          if overdue_res and overdue_res["overdue_count"] > 0:
1397              flash(f"Permission Denied: You have {overdue_res['overdue_count']} overdue book(s) that have not been returned. 
1398              return redirect(url_for("dashboard"))
1399
1400          # 3. Check if user reached maximum concurrent borrowing quota (3 books)
1401          cursor.execute("""
1402              SELECT COUNT(*) AS active_count
1403              FROM Loans
1404              WHERE Borrower_ID = :user_id AND Return_Date IS NULL
1405          """, {"user_id": user_id})
1406          if cursor.fetchone()["active_count"] >= 3:
1407              flash("Permission Denied: You have reached the maximum quota of 3 active loans.", "warning")
1408              return redirect(url_for("dashboard"))
1409
1410          # 4. Check for existing active loan for the SAME book (duplicate borrow guard)
1411          cursor.execute("""
1412              SELECT COUNT(*) AS same_book_count
1413              FROM Loans l
1414              JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
1415              WHERE l.Borrower_ID = :user_id AND l.Return_Date IS NULL AND bc.Book_ID = :bid
1416          """, {"user_id": user_id, "bid": book_id})
1417          if cursor.fetchone()["same_book_count"] > 0:
1418              flash("Duplicate Loan: You already have an active loan for this book. You cannot borrow the same book more than 
1419              cat_id = request.form.get("category_id")
1420              if cat_id:
1421                  try:
1422                      return redirect(url_for("category_books", category_id=int(cat_id)))
1423                  except (ValueError, TypeError):
1424                      pass
1425              return redirect(url_for("books"))
1426
1427          # 5. Check for existing pending request on this book
1428          cursor.execute("""
1429              SELECT COUNT(*) AS pending_count
1430              FROM Loan_Requests
1431              WHERE Book_ID = :bid AND Borrower_ID = :borrower_id AND Status = 'Pending'
1432          """, {"bid": book_id, "borrower_id": user_id})
1433          if cursor.fetchone()["pending_count"] > 0:
1434              flash("You already have an active pending loan request for this title.", "info")
1435              return redirect(url_for("books"))
1436
1437          # 4. Fetch Book Title for confirmation message
```

```
        cursor.execute("SELECT Title FROM Books WHERE Book_ID = :bid", {"bid": book_id})
        book_row = cursor.fetchone()
        book_title = book_row["title"] if book_row else f"Book #{book_id}"

        # 5. Insert Loan Request
        cursor.execute("""
            INSERT INTO Loan_Requests (Book_ID, Borrower_ID, Request_Date, Status)
            VALUES (:bid, :borrower_id, SYSDATE, 'Pending')
        """, {"bid": book_id, "borrower_id": user_id})
        conn.commit()

        flash(f"Loan request for '{book_title}' submitted successfully! Awaiting Librarian approval.", "success")

    except Exception as e:
        if conn:
            conn.rollback()
        flash(f"Failed to submit loan request: {str(e)}", "danger")
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    cat_id = request.form.get("category_id")
    if cat_id:
        try:
            return redirect(url_for("category_books", category_id=int(cat_id)))
        except (ValueError, TypeError):
            pass
    return redirect(url_for("books"))


# ------------------------------------------------------------------------
# Route: /loan_requests (Librarian/Admin Loan Requests Management)
# ------------------------------------------------------------------------
@app.route("/loan_requests", methods=["GET", "POST"])
@login_required
@role_required(["Admin", "Librarian"])
def loan_requests():
    conn = None
    cursor = None
    requests_list = []
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        if request.method == "POST":
            req_id = request.form.get("request_id")
            action = request.form.get("action")
            issuer_id = session.get("user_id")

            if action == "approve":
                # Find request details
                cursor.execute("""
                    SELECT Book_ID, Borrower_ID, Status
                    FROM Loan_Requests
                    WHERE Request_ID = :rid
                """, {"rid": req_id})
                req_data = cursor.fetchone()

                if not req_data or req_data["status"] != "Pending":
                    flash("Invalid or already processed request.", "warning")
                else:
                    borrower_id = req_data["borrower_id"]
                    cursor.execute("SELECT Username FROM Users WHERE User_ID = :bid", {"bid": borrower_id})
                    b_row = cursor.fetchone()
                    b_name = b_row["username"] if b_row else f"User #{borrower_id}"

                    # 1. Check if borrower has unpaid fines
                    cursor.execute("""
                        SELECT COUNT(*) AS unpaid_count, NVL(SUM(f.Fine_Amount), 0) AS unpaid_total
                        FROM Fines f
```

```
                        JOIN Loans l ON f.Loan_ID = l.Loan_ID
                        WHERE l.Borrower_ID = :bid AND f.Payment_Status = 'Unpaid'
                    """, {"bid": borrower_id})
                    fine_res = cursor.fetchone()

                    # 2. Check if borrower has overdue loans
                    cursor.execute("""
                        SELECT COUNT(*) AS overdue_count
                        FROM Loans
                        WHERE Borrower_ID = :bid AND Return_Date IS NULL AND Due_Date < SYSDATE
                    """, {"bid": borrower_id})
                    od_res = cursor.fetchone()

                    # 3. Enforce strict 3 active loans quota threshold
                    cursor.execute("""
                        SELECT COUNT(*) AS active_count
                        FROM Loans
                        WHERE Borrower_ID = :bid AND Return_Date IS NULL
                    """, {"bid": borrower_id})
                    active_loans = cursor.fetchone()["active_count"]

                    # 4. Check if borrower already holds an active loan for the SAME book
                    cursor.execute("""
                        SELECT COUNT(*) AS same_book_count
                        FROM Loans l
                        JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
                        WHERE l.Borrower_ID = :bid AND l.Return_Date IS NULL AND bc.Book_ID = :book_id
                    """, {"bid": borrower_id, "book_id": req_data["book_id"]})
                    same_book_res = cursor.fetchone()

                    if fine_res and fine_res["unpaid_count"] > 0:
                        flash(f"Approval Prohibited: Patron '{b_name}' has unpaid overdue fines totaling Rs. {fine_res['unpa
                    elif od_res and od_res["overdue_count"] > 0:
                        flash(f"Approval Prohibited: Patron '{b_name}' currently has {od_res['overdue_count']} loan(s) overd
                    elif same_book_res and same_book_res["same_book_count"] > 0:
                        flash(f"Approval Prohibited: Patron '{b_name}' already has an active loan for this book. A patron ca
                    elif active_loans >= 3:
                        flash(f"Approval Prohibited: Patron '{b_name}' already has {active_loans} active loans, reaching the
                    else:
                        # Find an available physical copy of this book
                        cursor.execute("""
                            SELECT Copy_ID
                            FROM Book_Copies
                            WHERE Book_ID = :bid AND Status = 'Available' AND ROWNUM = 1
                            FOR UPDATE
                        """, {"bid": req_data["book_id"]})
                        avail_copy = cursor.fetchone()

                        if not avail_copy:
                            flash("No physical copies are currently Available for this book.", "danger")
                        else:
                            copy_id = avail_copy["copy_id"]
                            # Create Loan
                            cursor.execute("""
                                INSERT INTO Loans (Copy_ID, Borrower_ID, Issued_By, Issue_Date, Due_Date)
                                VALUES (:cid, :bid, :issued_by, SYSDATE, SYSDATE + 14)
                            """, {
                                "cid": copy_id,
                                "bid": req_data["borrower_id"],
                                "issued_by": issuer_id
                            })
                            # Update Copy Status
                            cursor.execute("UPDATE Book_Copies SET Status = 'Borrowed' WHERE Copy_ID = :cid", {"cid": copy_i
                            # Update Request Status
                            cursor.execute("UPDATE Loan_Requests SET Status = 'Approved' WHERE Request_ID = :rid", {"rid": r
                            conn.commit()

                            flash(f"Loan Request #{req_id} approved! Physical Copy #{copy_id} issued to patron '{b_name}'.",

            elif action == "reject":
                cursor.execute("UPDATE Loan_Requests SET Status = 'Rejected' WHERE Request_ID = :rid", {"rid": req_id})
                conn.commit()
```

```
                flash(f"Loan Request #{req_id} rejected.", "info")

        # Fetch all pending requests with book and borrower information including active loan count, overdue loans, and fines
        cursor.execute("""
            SELECT
                lr.Request_ID,
                lr.Book_ID,
                b.Title AS book_title,
                b.ISBN,
                lr.Borrower_ID,
                u.Username AS borrower_name,
                u.Email AS borrower_email,
                u.Role AS borrower_role,
                lr.Request_Date,
                (SELECT COUNT(*) FROM Book_Copies bc WHERE bc.Book_ID = lr.Book_ID AND bc.Status = 'Available') AS available_
                (SELECT COUNT(*) FROM Loans l WHERE l.Borrower_ID = lr.Borrower_ID AND l.Return_Date IS NULL) AS borrower_ac
                (SELECT COUNT(*) FROM Loans l WHERE l.Borrower_ID = lr.Borrower_ID AND l.Return_Date IS NULL AND l.Due_Date <
                NVL((SELECT SUM(f.Fine_Amount) FROM Fines f JOIN Loans l ON f.Loan_ID = l.Loan_ID WHERE l.Borrower_ID = lr.Bo
            FROM Loan_Requests lr
            JOIN Books b ON lr.Book_ID = b.Book_ID
            JOIN Users u ON lr.Borrower_ID = u.User_ID
            WHERE lr.Status = 'Pending'
            ORDER BY lr.Request_Date ASC
        """)
        requests_list = cursor.fetchall()

    except Exception as e:
        if conn:
            conn.rollback()
        flash(f"Error handling loan requests: {str(e)}", "danger")
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    return render_template("loan_requests.html", requests=requests_list)


# ------------------------------------------------------------------------------
# Route: /return_book/<int:loan_id> (Admin/Librarian Return Book Workflow)
# ------------------------------------------------------------------------------
@app.route("/return_book/<int:loan_id>", methods=["POST"])
@login_required
@role_required(["Admin", "Librarian"])
def return_book(loan_id):
    conn = None
    cursor = None
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 1. Fetch loan details and lock
        cursor.execute("""
            SELECT l.Loan_ID, l.Copy_ID, l.Borrower_ID, l.Due_Date, l.Return_Date, b.Title
            FROM Loans l
            JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
            JOIN Books b ON bc.Book_ID = b.Book_ID
            WHERE l.Loan_ID = :lid
            FOR UPDATE
        """, {"lid": loan_id})
        loan = cursor.fetchone()

        if not loan:
            flash(f"Loan transaction #{loan_id} not found.", "warning")
            return redirect(url_for("dashboard"))

        if loan["return_date"] is not None:
            flash(f"Book '{loan['title']}' was already returned on {loan['return_date']}.", "info")
            return redirect(url_for("dashboard"))

        copy_id = loan["copy_id"]
```

```
        due_date = loan["due_date"]
        book_title = loan["title"]

        # 2. Mark loan as returned
        cursor.execute("""
            UPDATE Loans
            SET Return_Date = SYSDATE
            WHERE Loan_ID = :lid
        """, {"lid": loan_id})

        # 3. Transition copy inventory status back to 'Available'
        cursor.execute("""
            UPDATE Book_Copies
            SET Status = 'Available'
            WHERE Copy_ID = :cid
        """, {"cid": copy_id})

        # 4. Check if book is overdue and assess fine if applicable (Rs. 10 / day)
        cursor.execute("""
            SELECT TRUNC(SYSDATE) - TRUNC(:due_date) AS days_overdue
            FROM DUAL
        """, {"due_date": due_date})
        overdue_row = cursor.fetchone()
        days_overdue = overdue_row["days_overdue"] if overdue_row else 0

        fine_msg = ""
        if days_overdue > 0:
            fine_amount = days_overdue * 10.0
            # Check if fine already exists for this Loan_ID to prevent ORA-00001 unique constraint violation
            cursor.execute("""
                SELECT Fine_ID, Fine_Amount, Payment_Status
                FROM Fines
                WHERE Loan_ID = :lid
                FOR UPDATE
            """, {"lid": loan_id})
            existing_fine = cursor.fetchone()

            if existing_fine:
                if existing_fine["payment_status"] == "Unpaid":
                    cursor.execute("""
                        UPDATE Fines
                        SET Fine_Amount = :amount
                        WHERE Fine_ID = :fid
                    """, {"amount": fine_amount, "fid": existing_fine["fine_id"]})
                    fine_msg = f" Note: Book was overdue by {days_overdue} day(s). Outstanding fine updated to Rs. {fine_amou
                else:
                    fine_msg = f" Note: Book was overdue by {days_overdue} day(s). Existing fine record is already marked '{
            else:
                cursor.execute("""
                    INSERT INTO Fines (Loan_ID, Fine_Amount, Payment_Status)
                    VALUES (:lid, :amount, 'Unpaid')
                """, {"lid": loan_id, "amount": fine_amount})
                fine_msg = f" Note: Book was overdue by {days_overdue} day(s). Penalty fine of Rs. {fine_amount:.2f} assesse

        conn.commit()
        flash(f"Successfully processed return for '{book_title}' (Copy #{copy_id})! Physical copy is now Available in stock.

    except Exception as e:
        if conn:
            conn.rollback()
        flash(f"Error processing book return: {str(e)}", "danger")
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    return redirect(url_for("dashboard"))


# ------------------------------------------------------------------------
# Route: /pay_fine/<int:fine_id> (Admin/Librarian Settle Fine Workflow)
```

```
# ------------------------------------------------------------------------
@app.route("/pay_fine/<int:fine_id>", methods=["POST"])
@login_required
@role_required(["Admin", "Librarian"])
def pay_fine(fine_id):
    conn = None
    cursor = None
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT f.Fine_ID, f.Fine_Amount, f.Payment_Status, u.Username
            FROM Fines f
            JOIN Loans l ON f.Loan_ID = l.Loan_ID
            JOIN Users u ON l.Borrower_ID = u.User_ID
            WHERE f.Fine_ID = :fid
            FOR UPDATE
        """, {"fid": fine_id})
        fine = cursor.fetchone()

        if not fine:
            flash(f"Fine record #{fine_id} not found.", "warning")
            return redirect(url_for("dashboard"))

        if fine["payment_status"] == "Paid":
            flash(f"Fine #{fine_id} was already marked as Paid.", "info")
            return redirect(url_for("dashboard"))

        cursor.execute("""
            UPDATE Fines
            SET Payment_Status = 'Paid'
            WHERE Fine_ID = :fid
        """, {"fid": fine_id})
        conn.commit()

        flash(f"Fine #{fine_id} (Rs. {fine['fine_amount']:.2f}) for patron '{fine['username']}' marked as Paid! Account stan

    except Exception as e:
        if conn:
            conn.rollback()
        flash(f"Error updating fine payment status: {str(e)}", "danger")
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    return redirect(url_for("dashboard"))


# ------------------------------------------------------------------------
# Route: /delete_loan/<int:loan_id> (Admin/Librarian Delete Active / Overdue Loan)
# ------------------------------------------------------------------------
@app.route("/delete_loan/<int:loan_id>", methods=["POST"])
@login_required
@role_required(["Admin", "Librarian"])
def delete_loan(loan_id):
    conn = None
    cursor = None
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 1. Fetch loan details and lock
        cursor.execute("""
            SELECT l.Loan_ID, l.Copy_ID, l.Borrower_ID, u.Username AS borrower_name, b.Title AS book_title
            FROM Loans l
            JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
            JOIN Books b ON bc.Book_ID = b.Book_ID
            JOIN Users u ON l.Borrower_ID = u.User_ID
            WHERE l.Loan_ID = :lid
```

```
            FOR UPDATE
        """, {"lid": loan_id})
        loan = cursor.fetchone()

        if not loan:
            flash(f"Loan record #{loan_id} not found.", "warning")
            return redirect(url_for("dashboard"))

        copy_id = loan["copy_id"]
        borrower_name = loan["borrower_name"]
        book_title = loan["book_title"]

        # 2. Allow fine deletion by setting client info to bypass security trigger
        cursor.execute("BEGIN DBMS_APPLICATION_INFO.SET_CLIENT_INFO('ALLOW_DELETE'); END;")

        # 3. Delete any fines associated with this loan
        cursor.execute("DELETE FROM Fines WHERE Loan_ID = :lid", {"lid": loan_id})

        # 4. Reset client info
        cursor.execute("BEGIN DBMS_APPLICATION_INFO.SET_CLIENT_INFO(NULL); END;")

        # 5. Restore book copy inventory status to 'Available'
        cursor.execute("UPDATE Book_Copies SET Status = 'Available' WHERE Copy_ID = :cid", {"cid": copy_id})

        # 6. Delete the loan record itself (deletes both overdue loan and active loan for that person)
        cursor.execute("DELETE FROM Loans WHERE Loan_ID = :lid", {"lid": loan_id})

        conn.commit()
        flash(f"Overdue loan #{loan_id} and active loan for patron '{borrower_name}' ('{book_title}') were successfully dele

    except Exception as e:
        if conn:
            conn.rollback()
        flash(f"Error deleting loan record: {str(e)}", "danger")
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

    return redirect(url_for("dashboard"))


# ==============================================================================
# Entry Point
# ==============================================================================
if __name__ == "__main__":
    app.run(debug=True, host="127.0.0.1", port=5000)
```

ddl.sql

SQL - Schema DDL

```sql
-- ============================================================================
-- Enterprise Library Management System DDL Script (Oracle 21c)
-- ============================================================================

-- ============================================================================
-- 0. CLEANUP (Drop existing tables in reverse dependency order)
-- ============================================================================
DROP TABLE Fines CASCADE CONSTRAINTS PURGE;
DROP TABLE Loans CASCADE CONSTRAINTS PURGE;
DROP TABLE Book_Copies CASCADE CONSTRAINTS PURGE;
DROP TABLE Book_Authors CASCADE CONSTRAINTS PURGE;
DROP TABLE Books CASCADE CONSTRAINTS PURGE;
DROP TABLE Authors CASCADE CONSTRAINTS PURGE;
DROP TABLE Publishers CASCADE CONSTRAINTS PURGE;
DROP TABLE Categories CASCADE CONSTRAINTS PURGE;
DROP TABLE Users CASCADE CONSTRAINTS PURGE;

-- ============================================================================
-- 1. Users Table (Handles Authentication & Role-Based Access)
-- ============================================================================
CREATE TABLE Users (
    User_ID        NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Username       VARCHAR2(50) NOT NULL,
    Password_Hash  VARCHAR2(255) NOT NULL,
    Email          VARCHAR2(255) NOT NULL,
    Role           VARCHAR2(20) NOT NULL,
    Account_Status VARCHAR2(20) DEFAULT 'Active' NOT NULL,
    CONSTRAINT uq_users_username UNIQUE (Username),
    CONSTRAINT uq_users_email UNIQUE (Email),
    CONSTRAINT chk_users_role CHECK (Role IN ('Admin', 'Librarian', 'Staff', 'Student')),
    CONSTRAINT chk_users_status CHECK (Account_Status IN ('Active', 'Suspended'))
);

-- ============================================================================
-- 2. Categories Table
-- ============================================================================
CREATE TABLE Categories (
    Category_ID  NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Name         VARCHAR2(100) NOT NULL,
    Section_Code VARCHAR2(10),
    CONSTRAINT uq_categories_name UNIQUE (Name)
);

-- ============================================================================
-- 3. Publishers Table
-- ============================================================================
CREATE TABLE Publishers (
    Publisher_ID  NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Name          VARCHAR2(150) NOT NULL,
    Contact_Email VARCHAR2(255)
);

-- ============================================================================
-- 4. Authors Table
-- ============================================================================
CREATE TABLE Authors (
    Author_ID  NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    First_Name VARCHAR2(100) NOT NULL,
    Last_Name  VARCHAR2(100) NOT NULL
);

-- ============================================================================
-- 5. Books Table
-- ============================================================================
CREATE TABLE Books (
    Book_ID          NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    ISBN             VARCHAR2(20) NOT NULL,
    Title            VARCHAR2(255) NOT NULL,
    Publication_Year NUMBER(4),
```

```
    Category_ID      NUMBER,
    Publisher_ID     NUMBER,
    CONSTRAINT uq_books_isbn UNIQUE (ISBN),
    CONSTRAINT fk_books_category FOREIGN KEY (Category_ID) REFERENCES Categories (Category_ID),
    CONSTRAINT fk_books_publisher FOREIGN KEY (Publisher_ID) REFERENCES Publishers (Publisher_ID)
);

-- ============================================================================
-- 6. Book_Authors Table (Resolves Many-to-Many relationship)
-- ============================================================================
CREATE TABLE Book_Authors (
    Book_ID   NUMBER NOT NULL,
    Author_ID NUMBER NOT NULL,
    CONSTRAINT pk_book_authors PRIMARY KEY (Book_ID, Author_ID),
    CONSTRAINT fk_bookauthors_book FOREIGN KEY (Book_ID) REFERENCES Books (Book_ID),
    CONSTRAINT fk_bookauthors_author FOREIGN KEY (Author_ID) REFERENCES Authors (Author_ID)
);

-- ============================================================================
-- 7. Book_Copies Table (Tracks physical inventory)
-- ============================================================================
CREATE TABLE Book_Copies (
    Copy_ID   NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Book_ID   NUMBER NOT NULL,
    Status    VARCHAR2(20) DEFAULT 'Available' NOT NULL,
    Condition VARCHAR2(20) DEFAULT 'Good' NOT NULL,
    CONSTRAINT fk_bookcopies_book FOREIGN KEY (Book_ID) REFERENCES Books (Book_ID),
    CONSTRAINT chk_copies_status CHECK (Status IN ('Available', 'Borrowed', 'Lost', 'Maintenance')),
    CONSTRAINT chk_copies_condition CHECK (Condition IN ('New', 'Good', 'Fair', 'Poor'))
);

-- ============================================================================
-- 8. Loans Table (Tracks check-outs)
-- ============================================================================
CREATE TABLE Loans (
    Loan_ID     NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Copy_ID     NUMBER NOT NULL,
    Borrower_ID NUMBER NOT NULL,
    Issued_By   NUMBER NOT NULL,
    Issue_Date  DATE DEFAULT SYSDATE NOT NULL,
    Due_Date    DATE NOT NULL,
    Return_Date DATE,
    CONSTRAINT fk_loans_copy FOREIGN KEY (Copy_ID) REFERENCES Book_Copies (Copy_ID),
    CONSTRAINT fk_loans_borrower FOREIGN KEY (Borrower_ID) REFERENCES Users (User_ID),
    CONSTRAINT fk_loans_issued_by FOREIGN KEY (Issued_By) REFERENCES Users (User_ID)
);

-- ============================================================================
-- 9. Fines Table
-- ============================================================================
CREATE TABLE Fines (
    Fine_ID        NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Loan_ID        NUMBER NOT NULL,
    Fine_Amount    NUMBER(10, 2) NOT NULL,
    Payment_Status VARCHAR2(20) DEFAULT 'Unpaid' NOT NULL,
    CONSTRAINT uq_fines_loan UNIQUE (Loan_ID),
    CONSTRAINT fk_fines_loan FOREIGN KEY (Loan_ID) REFERENCES Loans (Loan_ID),
    CONSTRAINT chk_fines_payment_status CHECK (Payment_Status IN ('Paid', 'Unpaid', 'Waived'))
);
INSERT INTO Users (Username, Password_Hash, Email, Role, Account_Status)
VALUES ('admin', 'scrypt:32768:8:1$PzN3... (mock hash, we will use plain for testing if needed)', 'admin@vit.edu', 'Admin',
-- Let Antigravity know if you want to bypass hashing for testing, or I can give you a python script to generate a real hash
```

plsql.sql

SQL - PL/SQL Procedures and Triggers

```sql
-- ============================================================================
-- Enterprise Library Management System - Advanced PL/SQL Architecture
-- Target Database: Oracle Database 21c
-- Components: Packages, Explicit Cursors, Exception Handling, & Triggers
-- ============================================================================

SET SERVEROUTPUT ON;

-- ============================================================================
-- PART 1: PACKAGE SPECIFICATION
-- ============================================================================
CREATE OR REPLACE PACKAGE LIBRARY_OPS_PKG AS
    /*
     * Package: LIBRARY_OPS_PKG
     * Purpose: Encapsulates transactional business logic, domain rules, and
     *          integrity checks for book issuance, returns, and financial audits.
     */

    -- Custom Exception Declarations for Standardized Error Handling
    e_user_suspended        EXCEPTION;
    e_unpaid_fines_exist    EXCEPTION;
    e_book_unavailable      EXCEPTION;
    e_max_loans_exceeded    EXCEPTION;
    e_loan_already_returned EXCEPTION;

    PRAGMA EXCEPTION_INIT(e_user_suspended, -20001);
    PRAGMA EXCEPTION_INIT(e_unpaid_fines_exist, -20002);
    PRAGMA EXCEPTION_INIT(e_book_unavailable, -20003);
    PRAGMA EXCEPTION_INIT(e_max_loans_exceeded, -20004);
    PRAGMA EXCEPTION_INIT(e_loan_already_returned, -20006);

    -- Constant Definition: Daily fine assessment rate (in currency units)
    c_daily_fine_rate CONSTANT NUMBER(5, 2) := 10.00;

    -- Constant Definition: Maximum allowable concurrent active loans per user
    c_max_borrow_limit CONSTANT PLS_INTEGER := 3;

    /*
     * Function: fn_Get_Unpaid_Fines
     * Description: Aggregates total outstanding fines for a specified user
     *              using an explicit cursor.
     * Parameter: p_user_id - Unique identifier of the borrower.
     * Returns: Total outstanding fine balance (NUMBER). Returns 0 if none exist.
     */
    FUNCTION fn_Get_Unpaid_Fines(
        p_user_id IN Users.User_ID%TYPE
    ) RETURN NUMBER;

    /*
     * Procedure: sp_IssueBook
     * Description: Validates patron eligibility and inventory status before
     *              creating a loan record and transitioning copy inventory.
     * Parameters:
     *   - p_copy_id: Inventory copy identifier.
     *   - p_borrower_id: User ID of patron borrowing the item.
     *   - p_issued_by: User ID of staff/librarian authorizing issuance.
     */
    PROCEDURE sp_IssueBook(
        p_copy_id     IN Book_Copies.Copy_ID%TYPE,
        p_borrower_id IN Users.User_ID%TYPE,
        p_issued_by   IN Users.User_ID%TYPE
    );

    /*
     * Procedure: sp_ReturnBook
     * Description: Marks loan as returned, restores inventory copy status to
     *              'Available', and automatically assesses overdue penalties.
     * Parameter: p_loan_id - Loan record identifier.
     */
```

```
    PROCEDURE sp_ReturnBook(
        p_loan_id IN Loans.Loan_ID%TYPE
    );

END LIBRARY_OPS_PKG;
/

-- ============================================================================
-- PART 1: PACKAGE BODY
-- ============================================================================
CREATE OR REPLACE PACKAGE BODY LIBRARY_OPS_PKG AS

    -- ------------------------------------------------------------------------
    -- FUNCTION: fn_Get_Unpaid_Fines
    -- ------------------------------------------------------------------------
    FUNCTION fn_Get_Unpaid_Fines(
        p_user_id IN Users.User_ID%TYPE
    ) RETURN NUMBER IS
        -- Variable to accumulate the fine total
        v_total_unpaid NUMBER(10, 2) := 0;
        v_fine_amount  Fines.Fine_Amount%TYPE;

        /*
         * Explicit Cursor: Traverses individual unpaid fine line items
         * linked to the specified borrower through the Loans join table.
         */
        CURSOR cur_unpaid_fines IS
            SELECT f.Fine_Amount
            FROM Fines f
            JOIN Loans l ON f.Loan_ID = l.Loan_ID
            WHERE l.Borrower_ID = p_user_id
              AND f.Payment_Status = 'Unpaid';
    BEGIN
        -- Open explicit cursor
        OPEN cur_unpaid_fines;
        LOOP
            FETCH cur_unpaid_fines INTO v_fine_amount;
            EXIT WHEN cur_unpaid_fines%NOTFOUND;

            -- Accumulate balance
            v_total_unpaid := v_total_unpaid + NVL(v_fine_amount, 0);
        END LOOP;
        CLOSE cur_unpaid_fines;

        RETURN v_total_unpaid;

    EXCEPTION
        WHEN OTHERS THEN
            -- Defensive programming: Ensure cursor is closed before propagating
            IF cur_unpaid_fines%ISOPEN THEN
                CLOSE cur_unpaid_fines;
            END IF;
            RAISE_APPLICATION_ERROR(
                -20099,
                'Error calculating unpaid fines for User ID ' || p_user_id || ': ' || SQLERRM
            );
    END fn_Get_Unpaid_Fines;

    -- ------------------------------------------------------------------------
    -- PROCEDURE: sp_IssueBook
    -- ------------------------------------------------------------------------
    PROCEDURE sp_IssueBook(
        p_copy_id     IN Book_Copies.Copy_ID%TYPE,
        p_borrower_id IN Users.User_ID%TYPE,
        p_issued_by   IN Users.User_ID%TYPE
    ) IS
        v_account_status    Users.Account_Status%TYPE;
        v_unpaid_fines      NUMBER(10, 2);
        v_copy_status       Book_Copies.Status%TYPE;
        v_active_loans      PLS_INTEGER := 0;
        v_issuer_role       Users.Role%TYPE;
    BEGIN
```

```
142         -- 1. Verify borrower exists and validate account active status
143         BEGIN
144             SELECT Account_Status
145             INTO v_account_status
146             FROM Users
147             WHERE User_ID = p_borrower_id;
148         EXCEPTION
149             WHEN NO_DATA_FOUND THEN
150                 RAISE_APPLICATION_ERROR(-20010, 'Borrower User ID ' || p_borrower_id || ' does not exist.');
151         END;
152
153         IF v_account_status != 'Active' THEN
154             RAISE_APPLICATION_ERROR(
155                 -20001,
156                 'Transaction Blocked: Borrower account status is ' || v_account_status || '. Must be Active.'
157             );
158         END IF;
159
160         -- 2. Verify staff issuer credentials
161         BEGIN
162             SELECT Role
163             INTO v_issuer_role
164             FROM Users
165             WHERE User_ID = p_issued_by AND Account_Status = 'Active';
166
167             IF v_issuer_role NOT IN ('Admin', 'Librarian', 'Staff') THEN
168                 RAISE_APPLICATION_ERROR(-20011, 'Unauthorized: Issuing user lacks administrative privileges.');
169             END IF;
170         EXCEPTION
171             WHEN NO_DATA_FOUND THEN
172                 RAISE_APPLICATION_ERROR(-20012, 'Issuing user ID ' || p_issued_by || ' not found or inactive.');
173         END;
174
175         -- 3. Check for any delinquent fines via explicit cursor function
176         v_unpaid_fines := fn_Get_Unpaid_Fines(p_borrower_id);
177         IF v_unpaid_fines > 0 THEN
178             RAISE_APPLICATION_ERROR(
179                 -20002,
180                 'Transaction Blocked: Patron has outstanding unpaid fines of Rs. ' ||
181                 TO_CHAR(v_unpaid_fines, 'FM999,990.00') || '. Clearance required.'
182             );
183         END IF;
184
185         -- 4. Check book inventory copy status with row-level lock (pessimistic concurrency)
186         BEGIN
187             SELECT Status
188             INTO v_copy_status
189             FROM Book_Copies
190             WHERE Copy_ID = p_copy_id
191             FOR UPDATE;
192         EXCEPTION
193             WHEN NO_DATA_FOUND THEN
194                 RAISE_APPLICATION_ERROR(-20013, 'Physical Copy ID ' || p_copy_id || ' does not exist in inventory.');
195         END;
196
197         IF v_copy_status != 'Available' THEN
198             RAISE_APPLICATION_ERROR(
199                 -20003,
200                 'Transaction Blocked: Physical Copy ID ' || p_copy_id ||
201                 ' is currently unavailable (Status: ' || v_copy_status || ').'
202             );
203         END IF;
204
205         -- 5. Validate patron borrowing quota limit (Maximum 3 concurrent active loans)
206         SELECT COUNT(*)
207         INTO v_active_loans
208         FROM Loans
209         WHERE Borrower_ID = p_borrower_id
210           AND Return_Date IS NULL;
211
212         IF v_active_loans >= c_max_borrow_limit THEN
213             RAISE_APPLICATION_ERROR(
```

```
                -20004,
                'Transaction Blocked: Patron has reached the maximum quota of ' ||
                c_max_borrow_limit || ' concurrent active loans.'
            );
        END IF;

        -- 6. Insert new loan transaction (Standard 14-day checkout window)
        INSERT INTO Loans (
            Copy_ID,
            Borrower_ID,
            Issued_By,
            Issue_Date,
            Due_Date,
            Return_Date
        ) VALUES (
            p_copy_id,
            p_borrower_id,
            p_issued_by,
            SYSDATE,
            SYSDATE + 14,
            NULL
        );

        -- 7. Update copy inventory state to 'Borrowed'
        UPDATE Book_Copies
        SET Status = 'Borrowed'
        WHERE Copy_ID = p_copy_id;

        -- Commit atomic transaction
        COMMIT;

        DBMS_OUTPUT.PUT_LINE(
            'SUCCESS: Book Copy #' || p_copy_id || ' successfully issued to Patron #' ||
            p_borrower_id || '. Due Date: ' || TO_CHAR(SYSDATE + 14, 'YYYY-MM-DD')
        );

    EXCEPTION
        WHEN OTHERS THEN
            -- Rollback in-flight locks and mutations on unexpected failure
            ROLLBACK;
            RAISE;
    END sp_IssueBook;

    -- ------------------------------------------------------------------------
    -- PROCEDURE: sp_ReturnBook
    -- ------------------------------------------------------------------------
    PROCEDURE sp_ReturnBook(
        p_loan_id IN Loans.Loan_ID%TYPE
    ) IS
        v_copy_id       Book_Copies.Copy_ID%TYPE;
        v_due_date      Loans.Due_Date%TYPE;
        v_return_date   Loans.Return_Date%TYPE;
        v_days_overdue  PLS_INTEGER := 0;
        v_fine_amount   NUMBER(10, 2) := 0;
    BEGIN
        -- 1. Retrieve and lock active loan record for deterministic update
        BEGIN
            SELECT Copy_ID, Due_Date, Return_Date
            INTO v_copy_id, v_due_date, v_return_date
            FROM Loans
            WHERE Loan_ID = p_loan_id
            FOR UPDATE;
        EXCEPTION
            WHEN NO_DATA_FOUND THEN
                RAISE_APPLICATION_ERROR(-20014, 'Loan transaction ID ' || p_loan_id || ' does not exist.');
        END;

        -- Prevent duplicate return processing
        IF v_return_date IS NOT NULL THEN
            RAISE_APPLICATION_ERROR(
                -20006,
                'Transaction Blocked: Loan ID ' || p_loan_id ||
```

```
                ' was already completed on ' || TO_CHAR(v_return_date, 'YYYY-MM-DD HH24:MI:SS') || '.'
            );
        END IF;

        -- 2. Mark loan as returned
        UPDATE Loans
        SET Return_Date = SYSDATE
        WHERE Loan_ID = p_loan_id;

        -- 3. Return physical inventory copy status back to 'Available'
        UPDATE Book_Copies
        SET Status = 'Available'
        WHERE Copy_ID = v_copy_id;

        -- 4. Audit overdue status and assess automated penalty if applicable
        IF TRUNC(SYSDATE) > TRUNC(v_due_date) THEN
            -- Calculate calendar days overdue
            v_days_overdue := TRUNC(SYSDATE) - TRUNC(v_due_date);
            v_fine_amount  := v_days_overdue * c_daily_fine_rate;

            -- Insert calculated fine
            INSERT INTO Fines (
                Loan_ID,
                Fine_Amount,
                Payment_Status
            ) VALUES (
                p_loan_id,
                v_fine_amount,
                'Unpaid'
            );

            DBMS_OUTPUT.PUT_LINE(
                'PENALTY ASSESSED: Loan #' || p_loan_id || ' is ' || v_days_overdue ||
                ' day(s) overdue. Fine of Rs. ' || TO_CHAR(v_fine_amount, 'FM999,990.00') || ' recorded.'
            );
        ELSE
            DBMS_OUTPUT.PUT_LINE('SUCCESS: Book returned on time. No overdue penalties applied.');
        END IF;

        -- Commit atomic transaction
        COMMIT;

    EXCEPTION
        WHEN OTHERS THEN
            ROLLBACK;
            RAISE;
    END sp_ReturnBook;

END LIBRARY_OPS_PKG;
/

-- ============================================================================
-- PART 2: STANDALONE TRIGGERS
-- ============================================================================

-- ----------------------------------------------------------------------------
-- TRIGGER A: trg_Secure_Fines_Deletion
-- Timing: BEFORE DELETE ON Fines (Row-Level)
-- Objective: Enforce immutable ledger integrity by blocking deletion of fine
--            records to prevent concealment or tampering.
-- ----------------------------------------------------------------------------
CREATE OR REPLACE TRIGGER trg_Secure_Fines_Deletion
BEFORE DELETE ON Fines
FOR EACH ROW
BEGIN
    /*
     * Compliance Rule: Library audit governance forbids unauthorized deletion of
     * penalty records. Programmatic administrative purges set CLIENT_INFO = 'ALLOW_DELETE'.
     */
    IF SYS_CONTEXT('USERENV', 'CLIENT_INFO') = 'ALLOW_DELETE' THEN
        RETURN;
    END IF;
```

```
358
359      RAISE_APPLICATION_ERROR(
360          -20005,
361          'Security Violation: Fines cannot be deleted. They must be updated to ''Paid'' or ''Waived''.'
362      );
363  END;
364  /
365
366  -- ------------------------------------------------------------------------
367  -- TRIGGER B: trg_Audit_User_Suspension
368  -- Timing: AFTER UPDATE OF Account_Status ON Users (Row-Level)
369  -- Objective: Emits an audit alert whenever an administrative action changes
370  --            a user's account status into 'Suspended'.
371  -- ------------------------------------------------------------------------
372  CREATE OR REPLACE TRIGGER trg_Audit_User_Suspension
373  AFTER UPDATE OF Account_Status ON Users
374  FOR EACH ROW
375  WHEN (NEW.Account_Status = 'Suspended' AND (OLD.Account_Status IS NULL OR OLD.Account_Status != 'Suspended'))
376  BEGIN
377      /*
378       * Security Audit Logger:
379       * Outputs notification of suspension with account details and exact timestamp.
380       */
381      DBMS_OUTPUT.PUT_LINE(
382          '>>> [SECURITY AUDIT ALERT] User ''' || :NEW.Username ||
383          ''' (User_ID: ' || :NEW.User_ID || ') was SUSPENDED on ' ||
384          TO_CHAR(SYSDATE, 'YYYY-MM-DD "at" HH24:MI:SS') || ' <<<'
385      );
386  END;
387  /
```

requirements.txt

Text - Python Dependencies

```
1  ■blinker==1.9.0
2  cachelib==0.17.0
3  cffi==2.1.1
4  click==8.5.0
5  cryptography==50.0.2
6  Flask==3.1.3
7  Flask-Session==0.8.0
8  itsdangerous==2.2.0
9  Jinja2==3.1.6
10 MarkupSafe==3.0.4
11 msgspec==0.22.0
12 oracledb==26.0.1
13 pycparser==3.0
14 typing_extensions==4.16.0
15 Werkzeug==3.1.9
```

seed_all_data.py

Python - Database Seeder

```python
"""
Database Seeding Script for Oracle 21c Enterprise Library System
Populates at least 10 high-quality records in each of the 9 normalized tables.
"""

import oracledb
from werkzeug.security import generate_password_hash
from datetime import datetime, timedelta

def run_seed():
    conn = oracledb.connect(user="system", password="root", dsn="localhost:1521/XEPDB1")
    cursor = conn.cursor()
    print("Connected to Oracle Database. Beginning comprehensive seeding...")

    try:
        # ======================================================================
        # 1. SEED USERS (14 Users across all 4 RBAC Roles)
        # ======================================================================
        print("\n[1/9] Seeding Users...")
        users = [
            ("admin", "admin123", "admin@vit.edu", "Admin"),
            ("sysadmin", "admin123", "sysadmin@vit.edu", "Admin"),
            ("librarian", "librarian123", "librarian@vit.edu", "Librarian"),
            ("head_librarian", "librarian123", "head_lib@vit.edu", "Librarian"),
            ("clara_lib", "librarian123", "clara@vit.edu", "Librarian"),
            ("staff", "staff123", "staff@vit.edu", "Staff"),
            ("prof_turing", "staff123", "turing@vit.edu", "Staff"),
            ("prof_shannon", "staff123", "shannon@vit.edu", "Staff"),
            ("student", "student123", "student@vit.edu", "Student"),
            ("alex_stu", "student123", "alex@vit.edu", "Student"),
            ("priya_stu", "student123", "priya@vit.edu", "Student"),
            ("rahul_stu", "student123", "rahul@vit.edu", "Student"),
            ("emily_stu", "student123", "emily@vit.edu", "Student"),
            ("marcus_stu", "student123", "marcus@vit.edu", "Student"),
        ]

        merge_user_sql = """
            MERGE INTO Users u
            USING (SELECT :username AS uname FROM dual) d
            ON (u.Username = d.uname)
            WHEN MATCHED THEN
                UPDATE SET u.Password_Hash = :pw, u.Email = :email, u.Role = :role, u.Account_Status = 'Active'
            WHEN NOT MATCHED THEN
                INSERT (Username, Password_Hash, Email, Role, Account_Status)
                VALUES (:username, :pw, :email, :role, 'Active')
        """
        for u in users:
            pw_hash = generate_password_hash(u[1])
            cursor.execute(merge_user_sql, {
                "username": u[0],
                "pw": pw_hash,
                "email": u[2],
                "role": u[3]
            })
        print(f" -> Seeded {len(users)} users.")

        # Cache User IDs
        cursor.execute("SELECT Username, User_ID FROM Users")
        user_id_map = {row[0]: row[1] for row in cursor.fetchall()}

        # ======================================================================
        # 2. SEED CATEGORIES (10 Records)
        # ======================================================================
        print("[2/9] Seeding Categories...")
        categories = [
            ("Computer Science & AI", "CS"),
            ("Physics & Astronomy", "PHY"),
            ("Mathematics & Statistics", "MAT"),
            ("Electrical Engineering", "EE"),
```

```
            ("Mechanical Engineering", "ME"),
            ("World History & Civilizations", "HIS"),
            ("Philosophy & Ethics", "PHI"),
            ("Classical Literature", "LIT"),
            ("Economics & Public Policy", "ECO"),
            ("Data Architecture & Databases", "DB")
        ]

        merge_cat_sql = """
            MERGE INTO Categories c
            USING (SELECT :name AS cname FROM dual) d
            ON (c.Name = d.cname)
            WHEN MATCHED THEN
                UPDATE SET c.Section_Code = :code
            WHEN NOT MATCHED THEN
                INSERT (Name, Section_Code) VALUES (:name, :code)
        """
        for cat in categories:
            cursor.execute(merge_cat_sql, {"name": cat[0], "code": cat[1]})

        cursor.execute("SELECT Name, Category_ID FROM Categories")
        category_id_map = {row[0]: row[1] for row in cursor.fetchall()}
        print(f" -> Seeded {len(categories)} categories.")

        # ======================================================================
        # 3. SEED PUBLISHERS (10 Records)
        # ======================================================================
        print("[3/9] Seeding Publishers...")
        publishers = [
            ("O'Reilly Media", "contact@oreilly.com"),
            ("Pearson Education", "support@pearson.com"),
            ("MIT Press", "info@mitpress.mit.edu"),
            ("McGraw-Hill Education", "help@mheducation.com"),
            ("Addison-Wesley Professional", "inquiries@addisonwesley.com"),
            ("Cambridge University Press", "academic@cambridge.org"),
            ("Oxford University Press", "custserv@oup.com"),
            ("Springer Nature", "contact@springernature.com"),
            ("John Wiley & Sons", "support@wiley.com"),
            ("Morgan Kaufmann", "info@mkp.com")
        ]

        for pub in publishers:
            cursor.execute("""
                MERGE INTO Publishers p
                USING (SELECT :pname AS pname FROM dual) d
                ON (p.Name = d.pname)
                WHEN MATCHED THEN
                    UPDATE SET p.Contact_Email = :email
                WHEN NOT MATCHED THEN
                    INSERT (Name, Contact_Email) VALUES (:pname, :email)
            """, {"pname": pub[0], "email": pub[1]})

        cursor.execute("SELECT Name, Publisher_ID FROM Publishers")
        publisher_id_map = {row[0]: row[1] for row in cursor.fetchall()}
        print(f" -> Seeded {len(publishers)} publishers.")

        # ======================================================================
        # 4. SEED AUTHORS (10 Records)
        # ======================================================================
        print("[4/9] Seeding Authors...")
        authors = [
            ("Donald", "Knuth"),
            ("Martin", "Fowler"),
            ("Robert", "Martin"),
            ("Eric", "Evans"),
            ("Abraham", "Silberschatz"),
            ("Andrew", "Tanenbaum"),
            ("Thomas", "Cormen"),
            ("Stuart", "Russell"),
            ("Peter", "Norvig"),
            ("Richard", "Feynman")
        ]
```

```
142
143         for auth in authors:
144             cursor.execute("""
145                 MERGE INTO Authors a
146                 USING (SELECT :fn AS fn, :ln AS ln FROM dual) d
147                 ON (a.First_Name = d.fn AND a.Last_Name = d.ln)
148                 WHEN NOT MATCHED THEN
149                     INSERT (First_Name, Last_Name) VALUES (:fn, :ln)
150             """, {"fn": auth[0], "ln": auth[1]})
151
152         cursor.execute("SELECT First_Name || ' ' || Last_Name, Author_ID FROM Authors")
153         author_id_map = {row[0]: row[1] for row in cursor.fetchall()}
154         print(f" -> Seeded {len(authors)} authors.")
155
156         # ======================================================================
157         # 5. SEED BOOKS (10 Records)
158         # ======================================================================
159         print("[5/9] Seeding Books...")
160         books = [
161             ("978-1449373320", "Designing Data-Intensive Applications", 2017, "Data Architecture & Databases", "O'Reilly Med
162             ("978-0132350884", "Clean Code: Agile Software Craftsmanship", 2008, "Computer Science & AI", "Pearson Education
163             ("978-0262033848", "Introduction to Algorithms (CLRS)", 2009, "Computer Science & AI", "MIT Press", ["Thomas Cor
164             ("978-0136042594", "Artificial Intelligence: A Modern Approach", 2020, "Computer Science & AI", "Pearson Educati
165             ("978-0078022159", "Database System Concepts (7th Ed)", 2019, "Data Architecture & Databases", "McGraw-Hill Educ
166             ("978-0133591620", "Modern Operating Systems (4th Ed)", 2014, "Computer Science & AI", "Addison-Wesley Professio
167             ("978-0132126953", "Computer Networks (5th Ed)", 2010, "Computer Science & AI", "Pearson Education", ["Andrew Ta
168             ("978-0321125217", "Domain-Driven Design: Tackling Complexity", 2003, "Computer Science & AI", "Addison-Wesley P
169             ("978-0201896831", "The Art of Computer Programming, Vol 1", 1997, "Mathematics & Statistics", "Addison-Wesley P
170             ("978-0465023820", "The Feynman Lectures on Physics", 2011, "Physics & Astronomy", "Addison-Wesley Professional"
171         ]
172
173         for b in books:
174             cat_id = category_id_map.get(b[3])
175             pub_id = publisher_id_map.get(b[4])
176             cursor.execute("""
177                 MERGE INTO Books b
178                 USING (SELECT :isbn AS isbn FROM dual) d
179                 ON (b.ISBN = d.isbn)
180                 WHEN MATCHED THEN
181                     UPDATE SET b.Title = :title, b.Publication_Year = :pyear, b.Category_ID = :cat_id, b.Publisher_ID = :pub
182                 WHEN NOT MATCHED THEN
183                     INSERT (ISBN, Title, Publication_Year, Category_ID, Publisher_ID)
184                     VALUES (:isbn, :title, :pyear, :cat_id, :pub_id)
185             """, {
186                 "isbn": b[0],
187                 "title": b[1],
188                 "pyear": b[2],
189                 "cat_id": cat_id,
190                 "pub_id": pub_id
191             })
192
193         cursor.execute("SELECT ISBN, Book_ID FROM Books")
194         book_id_map = {row[0]: row[1] for row in cursor.fetchall()}
195         print(f" -> Seeded {len(books)} books.")
196
197         # ======================================================================
198         # 6. SEED BOOK_AUTHORS (Many-to-Many Relationship, 11+ Records)
199         # ======================================================================
200         print("[6/9] Seeding Book_Authors...")
201         for b in books:
202             b_id = book_id_map.get(b[0])
203             for a_name in b[5]:
204                 a_id = author_id_map.get(a_name)
205                 if b_id and a_id:
206                     cursor.execute("""
207                         MERGE INTO Book_Authors ba
208                         USING (SELECT :b_id AS bid, :a_id AS aid FROM dual) d
209                         ON (ba.Book_ID = d.bid AND ba.Author_ID = d.aid)
210                         WHEN NOT MATCHED THEN
211                             INSERT (Book_ID, Author_ID) VALUES (:b_id, :a_id)
212                     """, {"b_id": b_id, "a_id": a_id})
213         print(" -> Seeded Book_Authors mappings.")
```

```
214
215         # ======================================================================
216         # 7. SEED BOOK_COPIES (At least 18 Physical Copies)
217         # ======================================================================
218         print("[7/9] Seeding Book_Copies...")
219         cursor.execute("SELECT COUNT(*) FROM Book_Copies")
220         existing_copies = cursor.fetchone()[0]
221
222         if existing_copies < 15:
223             # Create 2 physical copies per book
224             for isbn, b_id in book_id_map.items():
225                 cursor.execute("""
226                     INSERT INTO Book_Copies (Book_ID, Status, Condition)
227                     VALUES (:b_id, 'Available', 'New')
228                 """, {"b_id": b_id})
229                 cursor.execute("""
230                     INSERT INTO Book_Copies (Book_ID, Status, Condition)
231                     VALUES (:b_id, 'Available', 'Good')
232                 """, {"b_id": b_id})
233
234         cursor.execute("SELECT Copy_ID, Book_ID FROM Book_Copies ORDER BY Copy_ID ASC")
235         copy_rows = cursor.fetchall()
236         print(f" -> Total physical copies available: {len(copy_rows)}.")
237
238         # ======================================================================
239         # 8. SEED LOANS (At least 10 Loans across active, overdue, returned)
240         # ======================================================================
241         print("[8/9] Seeding Loans...")
242         cursor.execute("SELECT COUNT(*) FROM Loans")
243         existing_loans = cursor.fetchone()[0]
244
245         librarian_id = user_id_map.get("librarian", user_id_map.get("admin"))
246         student_id   = user_id_map.get("student")
247         alex_id      = user_id_map.get("alex_stu")
248         priya_id     = user_id_map.get("priya_stu")
249         rahul_id     = user_id_map.get("rahul_stu")
250         staff_id     = user_id_map.get("staff")
251         turing_id    = user_id_map.get("prof_turing")
252
253         now = datetime.now()
254
255         # Define 12 diverse loan scenarios
256         # (copy_idx, borrower_id, issue_days_ago, due_days_ago, return_days_ago or None)
257         loan_configs = [
258             # 1-3: Active loans on time
259             (0, student_id, 3, -11, None),
260             (1, alex_id,    5, -9,  None),
261             (2, staff_id,   7, -7,  None),
262
263             # 4-6: Active loans OVERDUE (past due, unreturned)
264             (3, student_id, 25, 11, None),
265             (4, rahul_id,   20, 6,  None),
266             (5, priya_id,   30, 16, None),
267
268             # 7-9: Completed on time
269             (6, student_id, 30, 16, 17),
270             (7, alex_id,    20, 6,  7),
271             (8, turing_id,  15, 1,  2),
272
273             # 10-12: Completed LATE (returned past due date)
274             (9,  student_id, 40, 26, 20),
275             (10, rahul_id,   35, 21, 14),
276             (11, priya_id,   50, 36, 25),
277         ]
278
279         if existing_loans < 10 and len(copy_rows) >= 12:
280             for cfg in loan_configs:
281                 c_id = copy_rows[cfg[0]][0]
282                 b_id = cfg[1]
283                 issue_dt = now - timedelta(days=cfg[2])
284                 due_dt   = now - timedelta(days=cfg[3])
285                 ret_dt   = (now - timedelta(days=cfg[4])) if cfg[4] is not None else None
```

```
286
287                 cursor.execute("""
288                     INSERT INTO Loans (Copy_ID, Borrower_ID, Issued_By, Issue_Date, Due_Date, Return_Date)
289                     VALUES (:copy_id, :borrower_id, :issued_by, :issue_date, :due_date, :return_date)
290                 """, {
291                     "copy_id": c_id,
292                     "borrower_id": b_id,
293                     "issued_by": librarian_id,
294                     "issue_date": issue_dt,
295                     "due_date": due_dt,
296                     "return_date": ret_dt
297                 })
298
299                 # If currently unreturned, mark copy as 'Borrowed'
300                 if ret_dt is None:
301                     cursor.execute("UPDATE Book_Copies SET Status = 'Borrowed' WHERE Copy_ID = :cid", {"cid": c_id})
302
303         cursor.execute("SELECT Loan_ID, Borrower_ID, Issue_Date, Due_Date, Return_Date FROM Loans ORDER BY Loan_ID ASC")
304         loans_list = cursor.fetchall()
305         print(f" -> Total loan transactions recorded: {len(loans_list)}.")
306
307         # ======================================================================
308         # 9. SEED FINES (At least 10 Fine records: Paid, Unpaid, Waived)
309         # ======================================================================
310         print("[9/9] Seeding Fines...")
311         cursor.execute("SELECT COUNT(*) FROM Fines")
312         existing_fines = cursor.fetchone()[0]
313
314         if existing_fines < 10 and len(loans_list) >= 10:
315             # Assign fines to late and overdue loans
316             fine_specs = [
317                 # loan_idx, amount, status
318                 (3, 110.00, "Unpaid"),  # Active overdue loan 11 days
319                 (4, 60.00,  "Unpaid"),  # Active overdue loan 6 days
320                 (5, 160.00, "Unpaid"),  # Active overdue loan 16 days
321                 (9, 60.00,  "Paid"),    # Returned late by 6 days
322                 (10, 70.00, "Paid"),    # Returned late by 7 days
323                 (11, 110.00,"Waived"),  # Returned late by 11 days (Waived by librarian)
324                 (6, 25.00,  "Paid"),    # Administrative replacement fee
325                 (7, 40.00,  "Unpaid"),  # Minor damage assessment
326                 (8, 15.00,  "Paid"),    # Minor delay fine
327                 (0, 50.00,  "Unpaid"),  # Historical penalty
328             ]
329
330             for spec in fine_specs:
331                 l_id = loans_list[spec[0]][0]
332                 cursor.execute("""
333                     MERGE INTO Fines f
334                     USING (SELECT :loan_id AS lid FROM dual) d
335                     ON (f.Loan_ID = d.lid)
336                     WHEN MATCHED THEN
337                         UPDATE SET f.Fine_Amount = :amt, f.Payment_Status = :status
338                     WHEN NOT MATCHED THEN
339                         INSERT (Loan_ID, Fine_Amount, Payment_Status)
340                         VALUES (:loan_id, :amt, :status)
341                 """, {
342                     "loan_id": l_id,
343                     "amt": spec[1],
344                     "status": spec[2]
345                 })
346
347         cursor.execute("SELECT COUNT(*) FROM Fines")
348         final_fines_count = cursor.fetchone()[0]
349         print(f" -> Total disciplinary fines recorded: {final_fines_count}.")
350
351         # Final commit
352         conn.commit()
353         print("\n[SUCCESS] All 9 tables seeded with 10+ records each!")
354
355     except Exception as e:
356         conn.rollback()
357         print(f"\n[ERROR] Seeding aborted: {e}")
```

```
358          raise
359      finally:
360          cursor.close()
361          conn.close()
362
363  if __name__ == "__main__":
364      run_seed()
```

test_user_requirements.py

Python - Test Suite

```python
import oracledb
from app import app

def run_suite():
    client = app.test_client()
    conn = oracledb.connect(user="system", password="root", dsn="localhost:1521/XEPDB1")
    cur = conn.cursor()

    print("======================================================================")
    print("TEST SUITE: Zero Fine, Borrowing Block for Overdue/Unpaid, Brand Cleanup")
    print("======================================================================")

    # ----------------------------------------------------------------------
    # TEST 1: Removal of 'Oracle Database 21c Engine' & Top-Left '21c' Badge
    # ----------------------------------------------------------------------
    print("\n--- 1. Testing Removal of 'Oracle Database 21c Engine' & '21c' ---")
    with client.session_transaction() as sess:
        sess["user_id"] = 1
        sess["username"] = "admin"
        sess["role"] = "Admin"

    res_dash = client.get("/dashboard")
    assert res_dash.status_code == 200
    html_dash = res_dash.data.decode("utf-8")

    assert "Oracle Database 21c Engine" not in html_dash, "Found 'Oracle Database 21c Engine' in dashboard!"
    assert "21c Engine" not in html_dash, "Found '21c Engine' in dashboard!"
    assert '<span class="brand-badge">21c</span>' not in html_dash, "Found top-left '21c' brand badge in layout!"
    assert "brand-badge" not in html_dash, "Found brand-badge in layout!"
    assert "VITCC LIBRARY" in html_dash, "Missing VITCC LIBRARY title in layout"
    print("PASS: 'Oracle Database 21c Engine' and top-left '21c' badge are completely removed.")

    # ----------------------------------------------------------------------
    # TEST 2: 'Zero Fine' text in Fine Status for Admin, Librarian, Student, Staff
    # ----------------------------------------------------------------------
    print("\n--- 2. Testing 'Zero Fine' Text (Instead of 'Zero Outstanding Fine') ---")
    assert "Zero Outstanding Fine" not in html_dash, "Found 'Zero Outstanding Fine' in Admin dashboard!"
    assert "Zero Outstanding Fines" not in html_dash, "Found 'Zero Outstanding Fines' in Admin dashboard!"
    assert "Zero Fine" in html_dash, "Expected 'Zero Fine' in Admin active loans ledger!"
    print("PASS: Admin Dashboard displays 'Zero Fine' for loans without fines.")

    # Check Student Dashboard & Ledgers
    cur.execute("""
        SELECT u.User_ID, u.Username
        FROM Users u
        WHERE u.Role = 'Student'
          AND NVL((SELECT SUM(f.Fine_Amount) FROM Fines f JOIN Loans l ON f.Loan_ID = l.Loan_ID WHERE l.Borrower_ID = u.User
          AND ROWNUM = 1
    """)
    clean_student = cur.fetchone()
    stu_id = clean_student[0]
    stu_name = clean_student[1]

    with client.session_transaction() as sess:
        sess["user_id"] = stu_id
        sess["username"] = stu_name
        sess["role"] = "Student"

    res_stu = client.get("/dashboard")
    assert res_stu.status_code == 200
    html_stu = res_stu.data.decode("utf-8")
    assert "Zero Outstanding Fine" not in html_stu, "Found 'Zero Outstanding Fine' in Student dashboard!"
    assert "Zero Outstanding Fines" not in html_stu, "Found 'Zero Outstanding Fines' in Student dashboard!"
    assert "Zero Fine" in html_stu, "Expected 'Zero Fine' in Student stats / ledger!"

    res_fines = client.get("/my_fines")
    assert res_fines.status_code == 200
    html_fines = res_fines.data.decode("utf-8")
    assert "Zero Outstanding Fine" not in html_fines, "Found 'Zero Outstanding Fine' in /my_fines!"
```

```
    assert "Zero Outstanding Fines" not in html_fines, "Found 'Zero Outstanding Fines' in /my_fines!"
    print(f"PASS: Student view ({stu_name}) and /my_fines displays 'Zero Fine'.")

    # Check Staff Dashboard
    cur.execute("SELECT u.User_ID, u.Username FROM Users u WHERE u.Role = 'Staff' AND ROWNUM = 1")
    staff = cur.fetchone()
    staff_id = staff[0]
    staff_name = staff[1]

    with client.session_transaction() as sess:
        sess["user_id"] = staff_id
        sess["username"] = staff_name
        sess["role"] = "Staff"

    res_staff = client.get("/dashboard")
    assert res_staff.status_code == 200
    html_staff = res_staff.data.decode("utf-8")
    assert "Zero Outstanding Fine" not in html_staff, "Found 'Zero Outstanding Fine' in Staff dashboard!"
    assert "Zero Outstanding Fines" not in html_staff, "Found 'Zero Outstanding Fines' in Staff dashboard!"
    assert "Zero Fine" in html_staff, "Expected 'Zero Fine' in Staff dashboard!"
    print(f"PASS: Staff view ({staff_name}) displays 'Zero Fine'.")

    # ------------------------------------------------------------------------
    # TEST 3: Borrowing Block for Overdue Loans and Unpaid Fines
    # ------------------------------------------------------------------------
    print("\n--- 3. Testing Borrowing Block for Overdue Loans & Unpaid Fines ---")

    # Pick an available copy and category
    cur.execute("""
        SELECT bc.Copy_ID, bc.Book_ID, b.Category_ID
        FROM Book_Copies bc
        JOIN Books b ON bc.Book_ID = b.Book_ID
        WHERE bc.Status = 'Available'
        ORDER BY bc.Copy_ID
    """)
    avail_row = cur.fetchone()
    test_copy_id = avail_row[0]
    test_book_id = avail_row[1]
    test_cat_id = avail_row[2]

    temp_uid = None
    temp_lid = None
    temp_fid = None
    temp_rid = None

    # Create a temporary student user for isolated testing
    cur.execute("SELECT NVL(MAX(User_ID), 0) + 1 FROM Users")
    temp_uid = cur.fetchone()[0]
    cur.execute("""
        INSERT INTO Users (User_ID, Username, Email, Password_Hash, Role, Account_Status)
        VALUES (:p_uid, :p_uname, :p_email, :p_pwd, :p_role, :p_status)
    """, {
        "p_uid": temp_uid,
        "p_uname": "test_delinquent",
        "p_email": "test_delinquent@vit.ac.in",
        "p_pwd": "TestPass@123",
        "p_role": "Student",
        "p_status": "Active"
    })
    conn.commit()

    try:
        # Scenario A: Patron has an OVERDUE loan
        # Insert overdue loan for temp user (due 5 days ago, return_date NULL)
        cur.execute("SELECT NVL(MAX(Loan_ID), 0) + 1 FROM Loans")
        temp_lid = cur.fetchone()[0]
        cur.execute("""
            INSERT INTO Loans (Loan_ID, Copy_ID, Borrower_ID, Issued_By, Issue_Date, Due_Date, Return_Date)
            VALUES (:p_lid, :p_cid, :p_bid, 1, SYSDATE - 19, SYSDATE - 5, NULL)
        """, {"p_lid": temp_lid, "p_cid": test_copy_id, "p_bid": temp_uid})
        cur.execute("UPDATE Book_Copies SET Status = 'Borrowed' WHERE Copy_ID = :p_cid", {"p_cid": test_copy_id})
        conn.commit()
```

```
142
143         # Try to request loan as student with overdue loan
144         with client.session_transaction() as sess:
145             sess["user_id"] = temp_uid
146             sess["username"] = "test_delinquent"
147             sess["role"] = "Student"
148
149         # 1. Check category page shows Borrowing Blocked banner and badge
150         res_cat = client.get(f"/category/{test_cat_id}")
151         html_cat = res_cat.data.decode("utf-8")
152         assert "Borrowing Blocked" in html_cat, "Expected 'Borrowing Blocked' banner on category page for overdue user!"
153         assert "overdue" in html_cat.lower(), "Expected overdue explanation in block banner!"
154
155         # 2. Try POST /request_loan
156         res_req = client.post(f"/request_loan/{test_book_id}", follow_redirects=True)
157         html_req = res_req.data.decode("utf-8")
158         assert "Permission Denied" in html_req or "overdue" in html_req.lower(), "Overdue borrower was not blocked from requ
159         print("PASS: Patron with overdue loan is blocked from submitting loan requests.")
160
161         # 3. Check Admin /issue_loan blocks issuing to this borrower
162         with client.session_transaction() as sess:
163             sess["user_id"] = 1
164             sess["username"] = "admin"
165             sess["role"] = "Admin"
166
167         # Pick another available copy
168         cur.execute("SELECT Copy_ID FROM Book_Copies WHERE Status = 'Available' AND ROWNUM = 1")
169         another_copy = cur.fetchone()[0]
170
171         res_issue = client.post("/issue_loan", data={
172             "copy_id": str(another_copy),
173             "borrower_id": str(temp_uid),
174             "due_in_days": "14"
175         }, follow_redirects=True)
176         html_issue = res_issue.data.decode("utf-8")
177         assert "Borrowing Prohibited" in html_issue and "overdue" in html_issue.lower(), "Admin was able to issue loan to pa
178         print("PASS: Admin is blocked from directly issuing books to patron with overdue loans.")
179
180         # Scenario B: Return the overdue book, but an UNPAID FINE remains
181         # Process return and insert unpaid fine
182         cur.execute("UPDATE Loans SET Return_Date = SYSDATE WHERE Loan_ID = :p_lid", {"p_lid": temp_lid})
183         cur.execute("UPDATE Book_Copies SET Status = 'Available' WHERE Copy_ID = :p_cid", {"p_cid": test_copy_id})
184         cur.execute("SELECT NVL(MAX(Fine_ID), 0) + 1 FROM Fines")
185         temp_fid = cur.fetchone()[0]
186         cur.execute("""
187             INSERT INTO Fines (Fine_ID, Loan_ID, Fine_Amount, Payment_Status)
188             VALUES (:p_fid, :p_lid, 50.00, 'Unpaid')
189         """, {"p_fid": temp_fid, "p_lid": temp_lid})
190         conn.commit()
191
192         # Try requesting loan as student with unpaid fine
193         with client.session_transaction() as sess:
194             sess["user_id"] = temp_uid
195             sess["username"] = "test_delinquent"
196             sess["role"] = "Student"
197
198         # 1. Check category page shows Borrowing Blocked banner and badge
199         res_cat2 = client.get(f"/category/{test_cat_id}")
200         html_cat2 = res_cat2.data.decode("utf-8")
201         assert "Borrowing Blocked" in html_cat2, "Expected 'Borrowing Blocked' on category page for user with unpaid fine!"
202         assert "fine" in html_cat2.lower() or "50.00" in html_cat2, "Expected unpaid fine explanation in block banner!"
203
204         # 2. Try POST /request_loan
205         res_req2 = client.post(f"/request_loan/{test_book_id}", follow_redirects=True)
206         html_req2 = res_req2.data.decode("utf-8")
207         assert "Permission Denied" in html_req2 and "fine" in html_req2.lower(), "Borrower with unpaid fine was not blocked
208         print("PASS: Patron with unpaid fine is blocked from submitting loan requests.")
209
210         # 3. Check Admin /issue_loan blocks issuing to borrower with unpaid fine
211         with client.session_transaction() as sess:
212             sess["user_id"] = 1
213             sess["username"] = "admin"
```

```
            sess["role"] = "Admin"

        res_issue2 = client.post("/issue_loan", data={
            "copy_id": str(another_copy),
            "borrower_id": str(temp_uid),
            "due_in_days": "14"
        }, follow_redirects=True)
        html_issue2 = res_issue2.data.decode("utf-8")
        assert "Borrowing Prohibited" in html_issue2 and "fine" in html_issue2.lower(), "Admin was able to issue loan to pat
        print("PASS: Admin is blocked from directly issuing books to patron with unpaid fine.")

        # 4. Check Loan Requests Queue blocks approval for borrower with unpaid fine
        # Insert a pending request for this user directly
        cur.execute("SELECT NVL(MAX(Request_ID), 0) + 1 FROM Loan_Requests")
        temp_rid = cur.fetchone()[0]
        cur.execute("""
            INSERT INTO Loan_Requests (Request_ID, Book_ID, Borrower_ID, Request_Date, Status)
            VALUES (:p_rid, :p_bid, :p_uid, SYSDATE, 'Pending')
        """, {"p_rid": temp_rid, "p_bid": test_book_id, "p_uid": temp_uid})
        conn.commit()

        # Check loan requests page shows warning badge and disabled approve button
        res_queue = client.get("/loan_requests")
        html_queue = res_queue.data.decode("utf-8")
        assert f"#REQ-{temp_rid}" in html_queue
        assert "Unpaid Fine: Rs. 50.00" in html_queue, "Missing Unpaid Fine badge in loan requests queue!"

        # Attempt to approve
        res_appr = client.post("/loan_requests", data={
            "request_id": str(temp_rid),
            "action": "approve"
        }, follow_redirects=True)
        html_appr = res_appr.data.decode("utf-8")
        assert "Approval Prohibited" in html_appr and "fine" in html_appr.lower(), "Librarian/Admin was able to approve requ
        print("PASS: Loan Requests queue blocks approval and warns of unpaid fines.")

        # Clean up pending request
        cur.execute("DELETE FROM Loan_Requests WHERE Request_ID = :p_rid", {"p_rid": temp_rid})
        conn.commit()

    finally:
        # Clean up test records
        cur.execute("BEGIN DBMS_APPLICATION_INFO.SET_CLIENT_INFO('ALLOW_DELETE'); END;")
        if temp_lid:
            cur.execute("DELETE FROM Fines WHERE Loan_ID = :p_lid", {"p_lid": temp_lid})
        cur.execute("BEGIN DBMS_APPLICATION_INFO.SET_CLIENT_INFO(NULL); END;")
        if temp_lid:
            cur.execute("DELETE FROM Loans WHERE Loan_ID = :p_lid", {"p_lid": temp_lid})
        cur.execute("UPDATE Book_Copies SET Status = 'Available' WHERE Copy_ID = :p_cid", {"p_cid": test_copy_id})
        if temp_uid:
            cur.execute("DELETE FROM Users WHERE User_ID = :p_uid", {"p_uid": temp_uid})
        conn.commit()

    conn.close()
    print("\n========================================================================")
    print("ALL TESTS PASSED SUCCESSFULLY!")
    print("========================================================================")

if __name__ == "__main__":
    run_suite()
```

templates/base.html

HTML - Base Layout

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}VITCC LIBRARY | Enterprise Library System{% endblock %}</title>
    <!-- Modern Sans Font: Inter & Plus Jakarta Sans -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;500;600;700;800&family=JetBrains+Mono

    <style>
        :root {
            --bg-base: #0B0F19;
            --bg-surface: #111827;
            --bg-elevated: #1F2937;
            --bg-card: rgba(17, 24, 39, 0.7);
            --border-subtle: rgba(255, 255, 255, 0.08);
            --border-hover: rgba(255, 255, 255, 0.16);
            --accent-primary: #3B82F6;
            --accent-primary-hover: #2563EB;
            --accent-glow: rgba(59, 130, 246, 0.25);
            --text-primary: #F9FAFB;
            --text-secondary: #9CA3AF;
            --text-muted: #6B7280;
            --success-bg: rgba(16, 185, 129, 0.12);
            --success-text: #34D399;
            --success-border: rgba(16, 185, 129, 0.25);
            --warning-bg: rgba(245, 158, 11, 0.12);
            --warning-text: #FBBF24;
            --warning-border: rgba(245, 158, 11, 0.25);
            --danger-bg: rgba(239, 68, 68, 0.12);
            --danger-text: #F87171;
            --danger-border: rgba(239, 68, 68, 0.25);
            --radius-sm: 6px;
            --radius-md: 10px;
            --radius-lg: 16px;
            --radius-full: 9999px;
            --sidebar-width: 260px;
            --topbar-height: 70px;
            --font-sans: 'Plus Jakarta Sans', -apple-system, BlinkMacSystemFont, sans-serif;
            --font-mono: 'JetBrains Mono', monospace;
        }

        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
        }

        body {
            font-family: var(--font-sans);
            background-color: var(--bg-base);
            color: var(--text-primary);
            line-height: 1.5;
            -webkit-font-smoothing: antialiased;
            min-height: 100vh;
            display: flex;
        }

        /* App Wrapper */
        .app-layout {
            display: flex;
            width: 100%;
            min-height: 100vh;
        }

        /* Sidebar Styling */
        .sidebar {
```

```
            width: var(--sidebar-width);
            background: var(--bg-surface);
            border-right: 1px solid var(--border-subtle);
            display: flex;
            flex-direction: column;
            position: fixed;
            top: 0;
            bottom: 0;
            left: 0;
            z-index: 50;
            padding: 24px 16px;
        }

        .brand-container {
            display: flex;
            align-items: center;
            gap: 12px;
            padding: 0 12px 24px 12px;
            border-bottom: 1px solid var(--border-subtle);
            margin-bottom: 24px;
        }

        .brand-logo {
            width: 38px;
            height: 38px;
            background: linear-gradient(135deg, #3B82F6 0%, #1E40AF 100%);
            border-radius: var(--radius-md);
            display: flex;
            align-items: center;
            justify-content: center;
            box-shadow: 0 4px 12px var(--accent-glow);
        }

        .brand-logo svg {
            width: 20px;
            height: 20px;
            fill: #FFFFFF;
        }

        .brand-title {
            font-size: 1.15rem;
            font-weight: 700;
            letter-spacing: -0.02em;
            color: var(--text-primary);
        }

        /* Nav Links */
        .nav-section-title {
            font-size: 0.72rem;
            font-weight: 600;
            text-transform: uppercase;
            letter-spacing: 0.08em;
            color: var(--text-muted);
            padding: 8px 12px;
            margin-top: 12px;
        }

        .nav-list {
            list-style: none;
            display: flex;
            flex-direction: column;
            gap: 4px;
        }

        .nav-item a {
            display: flex;
            align-items: center;
            gap: 12px;
            padding: 10px 12px;
            color: var(--text-secondary);
            text-decoration: none;
            font-size: 0.9rem;
```

```
            font-weight: 500;
            border-radius: var(--radius-md);
            transition: all 0.15s ease-in-out;
        }

        .nav-item a:hover {
            background: rgba(255, 255, 255, 0.05);
            color: var(--text-primary);
        }

        .nav-item a.active {
            background: var(--accent-primary);
            color: #FFFFFF;
            font-weight: 600;
            box-shadow: 0 4px 14px var(--accent-glow);
        }

        .nav-item svg {
            width: 18px;
            height: 18px;
            stroke-width: 2;
            stroke: currentColor;
            fill: none;
        }

        /* Sidebar Footer / User Profile */
        .sidebar-footer {
            margin-top: auto;
            padding-top: 16px;
            border-top: 1px solid var(--border-subtle);
        }

        .user-card {
            display: flex;
            align-items: center;
            gap: 12px;
            padding: 10px;
            border-radius: var(--radius-md);
            background: rgba(255, 255, 255, 0.03);
            border: 1px solid var(--border-subtle);
        }

        .user-avatar {
            width: 36px;
            height: 36px;
            border-radius: var(--radius-full);
            background: linear-gradient(135deg, #4F46E5, #06B6D4);
            display: flex;
            align-items: center;
            justify-content: center;
            font-weight: 700;
            font-size: 0.85rem;
            color: #FFFFFF;
        }

        .user-info {
            flex: 1;
            overflow: hidden;
        }

        .user-name {
            font-size: 0.85rem;
            font-weight: 600;
            color: var(--text-primary);
            white-space: nowrap;
            overflow: hidden;
            text-overflow: ellipsis;
        }

        .user-role-badge {
            font-size: 0.7rem;
            color: var(--accent-primary);
```

```
            font-weight: 600;
        }

        .btn-logout {
            color: var(--text-muted);
            background: none;
            border: none;
            cursor: pointer;
            padding: 6px;
            border-radius: var(--radius-sm);
            transition: color 0.15s;
            display: flex;
            align-items: center;
            text-decoration: none;
        }

        .btn-logout:hover {
            color: var(--danger-text);
        }

        /* Main Workspace Content */
        .main-wrapper {
            margin-left: var(--sidebar-width);
            flex: 1;
            display: flex;
            flex-direction: column;
            min-height: 100vh;
        }

        /* Top Navbar */
        .topbar {
            height: var(--topbar-height);
            border-bottom: 1px solid var(--border-subtle);
            background: rgba(11, 15, 25, 0.7);
            backdrop-filter: blur(12px);
            display: flex;
            align-items: center;
            justify-content: space-between;
            padding: 0 32px;
            position: sticky;
            top: 0;
            z-index: 40;
        }

        .topbar-breadcrumb {
            display: flex;
            align-items: center;
            gap: 8px;
            font-size: 0.85rem;
            color: var(--text-muted);
        }

        .topbar-breadcrumb span.current {
            color: var(--text-primary);
            font-weight: 600;
        }

        .page-content {
            padding: 32px;
            flex: 1;
        }

        /* Alerts & Flash Messages */
        .flash-messages {
            margin-bottom: 24px;
            display: flex;
            flex-direction: column;
            gap: 10px;
        }

        .flash-alert {
            padding: 12px 16px;
```

```
            border-radius: var(--radius-md);
            font-size: 0.9rem;
            display: flex;
            align-items: center;
            justify-content: space-between;
            border: 1px solid;
            backdrop-filter: blur(8px);
        }

        .flash-alert.success {
            background: var(--success-bg);
            color: var(--success-text);
            border-color: var(--success-border);
        }

        .flash-alert.danger, .flash-alert.error {
            background: var(--danger-bg);
            color: var(--danger-text);
            border-color: var(--danger-border);
        }

        .flash-alert.warning {
            background: var(--warning-bg);
            color: var(--warning-text);
            border-color: var(--warning-border);
        }

        .flash-alert.info {
            background: rgba(59, 130, 246, 0.12);
            color: var(--accent-primary);
            border-color: rgba(59, 130, 246, 0.25);
        }

        /* Standard Buttons */
        .btn {
            display: inline-flex;
            align-items: center;
            justify-content: center;
            gap: 8px;
            padding: 10px 18px;
            border-radius: var(--radius-md);
            font-size: 0.88rem;
            font-weight: 600;
            text-decoration: none;
            cursor: pointer;
            transition: all 0.15s ease-in-out;
            border: 1px solid transparent;
        }

        .btn-primary {
            background: var(--accent-primary);
            color: #FFFFFF;
            box-shadow: 0 4px 12px var(--accent-glow);
        }

        .btn-primary:hover {
            background: var(--accent-primary-hover);
            transform: translateY(-1px);
        }

        .btn-secondary {
            background: var(--bg-elevated);
            color: var(--text-primary);
            border: 1px solid var(--border-subtle);
        }

        .btn-secondary:hover {
            background: rgba(255, 255, 255, 0.1);
            border-color: var(--border-hover);
        }

        .badge {
```

```
            display: inline-flex;
            align-items: center;
            padding: 4px 10px;
            border-radius: var(--radius-full);
            font-size: 0.75rem;
            font-weight: 600;
        }

        .badge-success { background: var(--success-bg); color: var(--success-text); border: 1px solid var(--success-border);
        .badge-warning { background: var(--warning-bg); color: var(--warning-text); border: 1px solid var(--warning-border);
        .badge-danger  { background: var(--danger-bg);  color: var(--danger-text);  border: 1px solid var(--danger-border); }
        .badge-info    { background: rgba(59, 130, 246, 0.12); color: var(--accent-primary); border: 1px solid rgba(59, 130,

        /* Responsive */
        @media (max-width: 900px) {
            .sidebar {
                transform: translateX(-100%);
                transition: transform 0.3s ease;
            }
            .sidebar.open {
                transform: translateX(0);
            }
            .main-wrapper {
                margin-left: 0;
            }
        }
    </style>
    {% block extra_css %}{% endblock %}
</head>
<body>
    <div class="app-layout">
        <!-- Sidebar Navigation -->
        {% if session.get('user_id') %}
        <aside class="sidebar">
            <div class="brand-container">
                <div class="brand-logo">
                    <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path>
                </div>
                <div>
                    <div class="brand-title">VITCC LIBRARY</div>
                </div>
            </div>

            <!-- Role-Based Navigation Links -->
            <div class="nav-section-title">Workspace</div>
            <ul class="nav-list">
                <!-- 1. Admin & Librarian Navigation -->
                {% if session.get('role') in ['Admin', 'Librarian'] or session.get('Role') in ['Admin', 'Librarian'] %}
                    <li class="nav-item">
                        <a href="{{ url_for('dashboard') }}" class="{% if request.endpoint == 'dashboard' %}active{% endif %
                            <svg viewBox="0 0 24 24"><rect x="3" y="3" width="7" height="7"></rect><rect x="14" y="3" width=
                            Dashboard
                        </a>
                    </li>
                    <li class="nav-item">
                        <a href="{{ url_for('books') }}" class="{% if request.endpoint == 'books' %}active{% endif %}">
                            <svg viewBox="0 0 24 24"><path d="M4 19.5v-15A2.5 2.5 0 0 1 6.5 2H20v20H6.5a2.5 2.5 0 0 1-2.5-2.5
                            Catalog
                        </a>
                    </li>
                    <li class="nav-item">
                        <a href="{{ url_for('issue_loan') }}" class="{% if request.endpoint == 'issue_loan' %}active{% endif
                            <svg viewBox="0 0 24 24"><circle cx="12" cy="12" r="10"></circle><polyline points="12 6 12 12 14
                            Issue Book
                        </a>
                    </li>
                    <li class="nav-item">
                        <a href="{{ url_for('loan_requests') }}" class="{% if request.endpoint == 'loan_requests' %}active{%
                            <svg viewBox="0 0 24 24"><path d="M16 4h2a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H6a2 2 0 0 1-2-2V6a2 2 0
                            Loan Requests
                        </a>
                    </li>
```

```
                {% if session.get('role') == 'Admin' or session.get('Role') == 'Admin' %}
                <li class="nav-item">
                    <a href="{{ url_for('manage_users') }}" class="{% if request.endpoint == 'manage_users' %}active{% e
                        <svg viewBox="0 0 24 24"><path d="M16 21v-2a4 4 0 0 0-4-4H6a4 4 0 0 0-4 4v2"></path><circle cx="
                        Manage Users
                    </a>
                </li>
                {% endif %}

            <!-- 2. Staff Navigation -->
            {% elif session.get('role') == 'Staff' or session.get('Role') == 'Staff' %}
                <li class="nav-item">
                    <a href="{{ url_for('dashboard') }}" class="{% if request.endpoint == 'dashboard' %}active{% endif %
                        <svg viewBox="0 0 24 24"><rect x="3" y="3" width="7" height="7"></rect><rect x="14" y="3" width=
                        Dashboard
                    </a>
                </li>
                <li class="nav-item">
                    <a href="{{ url_for('books') }}" class="{% if request.endpoint == 'books' %}active{% endif %}">
                        <svg viewBox="0 0 24 24"><path d="M4 19.5v-15A2.5 2.5 0 0 1 6.5 2H20v20H6.5a2.5 2.5 0 0 1-2.5-2.5
                        Catalog
                    </a>
                </li>
                <li class="nav-item">
                    <a href="{{ url_for('issue_loan') }}" class="{% if request.endpoint == 'issue_loan' %}active{% endif
                        <svg viewBox="0 0 24 24"><circle cx="12" cy="12" r="10"></circle><polyline points="12 6 12 12 14
                        Issue Book
                    </a>
                </li>

            <!-- 3. Student Navigation -->
            {% elif session.get('role') == 'Student' or session.get('Role') == 'Student' %}
                <li class="nav-item">
                    <a href="{{ url_for('dashboard') }}" class="{% if request.endpoint == 'dashboard' %}active{% endif %
                        <svg viewBox="0 0 24 24"><rect x="3" y="3" width="7" height="7"></rect><rect x="14" y="3" width=
                        My Dashboard
                    </a>
                </li>
                <li class="nav-item">
                    <a href="{{ url_for('books') }}" class="{% if request.endpoint == 'books' %}active{% endif %}">
                        <svg viewBox="0 0 24 24"><circle cx="11" cy="11" r="8"></circle><line x1="21" y1="21" x2="16.65"
                        Search Books
                    </a>
                </li>
                <li class="nav-item">
                    <a href="{{ url_for('my_fines') }}" class="{% if request.endpoint == 'my_fines' %}active{% endif %}"
                        <svg viewBox="0 0 24 24"><line x1="12" y1="1" x2="12" y2="23"></line><path d="M17 5H9.5a3.5 3.5
                        My Fines
                    </a>
                </li>
            {% endif %}
            <li class="nav-item">
                <a href="{{ url_for('change_password') }}" class="{% if request.endpoint == 'change_password' %}active{%
                    <svg viewBox="0 0 24 24"><rect x="3" y="11" width="18" height="11" rx="2" ry="2"></rect><path d="M7
                    Change Password
                </a>
            </li>
        </ul>

        <!-- Authenticated User Profile Summary -->
        <div class="sidebar-footer">
            <div class="user-card">
                <div class="user-avatar">
                    {{ session.get('username', 'U')[0]|upper }}
                </div>
                <div class="user-info">
                    <div class="user-name">{{ session.get('username') }}</div>
                    <div class="user-role-badge">{{ session.get('role') }}</div>
                </div>
                <a href="{{ url_for('change_password') }}" class="btn-logout" title="Change Password" style="color: var(
                    <svg viewBox="0 0 24 24" width="16" height="16" stroke="currentColor" fill="none" stroke-width="2"><
                </a>
```

```
                    <a href="{{ url_for('logout') }}" class="btn-logout" title="Sign Out">
                        <svg viewBox="0 0 24 24" width="18" height="18" stroke="currentColor" fill="none" stroke-width="2"><p
                    </a>
                </div>
            </div>
        </aside>
        {% endif %}

        <!-- Main Workspace Area -->
        <div class="main-wrapper" {% if not session.get('user_id') %}style="margin-left: 0;"{% endif %}>
            {% if session.get('user_id') %}
            <header class="topbar">
                <div class="topbar-breadcrumb">
                    <span>VITCC LIBRARY</span>
                    <span>/</span>
                    <span class="current">{% block breadcrumb %}Workspace{% endblock %}</span>
                </div>
                <div>
                    <span class="badge badge-info">{{ session.get('role') }} Portal</span>
                </div>
            </header>
            {% endif %}

            <main class="page-content">
                <!-- Flash Notification Banner Stack -->
                {% with messages = get_flashed_messages(with_categories=true) %}
                    {% if messages %}
                        <div class="flash-messages">
                            {% for category, message in messages %}
                                <div class="flash-alert {{ category }}">
                                    <span>{{ message }}</span>
                                    <span style="cursor: pointer;" onclick="this.parentElement.remove();">&times;</span>
                                </div>
                            {% endfor %}
                        </div>
                    {% endif %}
                {% endwith %}

                {% block content %}{% endblock %}
            </main>
        </div>
    </div>

    {% block extra_js %}{% endblock %}
</body>
</html>
```

templates/login.html

HTML - Login Page

```
{% extends "base.html" %}

{% block title %}Sign In | VITCC LIBRARY{% endblock %}

{% block extra_css %}
<style>
    .login-container {
        display: flex;
        align-items: center;
        justify-content: center;
        min-height: calc(100vh - 120px);
        position: relative;
    }

    /* Ambient glow elements */
    .ambient-glow-1 {
        position: absolute;
        width: 350px;
        height: 350px;
        background: radial-gradient(circle, rgba(59, 130, 246, 0.15) 0%, rgba(0, 0, 0, 0) 70%);
        top: 20%;
        left: 30%;
        filter: blur(40px);
        pointer-events: none;
        z-index: 0;
    }

    .ambient-glow-2 {
        position: absolute;
        width: 300px;
        height: 300px;
        background: radial-gradient(circle, rgba(99, 102, 241, 0.12) 0%, rgba(0, 0, 0, 0) 70%);
        bottom: 20%;
        right: 30%;
        filter: blur(40px);
        pointer-events: none;
        z-index: 0;
    }

    /* Glass Card */
    .login-card {
        width: 100%;
        max-width: 440px;
        background: rgba(17, 24, 39, 0.82);
        backdrop-filter: blur(20px);
        -webkit-backdrop-filter: blur(20px);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 40px;
        box-shadow: 0 20px 40px -15px rgba(0, 0, 0, 0.7);
        position: relative;
        z-index: 10;
    }

    .login-card:hover {
        border-color: var(--border-hover);
    }

    .login-header {
        text-align: center;
        margin-bottom: 32px;
    }

    .login-badge {
        display: inline-flex;
        align-items: center;
        gap: 6px;
        font-size: 0.75rem;
        font-weight: 600;
```

```
        letter-spacing: 0.05em;
        text-transform: uppercase;
        color: var(--accent-primary);
        background: rgba(59, 130, 246, 0.1);
        padding: 4px 12px;
        border-radius: var(--radius-full);
        border: 1px solid rgba(59, 130, 246, 0.2);
        margin-bottom: 12px;
    }

    .login-title {
        font-size: 1.6rem;
        font-weight: 800;
        letter-spacing: -0.03em;
        color: var(--text-primary);
        margin-bottom: 6px;
    }

    .login-subtitle {
        font-size: 0.88rem;
        color: var(--text-secondary);
    }

    /* Form Fields */
    .form-group {
        margin-bottom: 20px;
    }

    .form-label {
        display: block;
        font-size: 0.82rem;
        font-weight: 600;
        color: var(--text-secondary);
        margin-bottom: 8px;
        letter-spacing: 0.01em;
    }

    .input-wrapper {
        position: relative;
        display: flex;
        align-items: center;
    }

    .input-icon {
        position: absolute;
        left: 14px;
        width: 18px;
        height: 18px;
        stroke: var(--text-muted);
        stroke-width: 2;
        fill: none;
        pointer-events: none;
    }

    .form-input {
        width: 100%;
        padding: 12px 14px 12px 42px;
        background: rgba(11, 15, 25, 0.6);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-md);
        color: var(--text-primary);
        font-size: 0.92rem;
        font-family: inherit;
        transition: all 0.15s ease-in-out;
    }

    .form-input:focus {
        outline: none;
        border-color: var(--accent-primary);
        box-shadow: 0 0 0 3px var(--accent-glow);
        background: rgba(11, 15, 25, 0.9);
    }
```

```
142
143     .form-input::placeholder {
144         color: var(--text-muted);
145     }
146
147     .btn-submit {
148         width: 100%;
149         padding: 13px;
150         font-size: 0.92rem;
151         font-weight: 700;
152         margin-top: 10px;
153     }
154
155     .login-footer-hint {
156         margin-top: 24px;
157         text-align: center;
158         font-size: 0.78rem;
159         color: var(--text-muted);
160     }
161 </style>
162 {% endblock %}
163
164 {% block content %}
165 <div class="login-container">
166     <div class="ambient-glow-1"></div>
167     <div class="ambient-glow-2"></div>
168
169     <div class="login-card">
170         <div class="login-header">
171             <span class="login-badge">
172                 <svg width="8" height="8" viewBox="0 0 8 8" fill="currentColor"><circle cx="4" cy="4" r="3"></circle></svg>
173                 Secure Portal
174             </span>
175             <h1 class="login-title">VITCC LIBRARY</h1>
176             <p class="login-subtitle">Sign in to access catalog inventory & circulation</p>
177         </div>
178
179         <form method="POST" action="{{ url_for('login') }}">
180             <div class="form-group">
181                 <label class="form-label" for="username">Username</label>
182                 <div class="input-wrapper">
183                     <svg class="input-icon" viewBox="0 0 24 24"><path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2"></path><
184                     <input
185                         type="text"
186                         id="username"
187                         name="username"
188                         class="form-input"
189                         placeholder="e.g. jdoe_admin"
190                         required
191                         autofocus
192                     >
193                 </div>
194             </div>
195
196             <div class="form-group">
197                 <label class="form-label" for="password">Password</label>
198                 <div class="input-wrapper">
199                     <svg class="input-icon" viewBox="0 0 24 24"><rect x="3" y="11" width="18" height="11" rx="2" ry="2"></re
200                     <input
201                         type="password"
202                         id="password"
203                         name="password"
204                         class="form-input"
205                         placeholder="••••••••••••"
206                         required
207                     >
208                 </div>
209             </div>
210
211             <button type="submit" class="btn btn-primary btn-submit">
212                 Sign In to Dashboard &rarr;
213             </button>
```

```
214
215             <div style="margin-top: 18px; text-align: center; font-size: 0.85rem; color: var(--text-secondary);">
216                 Don't have an account?
217                 <a href="{{ url_for('register') }}" style="color: var(--accent-primary); font-weight: 600; text-decoration: 
218                     Sign Up
219                 </a>
220             </div>
221         </form>
222
223         <div class="login-footer-hint">
224             Role-Based Access Control &bull; Oracle Database 21c
225         </div>
226     </div>
227 </div>
228 {% endblock %}
```

templates/register.html

HTML - Register Page

```
{% extends "base.html" %}

{% block title %}Sign Up | VITCC LIBRARY{% endblock %}

{% block extra_css %}
<style>
    .register-container {
        display: flex;
        align-items: center;
        justify-content: center;
        min-height: calc(100vh - 120px);
        position: relative;
        padding: 30px 16px;
    }

    /* Ambient glow elements */
    .ambient-glow-1 {
        position: absolute;
        width: 380px;
        height: 380px;
        background: radial-gradient(circle, rgba(59, 130, 246, 0.16) 0%, rgba(0, 0, 0, 0) 70%);
        top: 15%;
        left: 28%;
        filter: blur(50px);
        pointer-events: none;
        z-index: 0;
    }

    .ambient-glow-2 {
        position: absolute;
        width: 320px;
        height: 320px;
        background: radial-gradient(circle, rgba(99, 102, 241, 0.14) 0%, rgba(0, 0, 0, 0) 70%);
        bottom: 15%;
        right: 28%;
        filter: blur(50px);
        pointer-events: none;
        z-index: 0;
    }

    /* Glass Card */
    .register-card {
        width: 100%;
        max-width: 480px;
        background: rgba(17, 24, 39, 0.85);
        backdrop-filter: blur(20px);
        -webkit-backdrop-filter: blur(20px);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 38px 40px;
        box-shadow: 0 20px 45px -15px rgba(0, 0, 0, 0.75);
        position: relative;
        z-index: 10;
        transition: border-color 0.2s ease;
    }

    .register-card:hover {
        border-color: var(--border-hover);
    }

    .register-header {
        text-align: center;
        margin-bottom: 28px;
    }

    .register-badge {
        display: inline-flex;
        align-items: center;
        gap: 6px;
```

```
        font-size: 0.74rem;
        font-weight: 600;
        letter-spacing: 0.05em;
        text-transform: uppercase;
        color: var(--accent-primary);
        background: rgba(59, 130, 246, 0.1);
        padding: 4px 12px;
        border-radius: var(--radius-full);
        border: 1px solid rgba(59, 130, 246, 0.2);
        margin-bottom: 12px;
    }

    .register-title {
        font-size: 1.6rem;
        font-weight: 800;
        letter-spacing: -0.03em;
        color: var(--text-primary);
        margin-bottom: 6px;
    }

    .register-subtitle {
        font-size: 0.88rem;
        color: var(--text-secondary);
    }

    /* Form Fields */
    .form-group {
        margin-bottom: 18px;
    }

    .form-label {
        display: block;
        font-size: 0.82rem;
        font-weight: 600;
        color: var(--text-secondary);
        margin-bottom: 8px;
        letter-spacing: 0.01em;
    }

    .input-wrapper {
        position: relative;
        display: flex;
        align-items: center;
    }

    .input-icon {
        position: absolute;
        left: 14px;
        width: 18px;
        height: 18px;
        stroke: var(--text-muted);
        stroke-width: 2;
        fill: none;
        pointer-events: none;
    }

    .form-input {
        width: 100%;
        padding: 12px 14px 12px 42px;
        background: rgba(11, 15, 25, 0.65);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-md);
        color: var(--text-primary);
        font-size: 0.92rem;
        font-family: inherit;
        transition: all 0.15s ease-in-out;
    }

    .form-input:focus {
        outline: none;
        border-color: var(--accent-primary);
        box-shadow: 0 0 0 3px var(--accent-glow);
```

```
        background: rgba(11, 15, 25, 0.95);
    }

    .form-input::placeholder {
        color: var(--text-muted);
    }

    .btn-submit {
        width: 100%;
        padding: 13px;
        font-size: 0.92rem;
        font-weight: 700;
        margin-top: 10px;
    }

    .login-redirect-link {
        margin-top: 20px;
        text-align: center;
        font-size: 0.85rem;
        color: var(--text-secondary);
    }

    .login-redirect-link a {
        color: var(--accent-primary);
        font-weight: 600;
        text-decoration: none;
        transition: color 0.15s;
    }

    .login-redirect-link a:hover {
        text-decoration: underline;
    }
</style>
{% endblock %}

{% block content %}
<div class="register-container">
    <div class="ambient-glow-1"></div>
    <div class="ambient-glow-2"></div>

    <div class="register-card">
        <div class="register-header">
            <span class="register-badge">
                <svg width="8" height="8" viewBox="0 0 8 8" fill="currentColor"><circle cx="4" cy="4" r="3"></circle></svg>
                Institutional Portal
            </span>
            <h1 class="register-title">Create Account</h1>
            <p class="register-subtitle">Register to borrow books, browse stock, & access digital loans</p>
        </div>

        <form method="POST" action="{{ url_for('register') }}">
            <div class="form-group">
                <label class="form-label" for="username">Username *</label>
                <div class="input-wrapper">
                    <svg class="input-icon" viewBox="0 0 24 24"><path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2"></path><
                    <input
                        type="text"
                        id="username"
                        name="username"
                        class="form-input"
                        placeholder="e.g. jsmith"
                        required
                        autofocus
                    >
                </div>
            </div>

            <div class="form-group">
                <label class="form-label" for="email">Institutional Email *</label>
                <div class="input-wrapper">
                    <svg class="input-icon" viewBox="0 0 24 24"><path d="M4 4h16c1.1 0 2 .9 2 2v12c0 1.1-.9 2-2 2H4c-1.1 0-2
                    <input
```

```
                        type="email"
                        id="email"
                        name="email"
                        class="form-input"
                        placeholder="jsmith@vit.edu"
                        required
                    >
                </div>
                <div style="font-size: 0.72rem; color: var(--text-muted); margin-top: 4px;">
                    Must contain characters <strong>'@'</strong> and <strong>'.'</strong> (e.g. name@vit.edu)
                </div>
            </div>

            <!-- Pre-Creation Password & Email Rules Notice -->
            <div style="background: rgba(30, 41, 59, 0.7); border: 1px solid rgba(59, 130, 246, 0.35); border-radius: var(--
                <div style="font-weight: 700; color: #93C5FD; display: flex; align-items: center; gap: 6px; margin-bottom: 6
                    <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.5"><ci
                    Credential & Password Requirements:
                </div>
                <ul style="margin: 0; padding-left: 18px; color: var(--text-secondary); line-height: 1.55;">
                    <li>Email must compulsory contain <strong>'@'</strong> and <strong>'.'</strong></li>
                    <li>Password length: <strong>5 to 13 characters</strong></li>
                    <li>At least <strong>1 uppercase</strong> letter (A-Z)</li>
                    <li>At least <strong>1 lowercase</strong> letter (a-z)</li>
                    <li>At least <strong>1 numeric</strong> digit (0-9)</li>
                    <li>At least <strong>1 special character</strong> (e.g. ! @ # $ % ^ & *)</li>
                </ul>
            </div>

            <div class="form-group">
                <label class="form-label" for="password">Password *</label>
                <div class="input-wrapper">
                    <svg class="input-icon" viewBox="0 0 24 24"><rect x="3" y="11" width="18" height="11" rx="2" ry="2"></re
                    <input
                        type="password"
                        id="password"
                        name="password"
                        class="form-input"
                        placeholder="••••••••••••"
                        minlength="5"
                        maxlength="13"
                        required
                    >
                </div>
            </div>

            <div class="form-group">
                <label class="form-label" for="role">Account Type (Role) *</label>
                <div class="input-wrapper">
                    <svg class="input-icon" viewBox="0 0 24 24"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2"></path><c
                    <select id="role" name="role" class="form-input" style="cursor: pointer;" required>
                        <option value="Student">Student (Academic Patron)</option>
                        <option value="Staff">Staff (Faculty / Department)</option>
                    </select>
                </div>
            </div>

            <button type="submit" class="btn btn-primary btn-submit">
                Complete Registration &rarr;
            </button>

            <div class="login-redirect-link">
                Already have an account?
                <a href="{{ url_for('login') }}">Sign In</a>
            </div>
        </form>
    </div>
</div>
{% endblock %}
```

templates/dashboard.html

HTML - Unified Dashboard

```
{% extends "base.html" %}

{% block title %}Dashboard | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}Dashboard{% endblock %}

{% block extra_css %}
<style>
    /* Welcome Header */
    .dashboard-header {
        display: flex;
        align-items: center;
        justify-content: space-between;
        margin-bottom: 32px;
        flex-wrap: wrap;
        gap: 16px;
    }

    .welcome-title {
        font-size: 1.75rem;
        font-weight: 800;
        letter-spacing: -0.03em;
        color: var(--text-primary);
    }

    .welcome-sub {
        font-size: 0.92rem;
        color: var(--text-secondary);
        margin-top: 4px;
    }

    .header-actions {
        display: flex;
        gap: 12px;
    }

    /* Bento Grid Stat Cards */
    .stat-grid {
        display: grid;
        grid-template-columns: repeat(auto-fit, minmax(230px, 1fr));
        gap: 20px;
        margin-bottom: 36px;
    }

    .stat-card {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 24px;
        display: flex;
        flex-direction: column;
        justify-content: space-between;
        position: relative;
        overflow: hidden;
        transition: transform 0.2s ease, border-color 0.2s ease;
    }

    .stat-card:hover {
        transform: translateY(-2px);
        border-color: var(--border-hover);
    }

    .stat-top {
        display: flex;
        align-items: center;
        justify-content: space-between;
        margin-bottom: 16px;
    }

    .stat-label {
```

```
        font-size: 0.8rem;
        font-weight: 600;
        text-transform: uppercase;
        letter-spacing: 0.05em;
        color: var(--text-muted);
    }

    .stat-icon {
        width: 36px;
        height: 36px;
        border-radius: var(--radius-md);
        display: flex;
        align-items: center;
        justify-content: center;
    }

    .stat-icon svg {
        width: 18px;
        height: 18px;
        stroke-width: 2;
        stroke: currentColor;
        fill: none;
    }

    .stat-value {
        font-size: 2.1rem;
        font-weight: 800;
        letter-spacing: -0.04em;
        color: var(--text-primary);
        font-feature-settings: "tnum";
    }

    .stat-footnote {
        font-size: 0.78rem;
        color: var(--text-muted);
        margin-top: 6px;
        display: flex;
        align-items: center;
        gap: 6px;
    }

    /* Icon Color Themes */
    .icon-blue   { background: rgba(59, 130, 246, 0.12); color: #60A5FA; }
    .icon-emerald{ background: rgba(16, 185, 129, 0.12); color: #34D399; }
    .icon-amber  { background: rgba(245, 158, 11, 0.12); color: #FBBF24; }
    .icon-rose   { background: rgba(239, 68, 68, 0.12);  color: #F87171; }
    .icon-purple { background: rgba(168, 85, 247, 0.12); color: #C084FC; }

    /* Action Deck */
    .section-title {
        font-size: 1.15rem;
        font-weight: 700;
        letter-spacing: -0.02em;
        margin-bottom: 16px;
        margin-top: 32px;
        display: flex;
        align-items: center;
        gap: 10px;
    }

    .quick-actions-bar {
        display: flex;
        gap: 14px;
        margin-bottom: 32px;
        flex-wrap: wrap;
    }

    .action-btn {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        padding: 14px 20px;
        border-radius: var(--radius-md);
```

```
        display: flex;
        align-items: center;
        gap: 12px;
        color: var(--text-primary);
        text-decoration: none;
        font-weight: 600;
        font-size: 0.88rem;
        transition: all 0.2s ease;
    }

    .action-btn:hover {
        border-color: var(--accent-primary);
        background: var(--bg-elevated);
        transform: translateY(-1px);
    }

    .action-btn svg {
        width: 18px;
        height: 18px;
        stroke: var(--accent-primary);
        stroke-width: 2;
        fill: none;
    }

    /* Modern Data Card Table */
    .data-panel {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        overflow: hidden;
        margin-bottom: 28px;
    }

    .panel-header {
        padding: 18px 24px;
        border-bottom: 1px solid var(--border-subtle);
        display: flex;
        align-items: center;
        justify-content: space-between;
    }

    .panel-title {
        font-size: 0.95rem;
        font-weight: 700;
        color: var(--text-primary);
    }

    .table-container {
        width: 100%;
        overflow-x: auto;
    }

    .data-table {
        width: 100%;
        border-collapse: collapse;
        text-align: left;
        font-size: 0.88rem;
    }

    .data-table th {
        background: rgba(255, 255, 255, 0.02);
        color: var(--text-muted);
        font-size: 0.72rem;
        text-transform: uppercase;
        letter-spacing: 0.06em;
        font-weight: 600;
        padding: 14px 24px;
        border-bottom: 1px solid var(--border-subtle);
    }

    .data-table td {
        padding: 16px 24px;
```

```
        border-bottom: 1px solid var(--border-subtle);
        color: var(--text-secondary);
    }

    .data-table tr:last-child td {
        border-bottom: none;
    }

    .data-table tr:hover td {
        background: rgba(255, 255, 255, 0.015);
        color: var(--text-primary);
    }

    .book-title-cell {
        color: var(--text-primary);
        font-weight: 600;
    }

    .empty-state {
        text-align: center;
        padding: 48px 20px;
        color: var(--text-muted);
    }

    .empty-state svg {
        width: 44px;
        height: 44px;
        stroke: var(--text-muted);
        stroke-width: 1.5;
        fill: none;
        margin-bottom: 12px;
    }
</style>
{% endblock %}

{% block content %}
<div class="dashboard-header">
    <div>
        <h1 class="welcome-title">Welcome back, {{ session.get('username', 'User') }}</h1>
        <p class="welcome-sub">
            Logged in as <strong style="color: var(--accent-primary);">{{ session.get('role') }}</strong>
        </p>
    </div>
    <div class="header-actions">
        <a href="{{ url_for('books') }}" class="btn btn-secondary">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="1
            Browse Catalog
        </a>
        {% if session.get('role') in ['Admin', 'Librarian'] %}
        <a href="{{ url_for('loan_requests') }}" class="btn btn-secondary">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M16 
            Loan Requests
        </a>
        <a href="{{ url_for('issue_loan') }}" class="btn btn-primary">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="1
            Issue Book
        </a>
        {% endif %}
    </div>
</div>

<!-- ========================================================================= -->
<!-- 1. ADMIN & LIBRARIAN VIEW -->
<!-- ========================================================================= -->
{% if session.get('role') in ['Admin', 'Librarian'] %}
<div class="stat-grid">
    <div class="stat-card">
        <div class="stat-top">
            <span class="stat-label">Total Titles</span>
            <div class="stat-icon icon-blue">
                <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path></sv
            </div>
```

```
        </div>
        <div class="stat-value">{{ data.total_books or 0 }}</div>
        <div class="stat-footnote">Unique registered titles in catalog</div>
    </div>

    <div class="stat-card">
        <div class="stat-top">
            <span class="stat-label">Active Loans</span>
            <div class="stat-icon icon-emerald">
                <svg viewBox="0 0 24 24"><circle cx="12" cy="12" r="10"></circle><polyline points="12 6 12 12 14 14"></polyl
            </div>
        </div>
        <div class="stat-value">{{ data.active_loans or data.borrowed_copies or 0 }}</div>
        <div class="stat-footnote">Physical copies currently checked out</div>
    </div>

    <div class="stat-card">
        <div class="stat-top">
            <span class="stat-label">Overdue Loans</span>
            <div class="stat-icon icon-rose">
                <svg viewBox="0 0 24 24"><path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2
            </div>
        </div>
        <div class="stat-value">{{ data.overdue_loans or 0 }}</div>
        <div class="stat-footnote">Loans exceeding return threshold</div>
    </div>

    <div class="stat-card">
        <div class="stat-top">
            <span class="stat-label">Unpaid Fines</span>
            <div class="stat-icon icon-amber">
                <svg viewBox="0 0 24 24"><line x1="12" y1="1" x2="12" y2="23"></line><path d="M17 5H9.5a3.5 3.5 0 0 0 0 7h5a
            </div>
        </div>
        <div class="stat-value">Rs. {{ "%.2f"|format(data.unpaid_fines or 0) }}</div>
        <div class="stat-footnote">Outstanding disciplinary penalties</div>
    </div>
</div>

<h2 class="section-title">Quick Administrative Actions</h2>
<div class="quick-actions-bar">
    <a href="{{ url_for('add_book') }}" class="action-btn">
        <svg viewBox="0 0 24 24"><line x1="12" y1="5" x2="12" y2="19"></line><line x1="5" y1="12" x2="19" y2="12"></line></sv
        Register New Book
    </a>
    <a href="{{ url_for('loan_requests') }}" class="action-btn">
        <svg viewBox="0 0 24 24"><path d="M16 4h2a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H6a2 2 0 0 1-2-2V6a2 2 0 0 1 2-2h2"></path><
        Loan Requests Queue
    </a>
    <a href="{{ url_for('issue_loan') }}" class="action-btn">
        <svg viewBox="0 0 24 24"><polyline points="9 11 12 14 22 4"></polyline><path d="M21 12v7a2 2 0 0 1-2 2H5a2 2 0 0 1-2
        Issue Physical Copy
    </a>
    <a href="{{ url_for('books') }}" class="action-btn">
        <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path></svg>
        Audit Catalog & Stock
    </a>
    {% if session.get('role') == 'Admin' %}
    <a href="{{ url_for('manage_users') }}" class="action-btn">
        <svg viewBox="0 0 24 24"><path d="M16 21v-2a4 4 0 0 0-4-4H6a4 4 0 0 0-4 4v2"></path><circle cx="9" cy="7" r="4"></ci
        Manage Users
    </a>
    {% endif %}
</div>

<!-- ========================================================================== -->
<!-- ADMIN & LIBRARIAN OPERATIONAL MONITORS & ACTIONS -->
<!-- ========================================================================== -->
{% if session.get('role') in ['Admin', 'Librarian'] %}
    <!-- Monitor 1: Who and all are currently loaning books (With Return Book Action) -->
    <h2 class="section-title">
        <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="var(--accent-primary)" stroke-width="2"><circle
```

```
        Currently Active Book Loans
    </h2>
    <div class="data-panel">
        <div class="panel-header">
            <span class="panel-title">Active Borrowers Ledger</span>
            <span class="badge badge-info">{{ data.active_loans_list|length if data.active_loans_list else 0 }} Active Loans
        </div>
        <div class="table-container">
            <table class="data-table">
                <thead>
                    <tr>
                        <th>Loan Ref</th>
                        <th>Borrower</th>
                        <th>Book Title</th>
                        <th>Copy Ref</th>
                        <th>Issue Date</th>
                        <th>Due Date</th>
                        <th>Fine Status</th>
                        <th style="text-align: right;">Circulation Action</th>
                    </tr>
                </thead>
                <tbody>
                    {% if data.active_loans_list %}
                        {% for loan in data.active_loans_list %}
                        <tr>
                            <td style="font-family: var(--font-mono); font-size: 0.8rem;">#LN-{{ loan.loan_id }}</td>
                            <td>
                                <div style="font-weight: 600; color: var(--text-primary);">{{ loan.borrower_name }}</div>
                                <div style="font-size: 0.76rem; color: var(--text-muted); display: flex; align-items: center
                                    <span>{{ loan.borrower_email }}</span>
                                    {% if loan.borrower_role == 'Staff' %}
                                        <span class="badge" style="background: rgba(168, 85, 247, 0.12); color: #C084FC; fon
                                    {% else %}
                                        <span class="badge" style="background: rgba(59, 130, 246, 0.12); color: #60A5FA; fon
                                    {% endif %}
                                </div>
                            </td>
                            <td class="book-title-cell">{{ loan.book_title }}</td>
                            <td style="font-family: var(--font-mono); font-size: 0.8rem;">Copy #{{ loan.copy_id }}</td>
                            <td>{{ loan.issue_date.strftime('%b %d, %Y') if loan.issue_date else 'N/A' }}</td>
                            <td style="font-weight: 600;">{{ loan.due_date.strftime('%b %d, %Y') if loan.due_date else 'N/A'
                            <td>
                                {% if (loan.fine_amount or 0) > 0 %}
                                    <span class="badge badge-danger">Rs. {{ "%.2f"|format(loan.fine_amount) }} Unpaid</span>
                                {% else %}
                                    <span class="badge badge-success" style="background: rgba(16, 185, 129, 0.12); color: #3
                                {% endif %}
                            </td>
                            <td style="text-align: right; white-space: nowrap;">
                                <form method="POST" action="{{ url_for('return_book', loan_id=loan.loan_id) }}" style="displa
                                    <button type="submit" class="btn" style="background: rgba(59, 130, 246, 0.15); color: #6
                                        <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" st
                                        Mark Returned
                                    </button>
                                </form>
                                <form method="POST" action="{{ url_for('delete_loan', loan_id=loan.loan_id) }}" style="displa
                                    <button type="submit" class="btn" style="background: rgba(239, 68, 68, 0.15); color: #F8
                                        <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" st
                                        Delete Loan
                                    </button>
                                </form>
                            </td>
                        </tr>
                        {% endfor %}
                    {% else %}
                        <tr>
                            <td colspan="8" class="empty-state">No books currently loaned out. All items are in inventory.</
                        </tr>
                    {% endif %}
                </tbody>
            </table>
        </div>
```

```
    </div>

    <!-- Monitor 2: Who are overdue (With Fast Return & Penalty Check & Delete Overdue) -->
    <h2 class="section-title" style="color: var(--danger-text);">
        <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="var(--danger-text)" stroke-width="2"><path d="M1
        Overdue Borrowers Watchlist
    </h2>
    <div class="data-panel" style="border-color: rgba(239, 68, 68, 0.25);">
        <div class="panel-header">
            <span class="panel-title" style="color: var(--danger-text);">Overdue Accounts Requiring Action</span>
            <span class="badge badge-danger">{{ data.overdue_loans_list|length if data.overdue_loans_list else 0 }} Overdue<
        </div>
        <div class="table-container">
            <table class="data-table">
                <thead>
                    <tr>
                        <th>Loan Ref</th>
                        <th>Delinquent Borrower</th>
                        <th>Book Title</th>
                        <th>Copy Ref</th>
                        <th>Due Date</th>
                        <th>Days Overdue</th>
                        <th style="text-align: right;">Circulation Action</th>
                    </tr>
                </thead>
                <tbody>
                    {% if data.overdue_loans_list %}
                        {% for ov in data.overdue_loans_list %}
                        <tr>
                            <td style="font-family: var(--font-mono); font-size: 0.8rem;">#LN-{{ ov.loan_id }}</td>
                            <td>
                                <div style="font-weight: 600; color: var(--text-primary);">{{ ov.borrower_name }}</div>
                                <div style="font-size: 0.76rem; color: var(--text-muted); display: flex; align-items: center
                                    <span>{{ ov.borrower_email }}</span>
                                    {% if ov.borrower_role == 'Staff' %}
                                        <span class="badge" style="background: rgba(168, 85, 247, 0.12); color: #C084FC; fon
                                    {% else %}
                                        <span class="badge" style="background: rgba(59, 130, 246, 0.12); color: #60A5FA; fon
                                    {% endif %}
                                </div>
                            </td>
                            <td class="book-title-cell">{{ ov.book_title }}</td>
                            <td style="font-family: var(--font-mono); font-size: 0.8rem;">Copy #{{ ov.copy_id }}</td>
                            <td style="color: var(--danger-text); font-weight: 600;">
                                {{ ov.due_date.strftime('%b %d, %Y') if ov.due_date else 'N/A' }}
                            </td>
                            <td>
                                <span class="badge badge-danger">
                                    {{ ov.days_overdue or 1 }} Day(s) Overdue
                                </span>
                            </td>
                            <td style="text-align: right; white-space: nowrap;">
                                <form method="POST" action="{{ url_for('return_book', loan_id=ov.loan_id) }}" style="display
                                    <button type="submit" class="btn" style="background: rgba(239, 68, 68, 0.15); color: #F8
                                        <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" st
                                        Receive Overdue
                                    </button>
                                </form>
                                <form method="POST" action="{{ url_for('delete_loan', loan_id=ov.loan_id) }}" style="display
                                    <button type="submit" class="btn" style="background: rgba(220, 38, 38, 0.25); color: #FC
                                        <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" st
                                        Delete Overdue
                                    </button>
                                </form>
                            </td>
                        </tr>
                        {% endfor %}
                    {% else %}
                        <tr>
                            <td colspan="7" class="empty-state">No loans currently overdue. All patrons in good standing!</t
                        </tr>
                    {% endif %}
```

```
                </tbody>
            </table>
        </div>
    </div>

    <!-- Monitor 3: Who didn't pay fine (With Collect & Mark Paid Action) -->
    <h2 class="section-title" style="color: var(--warning-text);">
        <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="var(--warning-text)" stroke-width="2"><line x1="
        Unpaid Fines & Disciplinary Ledger
    </h2>
    <div class="data-panel" style="border-color: rgba(245, 158, 11, 0.25);">
        <div class="panel-header">
            <span class="panel-title" style="color: var(--warning-text);">Outstanding Disciplinary Liabilities</span>
            <span class="badge badge-warning">{{ data.unpaid_fines_list|length if data.unpaid_fines_list else 0 }} Unpaid Pe
        </div>
        <div class="table-container">
            <table class="data-table">
                <thead>
                    <tr>
                        <th>Fine Ref</th>
                        <th>Borrower</th>
                        <th>Book Title</th>
                        <th>Originating Loan</th>
                        <th>Due Date</th>
                        <th>Outstanding Fine</th>
                        <th>Status</th>
                        <th style="text-align: right;">Cashier Action</th>
                    </tr>
                </thead>
                <tbody>
                    {% if data.unpaid_fines_list %}
                        {% for fine in data.unpaid_fines_list %}
                        <tr>
                            <td style="font-family: var(--font-mono); font-size: 0.8rem;">#FN-{{ fine.fine_id }}</td>
                            <td>
                                <div style="font-weight: 600; color: var(--text-primary);">{{ fine.borrower_name }}</div>
                                <div style="font-size: 0.76rem; color: var(--text-muted);">{{ fine.borrower_email }}</div>
                            </td>
                            <td class="book-title-cell">{{ fine.book_title }}</td>
                            <td style="font-family: var(--font-mono); font-size: 0.8rem;">#LN-{{ fine.loan_id }}</td>
                            <td>{{ fine.due_date.strftime('%b %d, %Y') if fine.due_date else 'N/A' }}</td>
                            <td style="font-weight: 700; color: var(--warning-text);">
                                Rs. {{ "%.2f"|format(fine.fine_amount) }}
                            </td>
                            <td>
                                <span class="badge badge-warning">Unpaid</span>
                            </td>
                            <td style="text-align: right;">
                                <form method="POST" action="{{ url_for('pay_fine', fine_id=fine.fine_id) }}" style="display:
                                    <button type="submit" class="btn" style="background: rgba(16, 185, 129, 0.15); color: #3
                                        <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" st
                                        Collect & Mark Paid
                                    </button>
                                </form>
                            </td>
                        </tr>
                        {% endfor %}
                    {% else %}
                        <tr>
                            <td colspan="8" class="empty-state">No unpaid fines recorded. All fines are settled.</td>
                        </tr>
                    {% endif %}
                </tbody>
            </table>
        </div>
    </div>
{% endif %}

<!-- ========================================================================== -->
<!-- 2. STUDENT VIEW -->
<!-- ========================================================================== -->
{% elif session.get('role') == 'Student' %}
```

```
<div class="stat-grid">
    <div class="stat-card">
        <div class="stat-top">
            <span class="stat-label">Active Loans</span>
            <div class="stat-icon icon-blue">
                <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path></sv
            </div>
        </div>
        <div class="stat-value">{{ data.active_loans|length if data.active_loans else 0 }} / 3</div>
        <div class="stat-footnote">Concurrent borrowing capacity quota</div>
    </div>

    <div class="stat-card">
        <div class="stat-top">
            <span class="stat-label">Outstanding Fines</span>
            <div class="stat-icon icon-amber">
                <svg viewBox="0 0 24 24"><line x1="12" y1="1" x2="12" y2="23"></line><path d="M17 5H9.5a3.5 3.5 0 0 0 0 7h5a
            </div>
        </div>
        {% if (data.total_fines or 0) > 0 %}
            <div class="stat-value" style="color: var(--danger-text);">Rs. {{ "%.2f"|format(data.total_fines) }}</div>
            <div class="stat-footnote">
                <span style="color: var(--danger-text); font-weight: 600;">Clear balance to borrow</span>
            </div>
        {% else %}
            <div class="stat-value" style="font-size: 1.25rem; color: #34D399; font-weight: 800;">Zero Fine</div>
            <div class="stat-footnote">
                <span style="color: var(--success-text); font-weight: 600;">Good standing (Rs. 0.00)</span>
            </div>
        {% endif %}
    </div>
</div>

<h2 class="section-title">My Active Loans</h2>
<div class="data-panel">
    <div class="panel-header">
        <span class="panel-title">Currently Borrowed Books</span>
        <span class="badge badge-info">{{ data.active_loans|length if data.active_loans else 0 }} Items</span>
    </div>
    <div class="table-container">
        <table class="data-table">
            <thead>
                <tr>
                    <th>Loan Ref</th>
                    <th>Book Title</th>
                    <th>Checkout Date</th>
                    <th>Due Date</th>
                    <th>Fine Status</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                {% if data.active_loans %}
                    {% for loan in data.active_loans %}
                    <tr>
                        <td style="font-family: var(--font-mono); font-size: 0.8rem;">#LN-{{ loan.loan_id }}</td>
                        <td class="book-title-cell">{{ loan.title }}</td>
                        <td>{{ loan.issue_date.strftime('%b %d, %Y') if loan.issue_date else 'N/A' }}</td>
                        <td>{{ loan.due_date.strftime('%b %d, %Y') if loan.due_date else 'N/A' }}</td>
                        <td>
                            {% if (loan.fine_amount or 0) > 0 %}
                                <span class="badge badge-danger">Rs. {{ "%.2f"|format(loan.fine_amount) }} Unpaid</span>
                            {% else %}
                                <span class="badge badge-success" style="background: rgba(16, 185, 129, 0.12); color: #34D39
                            {% endif %}
                        </td>
                        <td>
                            {% if loan.is_overdue == 1 %}
                                <span class="badge badge-danger">Overdue</span>
                            {% else %}
                                <span class="badge badge-success">Active Loan</span>
                            {% endif %}
```

```
                        </td>
                    </tr>
                    {% endfor %}
                {% else %}
                    <tr>
                        <td colspan="6">
                            <div class="empty-state">
                                <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v
                                <div>You have no active loans checked out.</div>
                                <a href="{{ url_for('books') }}" class="btn btn-secondary" style="margin-top: 14px;">Browse
                            </div>
                        </td>
                    </tr>
                {% endif %}
            </tbody>
        </table>
    </div>
</div>

<!-- Student Loan Requests Queue Status -->
<h2 class="section-title">My Loan Requests</h2>
<div class="data-panel">
    <div class="panel-header">
        <span class="panel-title">Submitted Permission Requests</span>
        <span class="badge badge-info">{{ data.loan_requests|length if data.loan_requests else 0 }} Requests</span>
    </div>
    <div class="table-container">
        <table class="data-table">
            <thead>
                <tr>
                    <th>Request Ref</th>
                    <th>Book Title</th>
                    <th>Requested On</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                {% if data.loan_requests %}
                    {% for req in data.loan_requests %}
                    <tr>
                        <td style="font-family: var(--font-mono); font-size: 0.8rem;">#REQ-{{ req.request_id }}</td>
                        <td class="book-title-cell">{{ req.book_title }}</td>
                        <td>{{ req.request_date.strftime('%b %d, %Y') if req.request_date else 'N/A' }}</td>
                        <td>
                            {% if req.status == 'Pending' %}
                                <span class="badge badge-warning">Awaiting Approval</span>
                            {% elif req.status == 'Approved' %}
                                <span class="badge badge-success">Approved & Issued</span>
                            {% else %}
                                <span class="badge badge-danger">Rejected</span>
                            {% endif %}
                        </td>
                    </tr>
                    {% endfor %}
                {% else %}
                    <tr>
                        <td colspan="4" class="empty-state">No loan requests submitted. You can request books from the Catal
                    </tr>
                {% endif %}
            </tbody>
        </table>
    </div>
</div>

<!-- ========================================================================== -->
<!-- 3. STAFF VIEW -->
<!-- ========================================================================== -->
{% elif session.get('role') == 'Staff' %}
<div class="stat-grid">
    <div class="stat-card">
        <div class="stat-top">
            <span class="stat-label">My Current Loans</span>
```

```
            <div class="stat-icon icon-purple">
                <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path></sv
            </div>
        </div>
        <div class="stat-value">{{ data.current_loans|length if data.current_loans else 0 }}</div>
        <div class="stat-footnote">Checked out for academic use</div>
    </div>

    <div class="stat-card">
        <div class="stat-top">
            <span class="stat-label">Outstanding Fines</span>
            <div class="stat-icon icon-amber">
                <svg viewBox="0 0 24 24"><line x1="12" y1="1" x2="12" y2="23"></line><path d="M17 5H9.5a3.5 3.5 0 0 0 0 7h5a
            </div>
        </div>
        {% if (data.total_fines or 0) > 0 %}
            <div class="stat-value" style="color: var(--danger-text);">Rs. {{ "%.2f"|format(data.total_fines) }}</div>
            <div class="stat-footnote">
                <span style="color: var(--danger-text); font-weight: 600;">Clear balance to borrow</span>
            </div>
        {% else %}
            <div class="stat-value" style="font-size: 1.25rem; color: #34D399; font-weight: 800;">Zero Fine</div>
            <div class="stat-footnote">
                <span style="color: var(--success-text); font-weight: 600;">Good standing (Rs. 0.00)</span>
            </div>
        {% endif %}
    </div>
</div>

<h2 class="section-title">My Checked Out Books</h2>
<div class="data-panel">
    <div class="table-container">
        <table class="data-table">
            <thead>
                <tr>
                    <th>Loan #</th>
                    <th>Book Title</th>
                    <th>Copy ID</th>
                    <th>Issued On</th>
                    <th>Due Date</th>
                    <th>Fine Status</th>
                </tr>
            </thead>
            <tbody>
                {% if data.current_loans %}
                    {% for loan in data.current_loans %}
                    <tr>
                        <td style="font-family: var(--font-mono);">#{{ loan.loan_id }}</td>
                        <td class="book-title-cell">{{ loan.title }}</td>
                        <td>Copy #{{ loan.copy_id }}</td>
                        <td>{{ loan.issue_date.strftime('%b %d, %Y') if loan.issue_date else 'N/A' }}</td>
                        <td>{{ loan.due_date.strftime('%b %d, %Y') if loan.due_date else 'N/A' }}</td>
                        <td>
                            {% if (loan.fine_amount or 0) > 0 %}
                                <span class="badge badge-danger">Rs. {{ "%.2f"|format(loan.fine_amount) }} Unpaid</span>
                            {% else %}
                                <span class="badge badge-success" style="background: rgba(16, 185, 129, 0.12); color: #34D39
                            {% endif %}
                        </td>
                    </tr>
                    {% endfor %}
                {% else %}
                    <tr>
                        <td colspan="6" class="empty-state">No items currently checked out.</td>
                    </tr>
                {% endif %}
            </tbody>
        </table>
    </div>
</div>

<!-- Staff Loan Requests Queue Status -->
```

```
<h2 class="section-title">My Loan Requests</h2>
<div class="data-panel">
    <div class="panel-header">
        <span class="panel-title">Submitted Permission Requests</span>
        <span class="badge badge-info">{{ data.loan_requests|length if data.loan_requests else 0 }} Requests</span>
    </div>
    <div class="table-container">
        <table class="data-table">
            <thead>
                <tr>
                    <th>Request Ref</th>
                    <th>Book Title</th>
                    <th>Requested On</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                {% if data.loan_requests %}
                    {% for req in data.loan_requests %}
                    <tr>
                        <td style="font-family: var(--font-mono); font-size: 0.8rem;">#REQ-{{ req.request_id }}</td>
                        <td class="book-title-cell">{{ req.book_title }}</td>
                        <td>{{ req.request_date.strftime('%b %d, %Y') if req.request_date else 'N/A' }}</td>
                        <td>
                            {% if req.status == 'Pending' %}
                                <span class="badge badge-warning">Awaiting Approval</span>
                            {% elif req.status == 'Approved' %}
                                <span class="badge badge-success">Approved & Issued</span>
                            {% else %}
                                <span class="badge badge-danger">Rejected</span>
                            {% endif %}
                        </td>
                    </tr>
                    {% endfor %}
                {% else %}
                    <tr>
                        <td colspan="4" class="empty-state">No loan requests submitted. You can request books from the Catal
                    </tr>
                {% endif %}
            </tbody>
        </table>
    </div>
</div>
{% endif %}
{% endblock %}
```

templates/catalog.html

HTML - Book Catalog

```
{% extends "base.html" %}

{% block title %}Category Directory & Catalog | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}Book Catalog{% endblock %}

{% block extra_css %}
<style>
    .hidden {
        display: none !important;
    }

    /* Page Header */
    .catalog-header {
        display: flex;
        align-items: center;
        justify-content: space-between;
        margin-bottom: 24px;
        flex-wrap: wrap;
        gap: 16px;
    }

    .catalog-title {
        font-size: 1.65rem;
        font-weight: 800;
        letter-spacing: -0.03em;
        color: var(--text-primary);
        display: flex;
        align-items: center;
        gap: 10px;
    }

    .catalog-sub {
        font-size: 0.9rem;
        color: var(--text-secondary);
        margin-top: 4px;
    }

    /* Search Section */
    .search-card {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 18px 22px;
        margin-bottom: 28px;
        box-shadow: 0 4px 18px rgba(0, 0, 0, 0.2);
    }

    .search-wrapper {
        display: flex;
        align-items: center;
        justify-content: space-between;
        flex-wrap: wrap;
        gap: 16px;
    }

    .search-box {
        position: relative;
        flex: 1;
        min-width: 280px;
    }

    .search-box svg {
        position: absolute;
        left: 14px;
        top: 50%;
        transform: translateY(-50%);
        width: 18px;
        height: 18px;
        stroke: var(--text-muted);
```

```
        fill: none;
        stroke-width: 2;
        pointer-events: none;
    }

    .search-input {
        width: 100%;
        padding: 11px 40px 11px 44px;
        background: var(--bg-surface-soft);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-md);
        color: var(--text-primary);
        font-size: 0.92rem;
        transition: all 0.2s ease;
    }

    .search-input:focus {
        outline: none;
        border-color: var(--accent-primary);
        box-shadow: 0 0 0 3px var(--accent-glow);
        background: rgba(30, 41, 59, 0.7);
    }

    .btn-clear-search {
        position: absolute;
        right: 12px;
        top: 50%;
        transform: translateY(-50%);
        background: none;
        border: none;
        color: var(--text-muted);
        font-size: 1rem;
        cursor: pointer;
        padding: 4px 6px;
        border-radius: 4px;
    }

    .btn-clear-search:hover {
        color: var(--text-primary);
        background: rgba(255, 255, 255, 0.08);
    }

    .category-counter {
        font-size: 0.88rem;
        color: var(--text-muted);
        white-space: nowrap;
    }

    .category-counter strong {
        color: var(--accent-primary);
    }

    /* Category Cards Grid */
    .category-grid {
        display: grid;
        grid-template-columns: repeat(auto-fill, minmax(290px, 1fr));
        gap: 20px;
    }

    .category-card {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 24px;
        text-decoration: none;
        color: inherit;
        display: flex;
        flex-direction: column;
        justify-content: space-between;
        min-height: 190px;
        transition: all 0.25s cubic-bezier(0.16, 1, 0.3, 1);
        position: relative;
```

```
        overflow: hidden;
    }

    .category-card::before {
        content: '';
        position: absolute;
        top: 0;
        left: 0;
        right: 0;
        height: 3px;
        background: linear-gradient(90deg, var(--accent-primary), #818CF8);
        opacity: 0;
        transition: opacity 0.25s ease;
    }

    .category-card:hover {
        transform: translateY(-4px);
        border-color: rgba(59, 130, 246, 0.5);
        box-shadow: 0 12px 28px -6px rgba(0, 0, 0, 0.5), 0 0 0 1px rgba(59, 130, 246, 0.3);
        background: rgba(30, 41, 59, 0.6);
    }

    .category-card:hover::before {
        opacity: 1;
    }

    .cat-top {
        display: flex;
        align-items: center;
        justify-content: space-between;
        margin-bottom: 14px;
    }

    .cat-code-badge {
        font-family: var(--font-mono);
        font-size: 0.72rem;
        font-weight: 700;
        letter-spacing: 0.08em;
        text-transform: uppercase;
        color: #93C5FD;
        background: rgba(59, 130, 246, 0.12);
        border: 1px solid rgba(59, 130, 246, 0.3);
        padding: 3px 9px;
        border-radius: var(--radius-sm);
    }

    .cat-count-badge {
        font-size: 0.78rem;
        font-weight: 600;
        color: var(--text-secondary);
        background: rgba(255, 255, 255, 0.05);
        border: 1px solid var(--border-subtle);
        padding: 3px 10px;
        border-radius: var(--radius-full);
        display: flex;
        align-items: center;
        gap: 5px;
    }

    .cat-title {
        font-size: 1.25rem;
        font-weight: 700;
        color: var(--text-primary);
        letter-spacing: -0.01em;
        margin-bottom: 8px;
        line-height: 1.3;
    }

    .cat-desc {
        font-size: 0.84rem;
        color: var(--text-muted);
        line-height: 1.5;
```

```
        margin-bottom: 18px;
    }

    .cat-footer {
        display: flex;
        align-items: center;
        justify-content: space-between;
        padding-top: 14px;
        border-top: 1px solid rgba(255, 255, 255, 0.05);
        color: var(--accent-primary);
        font-size: 0.84rem;
        font-weight: 600;
        transition: gap 0.2s ease;
    }

    .cat-footer svg {
        transition: transform 0.2s ease;
    }

    .category-card:hover .cat-footer svg {
        transform: translateX(4px);
    }

    /* Empty state */
    .empty-category-match {
        grid-column: 1 / -1;
        background: var(--bg-surface);
        border: 1px dashed var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 48px 24px;
        text-align: center;
    }

    .empty-icon {
        width: 56px;
        height: 56px;
        margin: 0 auto 16px auto;
        border-radius: var(--radius-full);
        background: rgba(59, 130, 246, 0.1);
        display: flex;
        align-items: center;
        justify-content: center;
        color: var(--accent-primary);
    }

    .empty-title {
        font-size: 1.2rem;
        font-weight: 700;
        color: var(--text-primary);
        margin-bottom: 6px;
    }

    .empty-sub {
        font-size: 0.88rem;
        color: var(--text-muted);
        max-width: 440px;
        margin: 0 auto 18px auto;
    }
</style>
{% endblock %}

{% block content %}
<!-- Catalog Header -->
<div class="catalog-header">
    <div>
        <h1 class="catalog-title">
            <svg width="26" height="26" viewBox="0 0 24 24" fill="none" stroke="var(--accent-primary)" stroke-width="2.2"><pa
            Library Catalog & Categories
        </h1>
        <p class="catalog-sub">Browse library collections by department category. Click any category to view and search its
    </div>

```

```
    {% if session.get('role') in ['Admin', 'Librarian'] or session.get('Role') in ['Admin', 'Librarian'] %}
    <div>
        <a href="{{ url_for('add_book') }}" class="btn btn-primary">
            <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="12"
            + Register New Book
        </a>
    </div>
    {% endif %}
</div>

<!-- Search Categories Toolbar -->
<div class="search-card">
    <div class="search-wrapper">
        <div class="search-box">
            <svg viewBox="0 0 24 24"><circle cx="11" cy="11" r="8"></circle><line x1="21" y1="21" x2="16.65" y2="16.65"></li
            <input
                type="text"
                id="categorySearchInput"
                class="search-input"
                placeholder="Search categories by name or section code (e.g., Computer Science, CS-01)..."
                oninput="filterCategories()"
                autocomplete="off"
            >
            <button type="button" id="btnClearSearch" class="btn-clear-search hidden" onclick="clearCategorySearch()" title=
        </div>
        <div class="category-counter">
            Showing <strong id="visibleCategoryCount">{{ categories|length }}</strong> of {{ categories|length }} Categories
        </div>
    </div>
</div>

<!-- Categories Directory Grid -->
<div class="category-grid" id="categoryGrid">
    {% if categories %}
        {% for cat in categories %}
        <a
            href="{{ url_for('category_books', category_id=cat.category_id) }}"
            class="category-card"
            id="catCard_{{ cat.category_id }}"
            data-name="{{ cat.name|lower }}"
            data-code="{{ (cat.section_code or '')|lower }}"
        >
            <div>
                <div class="cat-top">
                    <span class="cat-code-badge">{{ cat.section_code or 'SEC' }}</span>
                    <span class="cat-count-badge">
                        <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><p
                        {{ cat.book_count or 0 }} Book{% if cat.book_count != 1 %}s{% endif %}
                    </span>
                </div>
                <h3 class="cat-title">{{ cat.name }}</h3>
                <p class="cat-desc">Click to browse titles, inspect copy availability, and manage loans in this category.</p
            </div>
            <div class="cat-footer">
                <span>Browse Collection</span>
                <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.5"><polyli
            </div>
        </a>
        {% endfor %}
    {% else %}
        <div class="empty-category-match">
            <div class="empty-icon">
                <svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M
            </div>
            <h3 class="empty-title">No Categories Found</h3>
            <p class="empty-sub">No department categories currently exist in the database.</p>
        </div>
    {% endif %}

    <!-- Real-time Filter No-Match State -->
    <div class="empty-category-match hidden" id="noCategoryMatch">
        <div class="empty-icon">
```

```
            <svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="1
        </div>
        <h3 class="empty-title">No Matching Categories</h3>
        <p class="empty-sub">We couldn't find any category matching your search query. Try another keyword or section code.<
        <button type="button" class="btn btn-secondary" onclick="clearCategorySearch()">Clear Search Filter</button>
    </div>
</div>
{% endblock %}

{% block extra_js %}
<script>
    function filterCategories() {
        const input = document.getElementById('categorySearchInput');
        const query = (input ? input.value : '').trim().toLowerCase();
        const clearBtn = document.getElementById('btnClearSearch');
        if (clearBtn) {
            clearBtn.classList.toggle('hidden', query.length === 0);
        }

        const cards = document.querySelectorAll('.category-card');
        let visibleCount = 0;

        cards.forEach(card => {
            const name = card.getAttribute('data-name') || '';
            const code = card.getAttribute('data-code') || '';
            const matches = name.includes(query) || code.includes(query);

            if (matches) {
                card.classList.remove('hidden');
                visibleCount++;
            } else {
                card.classList.add('hidden');
            }
        });

        const counter = document.getElementById('visibleCategoryCount');
        if (counter) {
            counter.textContent = visibleCount;
        }

        const noMatch = document.getElementById('noCategoryMatch');
        if (noMatch) {
            noMatch.classList.toggle('hidden', visibleCount > 0);
        }
    }

    function clearCategorySearch() {
        const input = document.getElementById('categorySearchInput');
        if (input) {
            input.value = '';
            filterCategories();
            input.focus();
        }
    }
</script>
{% endblock %}
```

templates/category_books.html

HTML - Category Books

```
{% extends "base.html" %}

{% block title %}{{ category.name }} Books | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}
    <a href="{{ url_for('books') }}" style="color: var(--text-muted); text-decoration: none;">Catalog</a>
    <span style="margin: 0 6px; color: var(--border-subtle);">/</span>
    <span style="color: var(--text-primary);">{{ category.name }}</span>
{% endblock %}

{% block extra_css %}
<style>
    .hidden {
        display: none !important;
    }

    /* Back Link & Header */
    .category-page-header {
        margin-bottom: 24px;
    }

    .back-nav-bar {
        margin-bottom: 14px;
    }

    .back-nav-btn {
        display: inline-flex;
        align-items: center;
        gap: 8px;
        color: var(--text-secondary);
        text-decoration: none;
        font-size: 0.88rem;
        font-weight: 600;
        padding: 6px 12px;
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-sm);
        transition: all 0.2s ease;
    }

    .back-nav-btn:hover {
        color: var(--accent-primary);
        border-color: var(--accent-primary);
        background: rgba(59, 130, 246, 0.08);
        transform: translateX(-2px);
    }

    .cat-banner {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 24px 28px;
        display: flex;
        align-items: center;
        justify-content: space-between;
        flex-wrap: wrap;
        gap: 20px;
        position: relative;
        overflow: hidden;
    }

    .cat-banner::before {
        content: '';
        position: absolute;
        top: 0;
        left: 0;
        bottom: 0;
        width: 4px;
        background: linear-gradient(180deg, var(--accent-primary), #818CF8);
    }
```

```
70
71      .cat-banner-info {
72          display: flex;
73          flex-direction: column;
74          gap: 6px;
75      }
76
77      .cat-banner-badges {
78          display: flex;
79          align-items: center;
80          gap: 8px;
81          flex-wrap: wrap;
82      }
83
84      .cat-code-pill {
85          font-family: var(--font-mono);
86          font-size: 0.75rem;
87          font-weight: 700;
88          letter-spacing: 0.08em;
89          text-transform: uppercase;
90          color: #93C5FD;
91          background: rgba(59, 130, 246, 0.15);
92          border: 1px solid rgba(59, 130, 246, 0.35);
93          padding: 3px 10px;
94          border-radius: var(--radius-sm);
95      }
96
97      .cat-total-pill {
98          font-size: 0.78rem;
99          font-weight: 600;
100         color: var(--text-secondary);
101         background: rgba(255, 255, 255, 0.06);
102         border: 1px solid var(--border-subtle);
103         padding: 3px 10px;
104         border-radius: var(--radius-full);
105     }
106
107     .cat-banner-title {
108         font-size: 1.8rem;
109         font-weight: 800;
110         letter-spacing: -0.03em;
111         color: var(--text-primary);
112         margin: 4px 0 0 0;
113     }
114
115     /* Clean Book Search Toolbar (No Filters) */
116     .search-toolbar {
117         background: var(--bg-surface);
118         border: 1px solid var(--border-subtle);
119         border-radius: var(--radius-lg);
120         padding: 16px 20px;
121         margin: 24px 0;
122         display: flex;
123         align-items: center;
124         justify-content: space-between;
125         flex-wrap: wrap;
126         gap: 16px;
127         box-shadow: 0 4px 18px rgba(0, 0, 0, 0.2);
128     }
129
130     .search-input-wrapper {
131         position: relative;
132         flex: 1;
133         min-width: 280px;
134     }
135
136     .search-input-wrapper svg {
137         position: absolute;
138         left: 14px;
139         top: 50%;
140         transform: translateY(-50%);
141         width: 18px;
```

```
        height: 18px;
        stroke: var(--text-muted);
        fill: none;
        stroke-width: 2;
        pointer-events: none;
    }

    .search-input {
        width: 100%;
        padding: 11px 40px 11px 44px;
        background: var(--bg-surface-soft);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-md);
        color: var(--text-primary);
        font-size: 0.92rem;
        transition: all 0.2s ease;
    }

    .search-input:focus {
        outline: none;
        border-color: var(--accent-primary);
        box-shadow: 0 0 0 3px var(--accent-glow);
        background: rgba(30, 41, 59, 0.7);
    }

    .btn-clear-search {
        position: absolute;
        right: 12px;
        top: 50%;
        transform: translateY(-50%);
        background: none;
        border: none;
        color: var(--text-muted);
        font-size: 1rem;
        cursor: pointer;
        padding: 4px 6px;
        border-radius: 4px;
    }

    .btn-clear-search:hover {
        color: var(--text-primary);
        background: rgba(255, 255, 255, 0.08);
    }

    .results-counter {
        font-size: 0.88rem;
        color: var(--text-muted);
        white-space: nowrap;
    }

    .results-counter strong {
        color: var(--accent-primary);
    }

    /* Books Grid */
    .books-grid {
        display: grid;
        grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
        gap: 20px;
    }

    .book-card {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 22px;
        display: flex;
        flex-direction: column;
        justify-content: space-between;
        transition: all 0.25s cubic-bezier(0.16, 1, 0.3, 1);
        position: relative;
    }
```

```
214
215     .book-card:hover {
216         transform: translateY(-3px);
217         border-color: rgba(59, 130, 246, 0.4);
218         box-shadow: 0 12px 28px -6px rgba(0, 0, 0, 0.5), 0 0 0 1px rgba(59, 130, 246, 0.2);
219     }
220
221     .book-meta-top {
222         display: flex;
223         align-items: center;
224         justify-content: space-between;
225         margin-bottom: 12px;
226         gap: 8px;
227     }
228
229     .badge-stock-avail {
230         background: rgba(16, 185, 129, 0.15);
231         color: #34D399;
232         border: 1px solid rgba(16, 185, 129, 0.35);
233         font-size: 0.72rem;
234         font-weight: 700;
235         padding: 3px 8px;
236         border-radius: var(--radius-full);
237     }
238
239     .badge-stock-out {
240         background: rgba(239, 68, 68, 0.15);
241         color: #F87171;
242         border: 1px solid rgba(239, 68, 68, 0.35);
243         font-size: 0.72rem;
244         font-weight: 700;
245         padding: 3px 8px;
246         border-radius: var(--radius-full);
247     }
248
249     .book-title {
250         font-size: 1.12rem;
251         font-weight: 700;
252         color: var(--text-primary);
253         letter-spacing: -0.01em;
254         line-height: 1.35;
255         margin-bottom: 8px;
256     }
257
258     .book-authors {
259         display: flex;
260         align-items: center;
261         gap: 6px;
262         font-size: 0.84rem;
263         color: var(--text-secondary);
264         margin-bottom: 16px;
265     }
266
267     .book-authors svg {
268         width: 14px;
269         height: 14px;
270         stroke: var(--text-muted);
271         fill: none;
272         flex-shrink: 0;
273     }
274
275     .book-details-list {
276         background: var(--bg-surface-soft);
277         border: 1px solid rgba(255, 255, 255, 0.04);
278         border-radius: var(--radius-md);
279         padding: 12px 14px;
280         margin-bottom: 16px;
281         display: flex;
282         flex-direction: column;
283         gap: 6px;
284     }
285
```

```
    .book-detail-row {
        display: flex;
        align-items: center;
        justify-content: space-between;
        font-size: 0.78rem;
    }

    .book-detail-row span {
        color: var(--text-muted);
    }

    .book-detail-row strong {
        color: var(--text-primary);
        font-weight: 600;
    }

    /* Admin Inventory Adjuster with Numerical Quantity Input */
    .inventory-adjust-box {
        background: rgba(59, 130, 246, 0.06);
        border: 1px solid rgba(59, 130, 246, 0.2);
        border-radius: var(--radius-md);
        padding: 12px 14px;
        margin-top: 10px;
    }

    .inventory-adjust-top {
        display: flex;
        align-items: center;
        justify-content: space-between;
        margin-bottom: 8px;
    }

    .inventory-adjust-label {
        font-size: 0.76rem;
        font-weight: 700;
        text-transform: uppercase;
        letter-spacing: 0.05em;
        color: #93C5FD;
    }

    .inventory-current-stock {
        font-size: 0.75rem;
        font-family: var(--font-mono);
        color: var(--text-muted);
    }

    .inventory-type-controls {
        display: flex;
        align-items: center;
        gap: 8px;
    }

    .qty-stepper-wrapper {
        display: inline-flex;
        align-items: center;
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-sm);
        overflow: hidden;
    }

    .btn-stepper-step {
        background: rgba(255, 255, 255, 0.04);
        border: none;
        color: var(--text-secondary);
        font-size: 1rem;
        font-weight: 700;
        width: 28px;
        height: 32px;
        display: flex;
        align-items: center;
        justify-content: center;
```

```
        cursor: pointer;
        transition: all 0.15s ease;
        user-select: none;
    }

    .btn-stepper-step:hover {
        background: rgba(59, 130, 246, 0.2);
        color: var(--accent-primary);
    }

    .btn-stepper-step:active {
        background: rgba(59, 130, 246, 0.35);
    }

    .qty-type-input {
        width: 48px;
        height: 32px;
        padding: 4px;
        background: transparent;
        border: none;
        border-left: 1px solid var(--border-subtle);
        border-right: 1px solid var(--border-subtle);
        color: var(--text-primary);
        font-family: var(--font-mono);
        font-size: 0.9rem;
        font-weight: 700;
        text-align: center;
        transition: all 0.2s ease;
        -moz-appearance: textfield;
    }

    .qty-type-input::-webkit-outer-spin-button,
    .qty-type-input::-webkit-inner-spin-button {
        -webkit-appearance: none;
        margin: 0;
    }

    .qty-type-input:focus {
        outline: none;
        background: rgba(59, 130, 246, 0.08);
    }

    .btn-qty-action {
        flex: 1;
        display: inline-flex;
        align-items: center;
        justify-content: center;
        gap: 5px;
        padding: 7px 10px;
        font-size: 0.76rem;
        font-weight: 700;
        border-radius: var(--radius-sm);
        cursor: pointer;
        transition: all 0.2s ease;
    }

    .btn-qty-decrease {
        background: rgba(239, 68, 68, 0.12);
        color: #F87171;
        border: 1px solid rgba(239, 68, 68, 0.3);
    }

    .btn-qty-decrease:hover:not(:disabled) {
        background: rgba(239, 68, 68, 0.25);
        border-color: #EF4444;
        color: #FFFFFF;
    }

    .btn-qty-increase {
        background: rgba(59, 130, 246, 0.15);
        color: #60A5FA;
        border: 1px solid rgba(59, 130, 246, 0.35);
```

```
    }

    .btn-qty-increase:hover {
        background: rgba(59, 130, 246, 0.3);
        border-color: var(--accent-primary);
        color: #FFFFFF;
    }

    /* Patron Request Action */
    .patron-action-box {
        margin-top: 14px;
    }

    .btn-request-loan {
        width: 100%;
        padding: 8px 12px;
        background: rgba(59, 130, 246, 0.15);
        color: #60A5FA;
        border: 1px solid rgba(59, 130, 246, 0.35);
        border-radius: var(--radius-md);
        font-size: 0.82rem;
        font-weight: 700;
        cursor: pointer;
        display: flex;
        align-items: center;
        justify-content: center;
        gap: 8px;
        transition: all 0.2s ease;
    }

    .btn-request-loan:hover {
        background: var(--accent-primary);
        color: #FFFFFF;
        border-color: transparent;
    }

    /* Empty states */
    .empty-state-panel {
        grid-column: 1 / -1;
        background: var(--bg-surface);
        border: 1px dashed var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 48px 24px;
        text-align: center;
    }

    .empty-state-icon {
        width: 56px;
        height: 56px;
        margin: 0 auto 16px auto;
        border-radius: var(--radius-full);
        background: rgba(59, 130, 246, 0.1);
        display: flex;
        align-items: center;
        justify-content: center;
        color: var(--accent-primary);
    }

    .empty-state-title {
        font-size: 1.2rem;
        font-weight: 700;
        color: var(--text-primary);
        margin-bottom: 6px;
    }

    .empty-state-sub {
        font-size: 0.88rem;
        color: var(--text-muted);
        max-width: 440px;
        margin: 0 auto 18px auto;
    }
</style>
```

```
{% endblock %}

{% block content %}
<!-- Back to All Categories Navigation -->
<div class="back-nav-bar">
    <a href="{{ url_for('books') }}" class="back-nav-btn">
        <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.5"><line x1="19" y
        Back to All Categories
    </a>
</div>

<!-- Category Banner -->
<div class="cat-banner">
    <div class="cat-banner-info">
        <div class="cat-banner-badges">
            <span class="cat-code-pill">{{ category.section_code or 'SEC' }}</span>
            <span class="cat-total-pill" id="totalBooksInCatBadge">{{ books|length }} Title{% if books|length != 1 %}s{% endi
        </div>
        <h1 class="cat-banner-title">{{ category.name }}</h1>
    </div>

    {% if session.get('role') in ['Admin', 'Librarian'] or session.get('Role') in ['Admin', 'Librarian'] %}
    <div>
        <a href="{{ url_for('add_book') }}" class="btn btn-primary">
            <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="12"
            + Register New Book
        </a>
    </div>
    {% endif %}
</div>

{% if user_borrowing_blocked %}
<div style="background: rgba(239, 68, 68, 0.12); border: 1px solid rgba(239, 68, 68, 0.35); border-radius: var(--radius-md);
    <svg width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="#F87171" stroke-width="2" style="flex-shrink: 0;"><c
    <div>
        <strong style="color: #F87171; font-size: 0.92rem;">Borrowing Blocked:</strong>
        <span style="color: var(--text-secondary); margin-left: 6px; font-size: 0.88rem;">{{ block_reason }}</span>
    </div>
</div>
{% endif %}

<!-- Clean Book Search Toolbar (No Filters) -->
<div class="search-toolbar">
    <div class="search-input-wrapper">
        <svg viewBox="0 0 24 24"><circle cx="11" cy="11" r="8"></circle><line x1="21" y1="21" x2="16.65" y2="16.65"></line><
        <input
            type="text"
            id="bookSearchInput"
            class="search-input"
            placeholder="Search books in {{ category.name }} by title, author, or ISBN..."
            oninput="filterBooks()"
            autocomplete="off"
        >
        <button type="button" id="btnClearBookSearch" class="btn-clear-search hidden" onclick="clearBookSearch()" title="Cle
    </div>
    <div class="results-counter">
        Showing <strong id="visibleBookCount">{{ books|length }}</strong> of {{ books|length }} Books
    </div>
</div>

<!-- Books Directory Grid -->
<div class="books-grid" id="booksGrid">
    {% if books %}
        {% for book in books %}
        <div
            class="book-card"
            id="bookCard_{{ book.book_id }}"
            data-book-id="{{ book.book_id }}"
            data-title="{{ book.title|lower }}"
            data-author="{{ (book.authors or '')|lower }}"
            data-isbn="{{ book.isbn|lower }}"
        >
```

```
            <div>
                <!-- Card Header: Category & Stock Status -->
                <div class="book-meta-top">
                    <span style="font-size: 0.72rem; font-weight: 700; color: var(--accent-primary); text-transform: upperca
                        {{ category.name }}
                    </span>
                    {% if (book.available_copies or 0) > 0 %}
                        <span class="badge-stock-avail">{{ book.available_copies }} Available</span>
                    {% else %}
                        <span class="badge-stock-out">Out of Stock</span>
                    {% endif %}
                </div>

                <!-- Book Title & Authors -->
                <h3 class="book-title">{{ book.title }}</h3>
                <div class="book-authors">
                    <svg viewBox="0 0 24 24"><path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2"></path><circle cx="12" cy="
                    <span>{{ book.authors or 'Author Unspecified' }}</span>
                </div>
            </div>

            <div>
                <!-- Metadata Attributes -->
                <div class="book-details-list">
                    <div class="book-detail-row">
                        <span>ISBN:</span>
                        <strong style="font-family: var(--font-mono); font-size: 0.76rem;">{{ book.isbn }}</strong>
                    </div>
                    <div class="book-detail-row">
                        <span>Publisher:</span>
                        <strong>{{ book.publisher_name or 'N/A' }}</strong>
                    </div>
                    <div class="book-detail-row">
                        <span>Publication Year:</span>
                        <strong>{{ book.publication_year or 'Unknown' }}</strong>
                    </div>
                    <div class="book-detail-row">
                        <span>Available / Total Copies:</span>
                        <strong>{{ book.available_copies or 0 }} / {{ book.total_copies or 0 }}</strong>
                    </div>
                </div>

                <!-- Admin & Librarian Physical Copy Inventory Adjuster with Numerical Quantity Input -->
                {% if session.get('role') in ['Admin', 'Librarian'] or session.get('Role') in ['Admin', 'Librarian'] %}
                <div class="inventory-adjust-box">
                    <div class="inventory-adjust-top">
                        <span class="inventory-adjust-label">Modify Copies:</span>
                        <span class="inventory-current-stock">{{ book.available_copies or 0 }} avail / {{ book.total_copies
                    </div>
                    <form method="POST" action="{{ url_for('adjust_inventory', book_id=book.book_id) }}" class="adjust-inven
                        <input type="hidden" name="category_id" value="{{ category.category_id }}">
                        <div class="inventory-type-controls">
                            <div class="qty-stepper-wrapper">
                                <button type="button" class="btn-stepper-step" onclick="stepQty(this, -1)" title="Decrease q
                                <input
                                    type="number"
                                    name="quantity"
                                    class="qty-type-input"
                                    value="1"
                                    min="1"
                                    max="999"
                                    step="1"
                                    placeholder="Qty"
                                    inputmode="numeric"
                                    pattern="[0-9]+"
                                    oninput="this.value = this.value.replace(/[^0-9]/g, ''); if(this.value === '' || parseIn
                                    onkeydown="handleQtyKey(event, this)"
                                    title="Enter number of books (Type with keyboard or click +/- with mouse)"
                                    required
                                >
                                <button type="button" class="btn-stepper-step" onclick="stepQty(this, 1)" title="Increase qua
                            </div>
```

```
                            <button
                                type="submit"
                                name="action"
                                value="decrease"
                                class="btn-qty-action btn-qty-decrease"
                                title="Decrease stock by entered quantity (Mouse or Keyboard)"
                                {% if (book.available_copies or 0) <= 0 %}disabled style="opacity: 0.35; cursor: not-allowed
                                onclick="return confirm('Decrease physical copies for \'{{ book.title|escape }}\' by ' + this
                            >
                                <svg width="12" height="12" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
                                - Decrease
                            </button>
                            <button
                                type="submit"
                                name="action"
                                value="increase"
                                class="btn-qty-action btn-qty-increase"
                                title="Increase stock by entered quantity (Mouse or Keyboard)"
                                onclick="return confirm('Add ' + this.form.quantity.value + ' new physical copy/copies for \
                            >
                                <svg width="12" height="12" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
                                + Increase
                            </button>
                        </div>
                    </form>
                </div>
                {% endif %}

                <!-- Student & Staff Borrowing Requests -->
                {% if session.get('role') in ['Student', 'Staff'] or session.get('Role') in ['Student', 'Staff'] %}
                <div class="patron-action-box">
                    {% if user_borrowing_blocked %}
                        <div style="text-align: center; padding: 7px 10px; background: rgba(239, 68, 68, 0.12); color: #FCA5
                            ■■ Borrowing Blocked
                        </div>
                    {% elif book.book_id in user_active_loans %}
                        <div style="text-align: center; padding: 7px 10px; background: rgba(59, 130, 246, 0.12); color: #93C
                            ■ Already Borrowed
                        </div>
                    {% elif book.book_id in user_requests %}
                        <div style="text-align: center; padding: 7px; background: rgba(245, 158, 11, 0.12); color: #FBBF24;
                            ✓ Request Pending Approval
                        </div>
                    {% elif (book.available_copies or 0) > 0 %}
                        <form method="POST" action="{{ url_for('request_loan', book_id=book.book_id) }}" style="margin: 0;">
                            <input type="hidden" name="category_id" value="{{ category.category_id }}">
                            <button type="submit" class="btn-request-loan" onclick="return confirm('Submit loan request for
                                <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
                                Request Physical Loan
                            </button>
                        </form>
                    {% else %}
                        <div style="text-align: center; padding: 7px; background: rgba(239, 68, 68, 0.1); color: #F87171; bo
                            Currently Out of Stock
                        </div>
                    {% endif %}
                </div>
                {% endif %}
            </div>
        </div>
        {% endfor %}
    {% else %}
        <div class="empty-state-panel">
            <div class="empty-state-icon">
                <svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="
            </div>
            <h3 class="empty-state-title">No Books in this Category</h3>
            <p class="empty-state-sub">There are currently no book titles registered under {{ category.name }}.</p>
            <a href="{{ url_for('books') }}" class="btn btn-secondary">Return to All Categories</a>
        </div>
    {% endif %}

```

```
    <!-- Filter No-Match State -->
    <div class="empty-state-panel hidden" id="noBookMatch">
        <div class="empty-state-icon">
            <svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="1
        </div>
        <h3 class="empty-state-title">No Books Match Search</h3>
        <p class="empty-state-sub">We couldn't find any books in this category matching your search terms.</p>
        <button type="button" class="btn btn-secondary" onclick="clearBookSearch()">Clear Search</button>
    </div>
</div>
{% endblock %}

{% block extra_js %}
<script>
    function filterBooks() {
        const searchInput = document.getElementById('bookSearchInput');
        const query = (searchInput ? searchInput.value : '').trim().toLowerCase();
        const clearBtn = document.getElementById('btnClearBookSearch');
        if (clearBtn) {
            clearBtn.classList.toggle('hidden', query.length === 0);
        }

        const cards = document.querySelectorAll('.book-card');
        let visibleCount = 0;

        cards.forEach(card => {
            const title = card.getAttribute('data-title') || '';
            const author = card.getAttribute('data-author') || '';
            const isbn = card.getAttribute('data-isbn') || '';

            const matches = query === '' || title.includes(query) || author.includes(query) || isbn.includes(query);
            card.classList.toggle('hidden', !matches);
            if (matches) visibleCount++;
        });

        const counter = document.getElementById('visibleBookCount');
        if (counter) {
            counter.textContent = visibleCount;
        }

        const noMatch = document.getElementById('noBookMatch');
        if (noMatch) {
            noMatch.classList.toggle('hidden', visibleCount > 0);
        }
    }

    function clearBookSearch() {
        const input = document.getElementById('bookSearchInput');
        if (input) {
            input.value = '';
            filterBooks();
            input.focus();
        }
    }

    /* Mouse & Keyboard Inventory Quantity Helpers */
    function stepQty(btn, delta) {
        const wrapper = btn.closest('.qty-stepper-wrapper');
        if (!wrapper) return;
        const input = wrapper.querySelector('.qty-type-input');
        if (input) {
            let val = parseInt(input.value, 10);
            if (isNaN(val) || val < 1) val = 1;
            val = Math.max(1, Math.min(999, val + delta));
            input.value = val;
        }
    }

    function handleQtyKey(event, input) {
        if (event.key === 'ArrowUp') {
            event.preventDefault();
            let val = parseInt(input.value, 10);
```

```
            if (isNaN(val) || val < 1) val = 1;
            input.value = Math.min(999, val + 1);
        } else if (event.key === 'ArrowDown') {
            event.preventDefault();
            let val = parseInt(input.value, 10);
            if (isNaN(val) || val < 1) val = 1;
            input.value = Math.max(1, val - 1);
        }
    }
</script>
{% endblock %}
```

templates/loan_requests.html

HTML - Loan Requests Queue

```
{% extends "base.html" %}

{% block title %}Loan Requests Queue | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}Loan Requests Queue{% endblock %}

{% block extra_css %}
<style>
    .queue-header {
        display: flex;
        align-items: center;
        justify-content: space-between;
        margin-bottom: 28px;
        flex-wrap: wrap;
        gap: 16px;
    }

    .queue-title {
        font-size: 1.6rem;
        font-weight: 800;
        letter-spacing: -0.03em;
        color: var(--text-primary);
    }

    .queue-sub {
        font-size: 0.88rem;
        color: var(--text-secondary);
        margin-top: 4px;
    }

    .btn-approve {
        display: inline-flex;
        align-items: center;
        gap: 6px;
        background: rgba(16, 185, 129, 0.15);
        color: #34D399;
        border: 1px solid rgba(16, 185, 129, 0.35);
        border-radius: var(--radius-sm);
        padding: 6px 14px;
        font-size: 0.8rem;
        font-weight: 700;
        cursor: pointer;
        transition: all 0.15s ease;
    }

    .btn-approve:hover {
        background: #059669;
        color: #FFFFFF;
        border-color: #059669;
        box-shadow: 0 2px 8px rgba(5, 150, 105, 0.35);
        transform: translateY(-1px);
    }

    .btn-reject {
        display: inline-flex;
        align-items: center;
        gap: 6px;
        background: rgba(239, 68, 68, 0.12);
        color: #F87171;
        border: 1px solid rgba(239, 68, 68, 0.3);
        border-radius: var(--radius-sm);
        padding: 6px 14px;
        font-size: 0.8rem;
        font-weight: 700;
        cursor: pointer;
        transition: all 0.15s ease;
    }

    .btn-reject:hover {
        background: #DC2626;
```

```
        color: #FFFFFF;
        border-color: #DC2626;
        box-shadow: 0 2px 8px rgba(220, 38, 38, 0.35);
        transform: translateY(-1px);
    }
</style>
{% endblock %}

{% block content %}
<div class="queue-header">
    <div>
        <h1 class="queue-title">Borrowing Permission Requests</h1>
        <p class="queue-sub">Review patron loan requests submitted by students and staff, and assign available inventory cop
    </div>
    <div>
        <span class="badge badge-info" style="font-size: 0.85rem; padding: 6px 14px;">
            {{ requests|length if requests else 0 }} Pending Review
        </span>
    </div>
</div>

<div class="data-panel">
    <div class="panel-header">
        <span class="panel-title">Pending Authorization Queue</span>
        <span style="font-size: 0.78rem; color: var(--text-muted);">
            Librarian & Admin Dispatch Desk
        </span>
    </div>

    <div class="table-container">
        <table class="data-table">
            <thead>
                <tr>
                    <th>Request ID</th>
                    <th>Borrower Name & Role</th>
                    <th>Book Details</th>
                    <th>Available Copies</th>
                    <th>Request Timestamp</th>
                    <th style="text-align: right;">Authorization</th>
                </tr>
            </thead>
            <tbody>
                {% if requests %}
                    {% for req in requests %}
                    <tr>
                        <td style="font-family: var(--font-mono); font-size: 0.82rem; font-weight: 600;">
                            #REQ-{{ req.request_id }}
                        </td>
                        <td>
                            <div style="font-weight: 600; color: var(--text-primary);">{{ req.borrower_name }}</div>
                            <div style="font-size: 0.76rem; color: var(--text-muted); display: flex; align-items: center; ga
                                <span>{{ req.borrower_email }}</span>
                                {% if req.borrower_role == 'Staff' %}
                                    <span class="badge" style="background: rgba(168, 85, 247, 0.15); color: #C084FC; font-si
                                {% else %}
                                    <span class="badge" style="background: rgba(59, 130, 246, 0.15); color: #60A5FA; font-si
                                {% endif %}
                            </div>
                            <div style="margin-top: 4px; display: flex; flex-wrap: wrap; gap: 4px;">
                                {% if (req.borrower_active_loans or 0) >= 3 %}
                                    <span class="badge badge-danger" style="font-size: 0.68rem; padding: 2px 7px;">
                                        Quota Full ({{ req.borrower_active_loans }}/3 Active Loans)
                                    </span>
                                {% else %}
                                    <span class="badge" style="background: rgba(255, 255, 255, 0.06); color: var(--text-seco
                                        Active Loans: {{ req.borrower_active_loans or 0 }} / 3
                                    </span>
                                {% endif %}
                                {% if (req.borrower_overdue_loans or 0) > 0 %}
                                    <span class="badge badge-danger" style="font-size: 0.68rem; padding: 2px 7px;">
                                        {{ req.borrower_overdue_loans }} Overdue Loan(s)
                                    </span>
```

```
                                {% endif %}
                                {% if (req.borrower_unpaid_fines or 0) > 0 %}
                                    <span class="badge badge-danger" style="font-size: 0.68rem; padding: 2px 7px;">
                                        Unpaid Fine: Rs. {{ "%.2f"|format(req.borrower_unpaid_fines) }}
                                    </span>
                                {% endif %}
                            </div>
                        </td>
                        <td>
                            <div class="book-title-cell">{{ req.book_title }}</div>
                            <div style="font-size: 0.76rem; font-family: var(--font-mono); color: var(--text-muted); margin-
                                ISBN: {{ req.isbn }}
                            </div>
                        </td>
                        <td>
                            {% if req.available_copies_count > 0 %}
                                <span class="badge badge-success">{{ req.available_copies_count }} In Stock</span>
                            {% else %}
                                <span class="badge badge-danger">0 Copies Available</span>
                            {% endif %}
                        </td>
                        <td style="font-size: 0.82rem;">
                            {{ req.request_date.strftime('%b %d, %Y %H:%M') if req.request_date else 'N/A' }}
                        </td>
                        <td style="text-align: right;">
                            <div style="display: flex; gap: 8px; justify-content: flex-end; align-items: center;">
                                <!-- Approve & Issue Button -->
                                <form method="POST" action="{{ url_for('loan_requests') }}" style="display: inline;">
                                    <input type="hidden" name="request_id" value="{{ req.request_id }}">
                                    <input type="hidden" name="action" value="approve">
                                    <button
                                        type="submit"
                                        class="btn-approve"
                                        {% if req.available_copies_count <= 0 or (req.borrower_active_loans or 0) >= 3 or (re
                                            disabled
                                            style="opacity: 0.45; cursor: not-allowed;"
                                            {% if (req.borrower_unpaid_fines or 0) > 0 %}
                                                title="Cannot approve: Patron has unpaid fines of Rs. {{ '%.2f'|format(req.bo
                                            {% elif (req.borrower_overdue_loans or 0) > 0 %}
                                                title="Cannot approve: Patron has {{ req.borrower_overdue_loans }} overdue lo
                                            {% elif (req.borrower_active_loans or 0) >= 3 %}
                                                title="Cannot approve: Patron reached maximum quota of 3 active loans. They m
                                            {% else %}
                                                title="Cannot approve: No physical copies available in inventory"
                                            {% endif %}
                                        {% endif %}
                                    >
                                        <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" st
                                        Approve & Assign Copy
                                    </button>
                                </form>

                                <!-- Reject Button -->
                                <form method="POST" action="{{ url_for('loan_requests') }}" style="display: inline;" onsubmi
                                    <input type="hidden" name="request_id" value="{{ req.request_id }}">
                                    <input type="hidden" name="action" value="reject">
                                    <button type="submit" class="btn-reject" title="Reject Request">
                                        <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" st
                                        Reject
                                    </button>
                                </form>
                            </div>
                        </td>
                    </tr>
                    {% endfor %}
                {% else %}
                    <tr>
                        <td colspan="6" class="empty-state">
                            <svg viewBox="0 0 24 24"><path d="M22 11.08V12a10 10 0 1 1-5.93-9.14"></path><polyline points="2
                            <div style="font-size: 1.05rem; font-weight: 700; color: var(--text-primary);">All Clear! No Pend
                            <p style="margin-top: 4px;">Patron loan permission requests will appear here when submitted by St
                        </td>
```

```
                    </tr>
                {% endif %}
            </tbody>
        </table>
    </div>
</div>
{% endblock %}
```

templates/issue_loan.html

HTML - Issue Loan

```
{% extends "base.html" %}

{% block title %}Issue Book Loan | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}Issue Loan{% endblock %}

{% block content %}
<div style="max-width: 680px; margin: 0 auto;">
    <div style="margin-bottom: 28px;">
        <h1 style="font-size: 1.6rem; font-weight: 800; letter-spacing: -0.03em;">Issue Book Loan</h1>
        <p style="font-size: 0.88rem; color: var(--text-secondary); margin-top: 4px;">Assign an available physical copy to a
    </div>

    <div class="login-card" style="max-width: 100%; padding: 32px; background: var(--bg-surface); border: 1px solid var(--bo
        <form method="POST" action="{{ url_for('issue_loan') }}">
            <div class="form-group" style="margin-bottom: 20px;">
                <label class="form-label" for="copy_id">Select Available Physical Copy *</label>
                <select name="copy_id" id="copy_id" class="form-input" required style="cursor: pointer;">
                    <option value="">-- Choose an Available Copy --</option>
                    {% for copy in copies %}
                        <option value="{{ copy.copy_id }}">
                            Copy #{{ copy.copy_id }} &mdash; {{ copy.title }} ({{ copy.condition }})
                        </option>
                    {% endfor %}
                </select>
            </div>

            <div class="form-group" style="margin-bottom: 20px;">
                <label class="form-label" for="borrower_id">Select Patron (Student / Staff) *</label>
                <select name="borrower_id" id="borrower_id" class="form-input" required style="cursor: pointer;">
                    <option value="">-- Choose Patron --</option>
                    {% for borrower in borrowers %}
                        <option value="{{ borrower.user_id }}">
                            {{ borrower.username }} ({{ borrower.role }}) &mdash; {{ borrower.email }}
                        </option>
                    {% endfor %}
                </select>
            </div>

            <div class="form-group" style="margin-bottom: 28px;">
                <label class="form-label" for="due_in_days">Loan Duration (Days)</label>
                <input type="number" id="due_in_days" name="due_in_days" class="form-input" value="14" min="1" max="60" requ
                <small style="color: var(--text-muted); font-size: 0.75rem; margin-top: 4px; display: block;">Standard polic
            </div>

            <div style="display: flex; gap: 12px; justify-content: flex-end;">
                <a href="{{ url_for('dashboard') }}" class="btn btn-secondary">Cancel</a>
                <button type="submit" class="btn btn-primary">Confirm & Issue Book</button>
            </div>
        </form>
    </div>
</div>
{% endblock %}
```

templates/my_fines.html

HTML - My Fines

```
{% extends "base.html" %}

{% block title %}My Fines | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}My Fines & Penalties{% endblock %}

{% block content %}
<div style="margin-bottom: 28px;">
    <h1 style="font-size: 1.6rem; font-weight: 800; letter-spacing: -0.03em;">My Financial Statements</h1>
    <p style="font-size: 0.88rem; color: var(--text-secondary); margin-top: 4px;">Review assessed overdue fines and payment
</div>

<div class="data-panel" style="background: var(--bg-surface); border: 1px solid var(--border-subtle); border-radius: var(--r
    <div style="padding: 18px 24px; border-bottom: 1px solid var(--border-subtle); display: flex; justify-content: space-betw
        <span style="font-weight: 700; font-size: 0.95rem;">Fine Ledger Records</span>
        <span class="badge badge-info">{{ fines|length if fines else 0 }} Incurred</span>
    </div>

    <div class="table-container" style="overflow-x: auto;">
        <table class="data-table" style="width: 100%; border-collapse: collapse; text-align: left; font-size: 0.88rem;">
            <thead>
                <tr style="background: rgba(255, 255, 255, 0.02); color: var(--text-muted); font-size: 0.72rem; text-transfo
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Fine ID</th>
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Book Title</th>
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Loan Ref</th>
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Due Date</th>
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Return Date</th>
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Fine Amount</th>
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Status</th>
                </tr>
            </thead>
            <tbody>
                {% if fines %}
                    {% for fine in fines %}
                    <tr style="border-bottom: 1px solid var(--border-subtle);">
                        <td style="padding: 16px 24px; font-family: var(--font-mono); font-size: 0.8rem;">#FN-{{ fine.fine_i
                        <td style="padding: 16px 24px; color: var(--text-primary); font-weight: 600;">{{ fine.title }}</td>
                        <td style="padding: 16px 24px; font-family: var(--font-mono); font-size: 0.8rem;">#LN-{{ fine.loan_i
                        <td style="padding: 16px 24px; color: var(--text-secondary);">{{ fine.due_date.strftime('%b %d, %Y')
                        <td style="padding: 16px 24px; color: var(--text-secondary);">{{ fine.return_date.strftime('%b %d, %
                        <td style="padding: 16px 24px; font-weight: 700; color: var(--danger-text);">Rs. {{ "%.2f"|format(fi
                        <td style="padding: 16px 24px;">
                            {% if fine.payment_status == 'Paid' %}
                                <span class="badge badge-success">Paid</span>
                            {% elif fine.payment_status == 'Waived' %}
                                <span class="badge badge-info">Waived</span>
                            {% else %}
                                <span class="badge badge-danger">Unpaid</span>
                            {% endif %}
                        </td>
                    </tr>
                    {% endfor %}
                {% else %}
                    <tr>
                        <td colspan="7" style="padding: 48px 24px; text-align: center; color: var(--text-muted);">
                            <div style="font-size: 1rem; font-weight: 600; color: var(--text-primary);">Zero Fine</div>
                            <p style="margin-top: 4px;">You have no disciplinary fines recorded on your account.</p>
                        </td>
                    </tr>
                {% endif %}
            </tbody>
        </table>
    </div>
</div>
{% endblock %}
```

templates/manage_users.html

HTML - Manage Users

```
{% extends "base.html" %}

{% block title %}User Management | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}User Management{% endblock %}

{% block extra_css %}
<style>
    .users-header {
        display: flex;
        align-items: center;
        justify-content: space-between;
        margin-bottom: 28px;
        flex-wrap: wrap;
        gap: 16px;
    }

    .users-title {
        font-size: 1.6rem;
        font-weight: 800;
        letter-spacing: -0.03em;
        color: var(--text-primary);
    }

    .users-sub {
        font-size: 0.88rem;
        color: var(--text-secondary);
        margin-top: 4px;
    }

    .form-panel {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 24px;
        margin-bottom: 32px;
    }

    .form-grid {
        display: grid;
        grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
        gap: 16px;
        align-items: flex-end;
    }

    .status-btn {
        background: none;
        border: 1px solid;
        border-radius: var(--radius-sm);
        padding: 4px 10px;
        font-size: 0.75rem;
        font-weight: 600;
        cursor: pointer;
        transition: all 0.15s ease;
    }

    .status-btn.suspend {
        color: var(--danger-text);
        border-color: var(--danger-border);
        background: var(--danger-bg);
    }

    .status-btn.suspend:hover {
        background: rgba(239, 68, 68, 0.25);
    }

    .status-btn.activate {
        color: var(--success-text);
        border-color: var(--success-border);
        background: var(--success-bg);
```

```
    }

    .status-btn.activate:hover {
        background: rgba(16, 185, 129, 0.25);
    }

    .btn-delete-account {
        display: inline-flex;
        align-items: center;
        gap: 6px;
        background: rgba(239, 68, 68, 0.14);
        color: #F87171;
        border: 1px solid rgba(239, 68, 68, 0.35);
        border-radius: var(--radius-sm);
        padding: 5px 12px;
        font-size: 0.78rem;
        font-weight: 700;
        cursor: pointer;
        transition: all 0.15s ease;
    }

    .btn-delete-account:hover {
        background: #DC2626;
        color: #FFFFFF;
        border-color: #DC2626;
        box-shadow: 0 2px 8px rgba(220, 38, 38, 0.4);
        transform: translateY(-1px);
    }
</style>
{% endblock %}

{% block content %}
<div class="users-header">
    <div>
        <h1 class="users-title">User Account Administration</h1>
        <p class="users-sub">Manage institutional patrons, role-based authorization, and account statuses</p>
    </div>
</div>

<!-- Provision New User Form -->
<div class="form-panel">
    <h2 style="font-size: 1.05rem; font-weight: 700; margin-bottom: 16px; color: var(--text-primary); display: flex; align-i
        <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="var(--accent-primary)" stroke-width="2"><path d=
        Provision New System User
    </h2>

    <!-- Policy info banner -->
    <div style="background: rgba(30, 41, 59, 0.6); border: 1px solid rgba(59, 130, 246, 0.25); border-radius: var(--radius-m
        <div>
            <strong style="color: #93C5FD;">Credential Requirements:</strong>
            Email must contain <strong>'@'</strong> and <strong>'.'</strong>.
            Password must be <strong>5 to 13 characters</strong> with at least <strong>1 uppercase</strong>, <strong>1 lower
        </div>
    </div>

    <form method="POST" action="{{ url_for('manage_users') }}">
        <input type="hidden" name="action" value="add_user">
        <div class="form-grid">
            <div class="form-group" style="margin-bottom: 0;">
                <label class="form-label" for="username">Username *</label>
                <input type="text" id="username" name="username" class="form-input" placeholder="e.g. jdoe" required>
            </div>

            <div class="form-group" style="margin-bottom: 0;">
                <label class="form-label" for="email">Institutional Email *</label>
                <input type="email" id="email" name="email" class="form-input" placeholder="jdoe@vit.edu" required>
            </div>

            <div class="form-group" style="margin-bottom: 0;">
                <label class="form-label" for="password">Initial Password *</label>
                <input type="password" id="password" name="password" class="form-input" placeholder="••••••••••••" minlength
            </div>
```

```
142
143             <div class="form-group" style="margin-bottom: 0;">
144                 <label class="form-label" for="role">Assign Role *</label>
145                 <select id="role" name="role" class="form-input" style="cursor: pointer;">
146                     <option value="Student">Student</option>
147                     <option value="Staff">Staff</option>
148                     <option value="Librarian">Librarian</option>
149                     <option value="Admin">Admin</option>
150                 </select>
151             </div>
152
153             <div>
154                 <button type="submit" class="btn btn-primary" style="width: 100%; padding: 11px;">
155                     + Create User
156                 </button>
157             </div>
158         </div>
159     </form>
160 </div>
161
162 <!-- Registered Users Data Table -->
163 <div class="data-panel">
164     <div class="panel-header">
165         <span class="panel-title">Active System Users</span>
166         <span class="badge badge-info">{{ users|length if users else 0 }} Registered</span>
167     </div>
168
169     <div class="table-container">
170         <table class="data-table">
171             <thead>
172                 <tr>
173                     <th>User ID</th>
174                     <th>Username</th>
175                     <th>Email Address</th>
176                     <th>Role</th>
177                     <th>Status</th>
178                     <th style="text-align: right;">Action</th>
179                 </tr>
180             </thead>
181             <tbody>
182                 {% if users %}
183                     {% for u in users %}
184                     <tr>
185                         <td style="font-family: var(--font-mono); font-size: 0.8rem;">#USR-{{ u.user_id }}</td>
186                         <td style="color: var(--text-primary); font-weight: 600;">{{ u.username }}</td>
187                         <td>{{ u.email }}</td>
188                         <td>
189                             {% if u.role == 'Admin' %}
190                                 <span class="badge badge-info">Admin</span>
191                             {% elif u.role == 'Librarian' %}
192                                 <span class="badge badge-warning">Librarian</span>
193                             {% elif u.role == 'Staff' %}
194                                 <span class="badge" style="background: rgba(168, 85, 247, 0.12); color: #C084FC; border: 1px
195                             {% else %}
196                                 <span class="badge" style="background: rgba(255, 255, 255, 0.05); color: var(--text-secondary
197                             {% endif %}
198                         </td>
199                         <td>
200                             {% if u.account_status == 'Active' %}
201                                 <span class="badge badge-success">Active</span>
202                             {% else %}
203                                 <span class="badge badge-danger">Suspended</span>
204                             {% endif %}
205                         </td>
206                         <td style="text-align: right;">
207                             <div style="display: flex; gap: 8px; justify-content: flex-end; align-items: center;">
208                                 <!-- Toggle Status Form -->
209                                 <form method="POST" action="{{ url_for('manage_users') }}" style="display: inline;">
210                                     <input type="hidden" name="action" value="toggle_status">
211                                     <input type="hidden" name="user_id" value="{{ u.user_id }}">
212                                     <input type="hidden" name="current_status" value="{{ u.account_status }}">
213                                     {% if u.account_status == 'Active' %}
```

```
                                    <button type="submit" class="status-btn suspend" title="Suspend account access">
                                        Suspend
                                    </button>
                                {% else %}
                                    <button type="submit" class="status-btn activate" title="Reactivate account">
                                        Activate
                                    </button>
                                {% endif %}
                            </form>

                            <!-- Highly Visible Styled Delete Account Button (Librarian Protected) -->
                            {% if u.role == 'Librarian' %}
                                <span class="badge badge-warning" title="Librarian account is protected by system policy
                                    Protected (Librarian)
                                </span>
                            {% else %}
                            <form method="POST" action="{{ url_for('delete_user', user_id=u.user_id) }}" style="display:
                                <button type="submit" class="btn-delete-account" title="Permanently delete user account"
                                    <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" st
                                    Delete Account
                                </button>
                            </form>
                            {% endif %}
                        </div>
                    </td>
                </tr>
                {% endfor %}
            {% else %}
                <tr>
                    <td colspan="6" class="empty-state">No users registered in database.</td>
                </tr>
            {% endif %}
        </tbody>
    </table>
</div>
</div>
{% endblock %}
```

templates/add_book.html

HTML - Add Book

```
{% extends "base.html" %}

{% block title %}Add New Book | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}Add Book{% endblock %}

{% block extra_css %}
<style>
    .form-section {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 28px;
        margin-bottom: 24px;
    }

    .section-headline {
        font-size: 0.95rem;
        font-weight: 700;
        text-transform: uppercase;
        letter-spacing: 0.06em;
        color: var(--accent-primary);
        margin-bottom: 18px;
        display: flex;
        align-items: center;
        gap: 8px;
    }

    .form-group {
        margin-bottom: 18px;
    }

    .form-label {
        display: block;
        font-size: 0.82rem;
        font-weight: 600;
        color: var(--text-secondary);
        margin-bottom: 6px;
    }

    .form-input {
        width: 100%;
        padding: 10px 14px;
        background: var(--bg-base);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-md);
        color: var(--text-primary);
        font-size: 0.88rem;
        font-family: inherit;
        transition: all 0.15s ease;
    }

    .form-input:focus {
        outline: none;
        border-color: var(--accent-primary);
        box-shadow: 0 0 0 3px var(--accent-glow);
    }

    .sub-box {
        background: rgba(255, 255, 255, 0.02);
        border: 1px dashed var(--border-subtle);
        border-radius: var(--radius-md);
        padding: 14px 16px;
        margin-top: 10px;
    }

    .sub-box-title {
        font-size: 0.76rem;
        font-weight: 700;
        color: var(--text-muted);
```

```
        text-transform: uppercase;
        letter-spacing: 0.05em;
        margin-bottom: 10px;
        display: flex;
        align-items: center;
        gap: 6px;
    }

    .hint-text {
        font-size: 0.75rem;
        color: var(--text-muted);
        margin-top: 4px;
    }
</style>
{% endblock %}

{% block content %}
<div style="max-width: 820px; margin: 0 auto;">
    <div style="margin-bottom: 28px;">
        <h1 style="font-size: 1.6rem; font-weight: 800; letter-spacing: -0.03em;">Add New Book Title</h1>
        <p style="font-size: 0.88rem; color: var(--text-secondary); margin-top: 4px;">
            Catalog new titles with automated inventory copies, or dynamically create new publishers, categories, and author
        </p>
    </div>

    <form method="POST" action="{{ url_for('add_book') }}">
        <!-- 1. Primary Metadata -->
        <div class="form-section">
            <div class="section-headline">
                <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="
                Primary Book Information
            </div>

            <div style="display: grid; grid-template-columns: 1fr 2fr; gap: 16px;">
                <div class="form-group">
                    <label class="form-label" for="isbn">ISBN (10 or 13) *</label>
                    <input type="text" id="isbn" name="isbn" class="form-input" placeholder="e.g. 978-0134685991" required>
                </div>
                <div class="form-group">
                    <label class="form-label" for="title">Book Title *</label>
                    <input type="text" id="title" name="title" class="form-input" placeholder="e.g. Database System Concepts
                </div>
            </div>

            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-bottom: 0;">
                <div class="form-group" style="margin-bottom: 0;">
                    <label class="form-label" for="publication_year">Publication Year</label>
                    <input type="number" id="publication_year" name="publication_year" class="form-input" placeholder="2024"
                </div>
                <div class="form-group" style="margin-bottom: 0;">
                    <label class="form-label" for="num_copies">Physical Copies to Generate</label>
                    <input type="number" id="num_copies" name="num_copies" class="form-input" value="1" min="1" max="50">
                    <div class="hint-text">Each copy will be assigned an automated inventory barcode.</div>
                </div>
            </div>
        </div>

        <!-- 2. Classification & Publishing (With On-the-Fly Entry) -->
        <div class="form-section">
            <div class="section-headline">
                <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle c
                Category & Publisher
            </div>

            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px;">
                <!-- Category Section -->
                <div>
                    <div class="form-group">
                        <label class="form-label" for="category_id">Select Existing Category</label>
                        <select name="category_id" id="category_id" class="form-input" style="cursor: pointer;">
                            <option value="">-- Choose Category --</option>
                            {% for cat in categories %}
```

```
                                <option value="{{ cat.category_id }}">{{ cat.name }} ({{ cat.section_code or 'General' }})</
                            {% endfor %}
                        </select>
                    </div>

                    <div class="sub-box">
                        <div class="sub-box-title">
                            <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="
                            Or Add New Category
                        </div>
                        <div style="display: flex; flex-direction: column; gap: 10px;">
                            <input type="text" name="new_category_name" class="form-input" placeholder="New category name (e
                            <input type="text" name="new_category_code" class="form-input" placeholder="Section Code (e.g. CS
                        </div>
                    </div>
                </div>

                <!-- Publisher Section -->
                <div>
                    <div class="form-group">
                        <label class="form-label" for="publisher_id">Select Existing Publisher</label>
                        <select name="publisher_id" id="publisher_id" class="form-input" style="cursor: pointer;">
                            <option value="">-- Choose Publisher --</option>
                            {% for pub in publishers %}
                                <option value="{{ pub.publisher_id }}">{{ pub.name }}</option>
                            {% endfor %}
                        </select>
                    </div>

                    <div class="sub-box">
                        <div class="sub-box-title">
                            <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="
                            Or Add New Publisher
                        </div>
                        <div style="display: flex; flex-direction: column; gap: 10px;">
                            <input type="text" name="new_publisher_name" class="form-input" placeholder="New publisher name
                            <input type="email" name="new_publisher_email" class="form-input" placeholder="Contact email (op
                        </div>
                    </div>
                </div>
            </div>
        </div>

        <!-- 3. Author Attribution (With On-the-Fly Entry) -->
        <div class="form-section">
            <div class="section-headline">
                <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M
                Author Attribution
            </div>

            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px;">
                <div>
                    <div class="form-group">
                        <label class="form-label" for="author_ids">Select Existing Authors (Hold Ctrl/Cmd to select multiple
                        <select name="author_ids" id="author_ids" class="form-input" multiple style="height: 125px; cursor: p
                            {% for auth in authors %}
                                <option value="{{ auth.author_id }}">{{ auth.full_name }}</option>
                            {% endfor %}
                        </select>
                    </div>
                </div>

                <div>
                    <div class="sub-box" style="margin-top: 0; height: 100%; display: flex; flex-direction: column; justify-c
                        <div class="sub-box-title">
                            <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="
                            Or Add New Author
                        </div>
                        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 10px; margin-bottom: 8px;">
                            <input type="text" name="new_author_first" class="form-input" placeholder="First Name (e.g. Alan
                            <input type="text" name="new_author_last" class="form-input" placeholder="Last Name (e.g. Turing
                        </div>
```

```
                        <div class="hint-text">If specified, the new author will be registered and linked automatically.</div
                    </div>
                </div>
            </div>
        </div>

        <div style="display: flex; gap: 14px; justify-content: flex-end; margin-bottom: 40px;">
            <a href="{{ url_for('books') }}" class="btn btn-secondary">Cancel</a>
            <button type="submit" class="btn btn-primary" style="padding: 12px 24px;">
                <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M
                Save & Register Book
            </button>
        </div>
    </form>
</div>
{% endblock %}
```

templates/change_password.html

HTML - Change Password

```
{% extends "base.html" %}

{% block title %}Change Password | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}Security & Credentials{% endblock %}

{% block extra_css %}
<style>
    .security-container {
        display: flex;
        justify-content: center;
        align-items: flex-start;
        padding: 40px 16px;
    }

    .security-card {
        width: 100%;
        max-width: 520px;
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 36px;
        box-shadow: 0 20px 45px -15px rgba(0, 0, 0, 0.75);
    }

    .security-header {
        text-align: center;
        margin-bottom: 28px;
    }

    .security-icon {
        width: 48px;
        height: 48px;
        border-radius: var(--radius-full);
        background: rgba(59, 130, 246, 0.15);
        color: var(--accent-primary);
        display: flex;
        align-items: center;
        justify-content: center;
        margin: 0 auto 14px auto;
    }

    .security-title {
        font-size: 1.45rem;
        font-weight: 800;
        letter-spacing: -0.03em;
        color: var(--text-primary);
        margin-bottom: 6px;
    }

    .security-subtitle {
        font-size: 0.85rem;
        color: var(--text-secondary);
        line-height: 1.5;
    }

    .rules-card {
        background: rgba(30, 41, 59, 0.6);
        border: 1px solid rgba(59, 130, 246, 0.3);
        border-radius: var(--radius-md);
        padding: 14px 16px;
        margin-bottom: 24px;
        font-size: 0.8rem;
    }

    .rules-header {
        font-weight: 700;
        color: #93C5FD;
        display: flex;
        align-items: center;
```

```
        gap: 6px;
        margin-bottom: 8px;
    }

    .rules-list {
        margin: 0;
        padding-left: 18px;
        color: var(--text-secondary);
        line-height: 1.6;
    }

    .rules-list strong {
        color: var(--text-primary);
    }

    .form-group {
        margin-bottom: 20px;
    }

    .form-label {
        display: block;
        font-size: 0.82rem;
        font-weight: 600;
        color: var(--text-secondary);
        margin-bottom: 8px;
    }

    .input-wrapper {
        position: relative;
        display: flex;
        align-items: center;
    }

    .input-icon {
        position: absolute;
        left: 14px;
        width: 18px;
        height: 18px;
        stroke: var(--text-muted);
        stroke-width: 2;
        fill: none;
        pointer-events: none;
    }

    .form-input {
        width: 100%;
        padding: 12px 14px 12px 42px;
        background: rgba(11, 15, 25, 0.65);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-md);
        color: var(--text-primary);
        font-size: 0.92rem;
        font-family: inherit;
        transition: all 0.15s ease-in-out;
    }

    .form-input:focus {
        outline: none;
        border-color: var(--accent-primary);
        box-shadow: 0 0 0 3px var(--accent-glow);
        background: rgba(11, 15, 25, 0.95);
    }

    .btn-submit {
        width: 100%;
        padding: 12px;
        font-size: 0.92rem;
        font-weight: 700;
        margin-top: 6px;
    }

    .back-link {
```

```
        display: block;
        text-align: center;
        margin-top: 18px;
        font-size: 0.85rem;
        color: var(--text-secondary);
        text-decoration: none;
    }

    .back-link:hover {
        color: var(--accent-primary);
        text-decoration: underline;
    }
</style>
{% endblock %}

{% block content %}
<div class="security-container">
    <div class="security-card">
        <div class="security-header">
            <div class="security-icon">
                <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><rect x="
            </div>
            <h1 class="security-title">Change Password</h1>
            <p class="security-subtitle">Update your account credentials to keep your VITCC LIBRARY profile secure.</p>
        </div>

        <!-- Password Requirements Specifications -->
        <div class="rules-card">
            <div class="rules-header">
                <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.5"><circle
                Password Complexity Requirements:
            </div>
            <ul class="rules-list">
                <li>Password length: <strong>5 to 13 characters</strong></li>
                <li>At least <strong>1 uppercase</strong> letter (A-Z)</li>
                <li>At least <strong>1 lowercase</strong> letter (a-z)</li>
                <li>At least <strong>1 numeric</strong> digit (0-9)</li>
                <li>At least <strong>1 special character</strong> (e.g. ! @ # $ % ^ & * _ -)</li>
            </ul>
        </div>

        <form method="POST" action="{{ url_for('change_password') }}">
            <div class="form-group">
                <label class="form-label" for="current_password">Current Password *</label>
                <div class="input-wrapper">
                    <svg class="input-icon" viewBox="0 0 24 24"><rect x="3" y="11" width="18" height="11" rx="2" ry="2"></re
                    <input
                        type="password"
                        id="current_password"
                        name="current_password"
                        class="form-input"
                        placeholder="Enter existing password"
                        required
                        autofocus
                    >
                </div>
            </div>

            <div class="form-group">
                <label class="form-label" for="new_password">New Password *</label>
                <div class="input-wrapper">
                    <svg class="input-icon" viewBox="0 0 24 24"><path d="M21 2l-2 2m-6 6l7-7M3 10a7 7 0 1 0 14 0 7 7 0 1 0-14
                    <input
                        type="password"
                        id="new_password"
                        name="new_password"
                        class="form-input"
                        placeholder="Enter new strong password (5-13 chars)"
                        minlength="5"
                        maxlength="13"
                        required
                    >
```

```
                </div>
            </div>

            <div class="form-group">
                <label class="form-label" for="confirm_password">Confirm New Password *</label>
                <div class="input-wrapper">
                    <svg class="input-icon" viewBox="0 0 24 24"><polyline points="20 6 9 17 4 12"></polyline></svg>
                    <input
                        type="password"
                        id="confirm_password"
                        name="confirm_password"
                        class="form-input"
                        placeholder="Re-enter new password"
                        minlength="5"
                        maxlength="13"
                        required
                    >
                </div>
            </div>

            <button type="submit" class="btn btn-primary btn-submit">
                Update Password &rarr;
            </button>

            <a href="{{ url_for('dashboard') }}" class="back-link">
                &larr; Return to Dashboard
            </a>
        </form>
    </div>
</div>
{% endblock %}
```

templates/unauthorized.html

HTML - Unauthorized

```
{% extends "base.html" %}

{% block title %}403 Access Denied | VITCC LIBRARY{% endblock %}

{% block content %}
<div style="display: flex; align-items: center; justify-content: center; min-height: 60vh;">
    <div style="text-align: center; max-width: 480px; padding: 40px; background: var(--bg-surface); border: 1px solid var(--
        <div style="width: 56px; height: 56px; border-radius: 50%; background: var(--danger-bg); color: var(--danger-text); 
            <svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="1
        </div>
        <h1 style="font-size: 1.5rem; font-weight: 800; margin-bottom: 8px;">Access Denied (403)</h1>
        <p style="color: var(--text-secondary); font-size: 0.9rem; margin-bottom: 24px;">
            Your current account role (<strong>{{ user_role }}</strong>) does not have sufficient administrative privileges 
        </p>
        <a href="{{ url_for('dashboard') }}" class="btn btn-primary">Return to Dashboard</a>
    </div>
</div>
{% endblock %}
```

templates/books.html

HTML - Books Redirect

```
1  {% extends "catalog.html" %}
```

templates/dashboard_admin.html

HTML - Dashboard Admin

```
1  {% extends "dashboard.html" %}
```

templates/dashboard_librarian.html

HTML - Dashboard Librarian

```
1  {% extends "dashboard.html" %}
```

templates/dashboard_staff.html

HTML - Dashboard Staff

```
1  {% extends "dashboard.html" %}
```

templates/dashboard_student.html

HTML - Dashboard Student

```
1  {% extends "dashboard.html" %}
```